SLAM Lab: 수학에서 오도메트리와 SLAM까지

문서 버전: v2.0.0
최종 갱신: 2026-11-09
고정 경로: docs/report.md → docs/report.pdf
통합 기준 코드: 681582f (origin/main)

저장소: SeunghwanByun/slam-lab
이번 교재 확장의 기준 코드: 178ec59724069e60b991f1f4b967b1d63fc3617c. 수학 기초 12개 단원, 신규 그림 15개, 연습문제와 해설 24개를 추가했다. 기존 VO/LO 알고리즘은 변경하지 않았다. 기초 계산 실험과 기존 알고리즘 정확도 실험을 구분한다.

현재 로컬 작업 경로: `/Users/youth/Project/slam-lab`. v1.1.1에서 폴더 이름과 두 가상환경의 실행 경로를 맞췄다. VSCode에서는 이 폴더를 다시 열어 사용한다. 데이터 복사나 알고리즘 변경은 하지 않았다. 아래 과거 실험의 경로 기록은 역사적 출처로 보존한다.

읽는 순서와 문서 정책

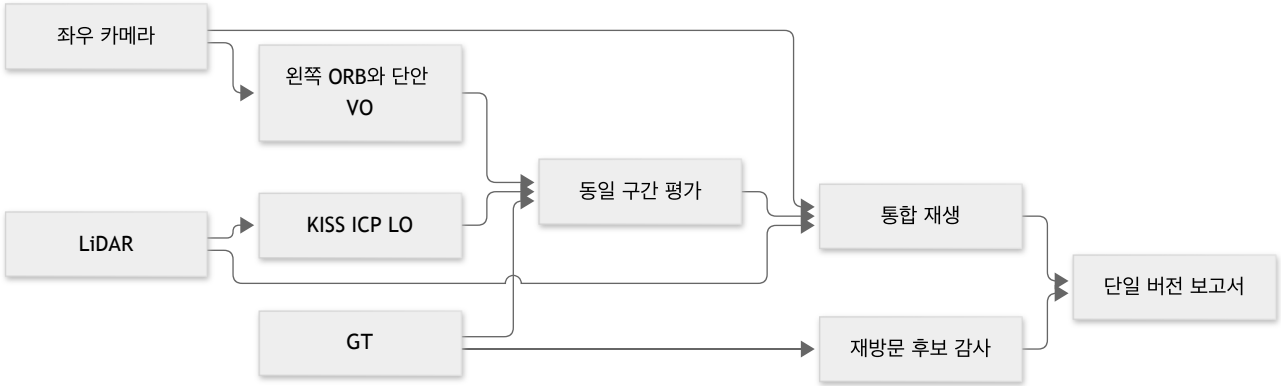
이 파일이 유일한 최신 보고서 원본이다. PDF는 같은 원본에서 생성한다. 이후 설명 보완·실험 결과·그림 수정은 이 문서 안에서 갱신하고 마지막 변경 이력의 버전을 올린다. 날짜나 버전이 붙은 새 report 파일은 만들지 않는다. 변경 전 내용은 Git 이력으로 확인한다.

기존 날짜별 연구 디렉터리는 코드·설정·실험 산출물과 과거 보고서 보관소다. 오래된 PDF를 최신 설명으로 사용하지 않는다. 자료를 합치면서 ORB의 구형 자동 확대 피라미드 설명은 상세 교정본으로 대체했고, 과거 감사는 역사적 기록으로 표시했다. 기존 파일을 삭제하거나 실행 경로를 이동하지 않았다.

- 1. 현재 상태
- 학습 범위와 필요한 수학
- 기초 교재: 문제에서 수학으로, 12개 단원
- 2. ORB 상세 학습
- 3. VO 기하·LiDAR Odometry·성능 비교
- 4. SLAM·재방문 후보
- 5. 통합 재생
- 6. 초기 감사와 IMU 기록
- 7. 재현 및 문서 갱신
- 8. 버전 변경 이력

1. 현재 상태

- 입력: KITTI Odometry grayscale stereo, 00 LiDAR, GT와 calibration. Raw OXTS도 있으나 시간 정렬은 검증되지 않았다.
- VO: ORB→BF Hamming cross-check→Essential RANSAC→recoverPose. 단안 이동 방향을 누적하며 metric scale 복원은 없다.
- LO: KISS-ICP 기반 첫 200프레임 연구 구현. 전역 loop detection·pose graph 최적화는 미구현이다.
- 화면: 좌우 원본, 왼쪽 ORB·시간 매칭, LiDAR 투영·BEV, GT/VO/LO를 동기 재생한다.
- 평가: GT 사후 Sim(3) 형상 비교와 미터 단위 LO 평가를 구분한다. 좋은 모양이 실제 VO 거리 정확도를 뜻하지 않는다.



오른쪽 카메라는 현재 재생용이고 VO 입력에는 쓰지 않는다. GT는 평가와 후보 감사에 사용하며, 자동 루프 검출·보정은 이 흐름에 구현되어 있지 않다.

이 문서의 버전과 알고리즘 성능 버전은 다르다. v1.0.0 통합 자체는 새 정확도 실험이나 알고리즘 변경이 아니다. 주제별 과거 실험에서 사용하는 상대 경로 'src/', 'artifacts/', 'configs/'는 각 장에 명시한 원 실험 디렉터리를 기준으로 읽는다. 쉘 실행 명령은 별도 지시가 없으면 레포 루트 기준이다.

1.1 SLAM, 오도메트리, 센서 융합의 관계

SLAM은 **Simultaneous Localization and Mapping**, 즉 움직이는 주체의 상태와 주변 지도를 함께 추정하는 문제다. pose는 위치와 방향을 묶은 말이다. 상태(state)는 시스템에 따라 pose 외에 속도, 센서 bias 등을 포함한다. bias는 측정에 지속해서 더해지는 편향이다. 지도는 랜드마크 점, 점군, 표면, 격자 등으로 표현할 수 있다. Cadena 외, SLAM 개관

오도메트리는 연속 관측으로 상대 이동을 추정하고 이를 이어 붙이는 접근이다. VO는 카메라, LO는 LiDAR를 사용한다. SLAM 시스템은 오도메트리로 얻은 인접 pose 사이 제약을 사용할 수 있다. 하지만 독립적인 오도메트리는 그 자체로 유용하며, 누적 궤적이나 임시 local map이 있다고 항상 완전한 SLAM이라고 부르지는 않는다.

센서 융합은 여러 센서의 관측을 하나의 상태 추정에 함께 사용하는 방법이다. 카메라·IMU로 pose를 추정하는 VIO(Visual-Inertial Odometry), LiDAR·IMU를 사용하는 LIO(LiDAR-Inertial Odometry)는 대표적인 융합 예다. 이들이 반드시 전역 지도·루프 보정을 갖추는 것은 아니다. 반대로 카메라 하나로도 SLAM을 구성할 수 있으므로 다중 센서가 SLAM의 필수 조건은 아니다.

이 레포에서 같은 화면에 VO와 LO를 비교하는 것은 **비교**이지 **융합**이 아니다. 융합하려면 공통 상태, 관측 모델, 센서 좌표변환, 시간 대응, 불확실성 처리까지 정의해야 한다. 단순히 두 궤적을 평균 내거나 나란히 그리는 것으로 끝나지 않는다.

루프 클로징(loop closure)은 이전에 방문한 장소를 다시 관측했을 때 멀리 떨어진 시점 사이 제약을 추가하는 기능이다. 오도메트리의 누적 drift를 줄이는 데 중요하지만, 루프를 아직 만나지 않았다고 위치와 지도 추정 문제가 사라지지는 않는다. 여기서 drift는 작은 추정 오차가 시간에 따라 누적되는 현상이다.

따라서 **SLAM이 모든 추정·융합을 포함하는 보편적인 상위어는 아니지만, 이 프로젝트의 학습 목표를 묶는 이름으로는 적절하다.** 로봇의 경로 계획과 제어까지 자동으로 포함하는 이름은 아니다. 'lab'은 laboratory의 줄임말이며, 이론을 읽고 코드로 재현하고 결과를 검증하는 실험 공간이라는 현재 성격에 맞는다. 이미 완성된 SLAM 제품이라는 의미는 아니다.

1.2 지금 오도메트리에서 시작한 것은 맞는가

맞다. 지금까지는 '센서 관측이 pose 변화로 바뀌는 과정'을 다루었다. 이것은 이후 지도와 전역 최적화에 넣을 관측 제약을 이해하는 첫 발걸음이다. 모든 SLAM이 동일한 개발 순서를 강제하는 것은 아니지만, 현재 데이터와 학습 방식에서는 자연스러운 출발이다.

학습 흐름은 **전체 SLAM 구조 확인 → VO/LO 출력 관찰 → 필요한 수학 → map/최적화 → 재방문 보정 → 센서 융합**으로 둔다. 각 단계에서 다시 전체 시스템으로 돌아와 새 구성 요소가 어떤 오차를 줄이고 무엇은 해결하지 못하는지 확인한다. 아래 순서는 이 프로젝트를 위한 학습 제안이며 모든 연구자에게 유일한 순서는 아니다.

단계	먼저 던질 질문	필요한 수학	현재 자료와 연결	스스로 확인할 결과
A. 좌표와 pose	같은 점을 카메라와 LiDAR에서 어떻게 표현하나?	벡터, 행렬, 회전, 역변환	calibration, LiDAR 영상 투영	변환 후 역변환하면 원래 점으로 돌아옴
B. 두 프레임 이동	왜 한 대응만으로 이동을 알 수 없나?	최소제곱, rank, SVD, 투영 기하	ORB 매칭, E, ICP	충분한 제약과 퇴화 조건을 구분
C. 믿을 정도	왜 두 센서 결과를 똑같이 믿으면 안 되나?	확률, 분산/공분산, Bayes, MLE/MAP	VO/LO 오차, inlier	측정 단위와 불확실성으로 가중치 설명
D. 반복 보정	pose를 어떻게 조금씩 개선하나?	미분, Jacobian, Taylor, 비선형 최적화	ICP, 재투영 오차	한 번의 업데이트 전후 오차 비교
E. 지도와 전체 궤적	여러 프레임을 함께 보면 무엇이 달라지나?	희소 선형대수, factor graph, gauge	BA, pose graph, loop closure	anchor 유무와 drift 보정 이해
F. IMU 융합	카메라 사이의 고속 운동을 어떻게 연결하나?	시간 적분, 잡음 모델, SO(3)/SE(3), 필터/평활화	Raw OXTS, VIO/LIO	시간·좌표·bias를 구분하고 검증

위 '확인할 결과'는 다음 학습의 도달 기준이다. 이번 문서 갱신에서 새 실험으로 실행했다고 주장하지 않는다.

1.3 당장 필요한 선형대수와 좌표변환

벡터는 점의 좌표나 이동량을 모은 값이다. 행렬은 그 값을 다른 표현으로 바꾸거나 여러 연립식을 한꺼번에 쓰는 도구다. 점의 좌표에는 **어느 좌표계인지**를 항상 붙인다. 같은 숫자 (1,0,0)이라도 카메라에서의 전방/오른쪽과 LiDAR에서의 전방/왼쪽 규약은 다를 수 있다.

다음처럼 정의하면 변환 방향이 명확해진다.

```
p_A = R_AB * p_B + t_AB
T_AB = [ R_AB  t_AB ]
       [   0      1 ]
```

p_B는 B 좌표계에서 본 점, p_A는 A 좌표계에서 본 같은 점이다. R_AB는 B의 축 방향을 A로 바꾸는 3×3 회전, t_AB는 A에서 본 B 원점 위치다. T_AB는 점에 마지막 좌표 1을 붙여 회전과 이동을 한 행렬곱으로 처리하는 4×4 동차변환이다. R의 성분은 무차원이고, t와 p는 여기서 m 단위다.

변환 합성은 T_AC=T_AB*T_BC*다. *역변환은 R_BA=R_AB의 전치, t_BA=-R_AB의 전치*t_AB다. 전치는 행과 열을 바꾸는 연산이다. 회전행렬은 R의 전치*R=I, det(R)=1을 만족한다. 일반 행렬의 전치가 항상 역행렬인 것은 아니다.

작은 계산 예: B의 점 (1,0,0)을 z축 주위로 +90도 회전하고 A에서 (2,0,0)만큼 옮기면 (2,1,0)이 된다. '옮긴 뒤 회전'하면 다른 값이므로 곱셈 순서를 바꿀 수 없다. 이것이 pose 누적과 camera↔LiDAR 변환에서 행렬 순서를 외우기만 하면 위험한 이유다.

내적은 두 벡터의 정렬 정도와 투영, 노름은 크기, 외적은 3D에서 수직 방향을 계산하는 데 쓰인다. rank는 독립적인 제약의 수를 나타내는 선형대수 개념이다. SVD(Singular Value Decomposition, 특잇값 분해)는 A=U*S*V의 전치 형태로 행렬을 분해해, 최소제곱이나 퇴화에 가까운 방향을 살피는 도구다. 처음에는 유도 전체보다 작은 예에서 차원·곱셈 방향·실패 조건을 확인하는 것으로 시작해도 된다.

1.4 오차, 최소제곱, 미분은 왜 필요한가

실제 측정은 모두 완벽하게 일치하지 않는다. 예측한 관측 h(x)와 실제 관측 z의 차이 r(x)=z-h(x)를 residual(잔차)이라고 한다. x는 추정하려는 pose 등의 상태다. 여러 잔차의 제곱합을 작게 만드는 것이 최소제곱이다. 예를 들어 같은 길이에 대한 측정이1.0m와1.2m이고 동일한 오차 모델을 가정하면, 제곱합을 최소로 만드는 값은1.1m다.

영상 재투영 잔차는 pixel, LiDAR 점 정합 잔차는 m일 수 있다. 서로 다른 단위의 값을 그대로 합산하면 의미가 흐려진다. 측정의 공분산으로 정규화하거나 적절한 관측 모델을 사용해야 한다. residual이 작다는 사실만으로 정답이라고 할 수도 없다. 잘못된 대응이나 잘못된 calibration에 잘 맞는 해일 수 있다.

비선형 예측은 상태를 바꿀 때 변화량이 일정하지 않다. Jacobian J는 상태의 각 성분을 조금 바꿨을 때 관측 또는 residual의 각 성분이 얼마나 바뀌는지를 모은 편미분 행렬이다. 일차 Taylor 근사는 r(x+delta)≈r(x)+J*delta다. 이를 이용해 작은 업데이트 delta를 반복해서 찾는다.

Gauss-Newton은 이 국소 선형 근사를 이용하는 비선형 최소제곱 방법이고, Levenberg-Marquardt는 업데이트가 불안정할 때를 대비해 감쇠를 도입하는 방법이다. 이 이름들을 알기 전에 우선 '잔차 정의 → 단위 → Jacobian 크기 → 업데이트 전후 오차'를 계산할 수 있어야 한다. 회전의 업데이트는 보통 일반 벡터처럼 원소별 덧셈을 하지 않으며 아래 Lie group 내용과 연결된다.

1.5 확률, Bayes, MLE와 MAP

확률은 모르는 상태와 불완전한 관측을 표현한다. 평균은 중심, 분산은 한 변수의 퍼짐, 공분산은 여러 변수 사이의 함께 변하는 정도를 나타낸다. 예를 들어 위치의 x/y 오차가 연관되면 공분산 행렬의 비대각 성분을 무시할 수 없을 수 있다. 분산은 m^2 같은 제곱 단위를 가진다. 정규분포(Gaussian)는 자주 쓰는 모델이지 모든 오차가 반드시 따르는 법칙은 아니다.

likelihood p(z|x)는 '상태 x를 가정하면 지금 관측 z가 얼마나 그럴듯한가'다. prior p(x)는 관측을 보기 전의 상태 정보, posterior p(x|z)는 관측을 반영한 뒤의 상태 정보다. Bayes 관계는 p(x|z) ∝ p(z|x)*p(x)다. 비례 기호는 x에 무관한 정규화 상수를 생략했다는 뜻이다.

- **MLE(Maximum Likelihood Estimation, 최대우도추정):** 관측의 likelihood를 가장 크게 만드는 x를 고른다.
- **MAP(Maximum A Posteriori, 최대사후확률추정):** prior도 반영해 posterior를 가장 크게 만드는 x를 고른다.

독립 Gaussian 측정이며 공분산이 상태에 따라 바뀌지 않는 등의 가정 아래, 음의 log likelihood를 최소화하면 가중 최소제곱 형태가 된다. log는 곱을 합으로 바꾸고, 음수를 붙이면 최대화가 최소화로 바뀐다. MAP에는 prior에 해당하는 항이 더해진다. 조건 없이 '모든 MLE는 최소제곱'이라고 말하면 틀리다. GTSAM의 확률 제약·최적화 설명

설명용 센서 융합 예: 동일한 스칼라 위치를 독립적이고 편향 없이 관측한다고 가정하고, 센서A가10m(표준편차1m), 센서B가 12m(표준편차2m)를 준다고 하자. 표준편차는 분산의 제곱근이다. 역분산 가중치는1과1/4이므로 결과는 (10*1+12*0.25)/1.25=10.4m다. 더 불확실한 B를 덜 믿는다. 이는 실제 VO/LO fusion 구현 결과가 아니라 가정이 명확한 1차원 설명 예다.

같은 이미지나 GT를 통해 얻은 두 추정은 서로 독립이 아닐 수 있다. 그 상관을 무시하면 정보를 두 번 센 것처럼 과신할 수 있다. 따라서 궤적을 단순 가중 평균하는 것보다 먼저 상태·좌표·시간·상관 모델을 정의해야 한다. 지금 GT로 맞춘 Sim(3) 결과를 독립적인 실제 센서 측정처럼 fusion 입력에 넣어서는 안 된다.

1.6 회전, Lie group, 지도 최적화

SO(3)는 유효한 3D 회전행렬들의 집합, SE(3)는 회전과 이동을 합친 3D 강체변환들의 집합이다. 이름의 S는 special, O는 orthogonal, E는 Euclidean을 가리킨다. 여기서 group은 합성·항등변환·역변환이 정의되는 구조라는 뜻이다. 회전행렬에는9개 숫자가 있지만 자유도는3이므로9개를 독립적으로 최적화하면 회전 조건을 깨뜨릴 수 있다.

Lie algebra는 이러한 변환 근처의 작은 변화를 벡터로 다루는 도구다. exp/log는 작은 변화 벡터와 변환 사이를 연결한다. 일반 행렬의 원소별 지수/로그와 혼동하면 안 된다. 처음부터 추상대수 전체를 끝낼 필요는 없다. '유효한 회전을 유지하며 작은 업데이트를 적용한다'는 목적에서 시작한다. quaternion은 회전을 표현하는 다른 방법이며, 네 숫자를 쓰지만 단위 노름 조건이 있다.
Solà·Deray·Atchuthan, 로봇 추정용 Lie 이론

factor graph(요인 그래프)는 미지 상태 노드와 관측·사전정보 제약 노드를 연결한다. pose graph는 pose들 사이의 상대변환 제약을 중심으로 구성하고, BA(Bundle Adjustment)는 pose와3D 랜드마크를 함께 조정해 영상 재투영 오차를 줄인다. 희소성은 각 관측이 모든 상태가 아니라 일부 상태에만 연결되는 구조다. 계산 비용을 줄이는 핵심이다.

gauge freedom은 모든 위치와 지도를 함께 옮기거나 돌려도 상대 관측이 같아 절대 기준이 정해지지 않는 자유도다. 첫 pose를 고정하는 등의 anchor가 필요한 이유다. 단안의 scale 모호성과도 연결된다. 이 단계에서는 지도 전체를 바꿔도 관측이 유지되는 경우와 센서 오차로 값이 달라지는 경우를 구분해야 한다.

1.7 IMU는 어느 수학이 더 필요한가

IMU(Inertial Measurement Unit)는 각속도와 specific force를 측정한다. specific force는 단순한 세계좌표 가속도와 다르며, 자세와 중력을 고려해 해석해야 한다. 각속도를 시간 적분하면 방향 변화, 가속도를 적분하면 속도·위치 변화와 연결되지만 bias와 잡음도 함께 누적된다. 이를 위해 시간 적분, 미분방정식의 기초, 확률 과정, bias 모델을 공부한다.

필터링은 현재까지 관측으로 현재 상태를 갱신하고, 평활화(smoothing)는 여러 시점의 상태를 함께 다루는 관점이다. EKF(Extended Kalman Filter)는 비선형 모델을 현재 추정 근처에서 선형화하는 필터다. IMU preintegration은 카메라 등 저속 센서의 두 시점 사이 다수 IMU 관측을 하나의 상대 제약으로 요약하는 접근이다. 용어만 먼저 익히고 구현은 시간 동기화·좌표·단위·잡음 검증 이후로 둔다.

우리 Raw OXTS에는 비양수 시간 간격이 관측되었으며 Odometry와의 정렬이 미완료다. 수학을 안다고 이 입력 문제가 자동으로 해결되지 않는다. GPS/INS 위치·자세와 순수 IMU 측정 채널을 구분해 GT 누출도 막아야 한다.

1.8 지금 먼저 할 공부와 나중에 할 공부

바로 필요한 최소 기반: 벡터·행렬곱·역행렬·삼각함수, 좌표변환, 평균·분산, 제곱오차, 미분의 의미. 이를 현재 point projection과 두 프레임 pose에 연결한다.

그다음 연결할 기반: SVD·rank, 카메라 투영·에피폴라 기하, Jacobian, Bayes·MLE/MAP, Gaussian 가중 최소제곱. 이를 ORB 매칭 이후 E 추정과 ICP에 연결한다.

지도·융합 단계에서 깊게 볼 내용: SO(3)/SE(3), robust loss(큰 이상치의 영향을 제한하는 비용 함수), 희소 최적화·factor graph, filtering/smoothing, IMU bias·preintegration, observability(관측으로 구별 가능한 상태 성분).

첫 학습 과제는 새 알고리즘 구현이 아니라 **기존 LiDAR 점 하나를 카메라 좌표로 옮겨 영상에 투영하고, 변환 순서와 단위를 설명하는 것**으로 제안한다. 그다음 두 프레임에서 잔차가 어떻게 pose 업데이트로 이어지는지 본다. 이는 top-down 학습을 유지하면서 필요한 수학을 빠뜨리지 않는 순서다. 이번 문서 갱신에서는 이 과제를 실행하지 않았다.

이 절의 자료는 SLAM 개관, GTSAM 공식 설명, 로봇 추정용 Lie 이론 원문을 참고한 학습 설계다. 전체 내용을 한 번에 외우라는 선행 조건이 아니다. 기초→작은 계산→현재 데이터→실패 사례 순으로 같은 문서의 설명을 확장한다.

기초 교재: 문제에서 수학으로

이 책을 사용하는 방법

출발 질문은 "자동차가 어디로 얼마나 움직였는가?"다. 영상을 관측하고, 같은 지점을 찾고, 이동을 계산하고, 누적한다. 그런데 픽셀은 미터가 아니고, 센서마다 축이 다르며, 측정에는 오차가 있고, 오래 누적하면 궤적이 틀어진다. 아래 수학은 이 네 가지 문제를 해결하기 위해 등장한다. 모든 수학을 끝낸 뒤 코드를 시작하는 순서가 아니다.

처음에는 A1-A4를 읽고 실제 LiDAR 투영 그림을 재현한다. 그다음 기존 2장의 ORB 그림과 3장의 VO/LO 결과를 본다. "왜 매칭이 틀리면 궤적이 망가지는가?"가 궁금해질 때 A5-A9로 돌아온다. 마지막으로 A10-A12와 기존 4장의 재방문 결과를 연결한다. 이해가 막히면 아래 지도에서 그 문제에 필요한 단원으로 이동한다.

눈앞의 문제	필요한 단원	손으로 확인할 결과
센서 축이 서로 다르다	A1 벡터, A2 좌표변환	같은 점이 (1,0)과 (2,1)로 표현됨
이미지에서 실제 거리를 모른다	A3 투영, A4 스테레오	27px 시차가 10m 깊이로 변환됨
잘못된 매칭이 결과를 끌고 간다	A5 잔차, A6 선형대수	평균 1.6m와 robust 추정 1.05m
무엇을 더 믿어야 하나	A7 불확실성, A8 MLE/MAP	두 측정의 결합 10.4m
최적화가 어떻게 움직이나	A9 Jacobian, A10 회전	1에서 시작해 2로 수렴하는 기록
재방문·IMU가 왜 필요한가	A11 그래프, A12 IMU	루프 잔차와 bias 적분 오차

기호 약속. 소문자 x는 문맥에 따라 상태 벡터 또는 스칼라다. 굵은 글꼴에 의존하지 않고 차원을 함께 적는다. p는 공간의 점, z는 관측, r은 잔차, R은 회전행렬, t는 병진, T는 동차변환, K는 카메라 내부행렬, J는 Jacobian, Σ는 공분산이다. 위첨자 T는 전치이고, T_AB의 T는 변환 이름이다. 두 표기를 혼동하지 않는다. `||v||`는 벡터 길이, `argmin`은 비용을 가장 작게 만드는 입력값, `argmax`는 가장 크게 만드는 입력값을 뜻한다.

그림의 출처와 범위. 신규 15개 그림 중 A3의 KITTI 투영만 실제 데이터다. 나머지는 설정과 정답을 아는 교육용 모델이다. 합성 실험에서의 개선을 KITTI 성능 개선으로 해석하면 안 된다. 기존 ORB·매칭·LO 실제 산출물은 뒤의 응용 장에 그대로 남겼다. 모든 신규 그림은 `research/foundations-textbook/src/generate.py`가 만든다.

A1. 벡터: 위치를 숫자로 표현하기

문제와 입력·출력

특징점은 이미지에서 (u,v), LiDAR 점은 센서 좌표에서 (X,Y,Z)로 주어진다. 벡터는 이런 순서 있는 숫자 묶음이다. 입력이 두 위치라면 출력은 상대 변위, 거리 또는 특정 방향으로의 성분이다. 단위와 좌표계가 같을 때만 두 위치를 빼거나 더할 수 있다.

점 p=(3,4)m는 원점에 대한 위치다. 변위 v=(3,4)m는 위치 사이의 차이다. 숫자가 같아도 의미는 다르다. 원점을 옮기면 점의 좌표는 변하지만 두 점 사이의 변위는 변하지 않는다. A2에서 점에는 이동 t를 더하고 방향에는 더하지 않는 이유다.

계산을 따라가기

```
v = (3,4) m
||v|| = sqrt(3² + 4²) = 5 m
단위 방향 n = v / ||v|| = (0.6,0.8), 길이 1, 단위 없음
내적 a·b = a1*b1 + a2*b2
v·(1,0) = 3 m: x 방향 성분
a·b = ||a|| ||b|| cos(theta)
```

내적은 방향의 정렬 정도를 측정한다. 단위벡터 두 개의 내적이 1이면 같은 방향, 0이면 직각, -1이면 반대 방향이다. **투영**은 어떤 방향으로 바라본 성분이다. 평면의 단위 법선 n과 점 오차 e의 내적 n·e는 그 평면에 수직인 거리 성분이다. 이는 뒤의 point-to-plane ICP를 읽을 때 필요한 수학이다. 현재 LO에 쓰인 KISS-ICP의 실제 잔차 형식은 응용 장에서 별도로 확인하며, 여기의 평면 예제를 현재 구현이라고 주장하지 않는다.

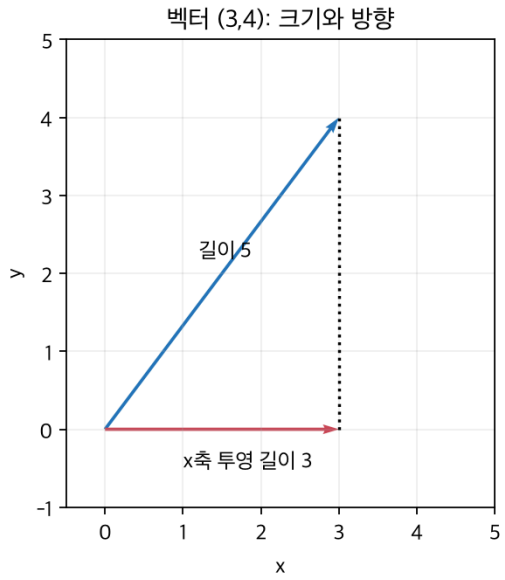


그림 A1. 파란 벡터의 길이는 5, 빨간 가로 성분은 3이다. 화살표 길이와 각도가 같은 축 비율에서 표현되어 있다.

행렬은 벡터를 다른 벡터로 바꾸는 규칙이다. 2×2 행렬 A의 첫 번째 열은 x 단위벡터가 이동한 곳, 두 번째 열은 y 단위벡터가 이동한 곳이다. 따라서 A(3,4)는 첫 번째 열의 3배와 두 번째 열의 4배를 더한 것이다. 행렬을 외우기보다 "기준 방향을 어디로 보내는가?"로 읽는다.

코드 연결과 한계

`generate.py:frames()`의 a, R, q를 `artifacts/frames/values.npz`에서 확인한다. NumPy의 `@`는 행렬곱, `*`는 원소별 곱이다. 미터 위치와 픽셀 위치의 유클리드 거리를 직접 비교하지 않는다. 방향을 만들 때 길이가 0이면 나누기를 할 수 없으므로 먼저 검사해야 한다.

연습문제와 해설

문제 A1-1. p=(1,2)m, q=(4,6)m일 때 p에서 q로 가는 변위와 거리는?

해설 A1-1. q−p=(3,4)m이고 거리는 sqrt(9+16)=5m다. p+q는 두 점 사이 변위가 아니다.

문제 A1-2. 오차 e=(3,4)m, 평면의 단위 법선 n=(0,1)일 때 수직 오차와 전체 거리는 같은가?

해설 A1-2. 수직 오차 n·e=4m, 전체 거리 ||e||=5m로 다르다. x 방향 3m는 평면에 평행하므로 수직 잔차에 나타나지 않는다. 이 정보 손실은 A6의 관측 불가능한 방향과 연결된다.

A2. 좌표변환: 같은 점을 다른 센서에서 보기

왜 필요한가

카메라는 보통 영상 오른쪽·아래쪽·전방을 축으로 쓰고, LiDAR는 다른 축을 쓴다. 축의 순서만 바꾸는 것이 아니라 센서 원점도 다르다. **외부 보정값(extrinsic calibration)**은 이 상대 회전과 병진이다. 입력은 B 좌표의 점 p_B와 B→A 변환이고, 출력은 A 좌표의 같은 물리적 점 p_A다.

```
p_A = R_AB p_B + t_AB
R_AB: 3×3, B의 벡터 성분을 A로 변환, 단위 없음
t_AB: 3×1, B 원점을 A에서 본 위치, m
T_AB = [ R_AB  t_AB ]       p_h = [X,Y,Z,1]^T
       [ 0 0 0   1  ]       p_Ah = T_AB p_Bh
```

끝에 1을 붙인 **동차좌표**를 쓰면 회전과 이동을 하나의 행렬곱으로 묶을 수 있다. 방향벡터에는 끝에 0을 붙인다. 그러면 t가 곱해지지 않는다. T_AB는 “B에서 A로”로 읽으며, 파일에서 pose를 받았을 때 이 방향부터 확인한다.

2D로 손계산하기

```
R = [0 −1]   t = (2,0)m   p_B = (1,0)m
    [1  0]
R p_B = (0,1)m
p_A = (0,1) + (2,0) = (2,1)m
R(p_B+t) = R(3,0) = (0,3)m    # 순서를 바꾸면 다른 결과
```

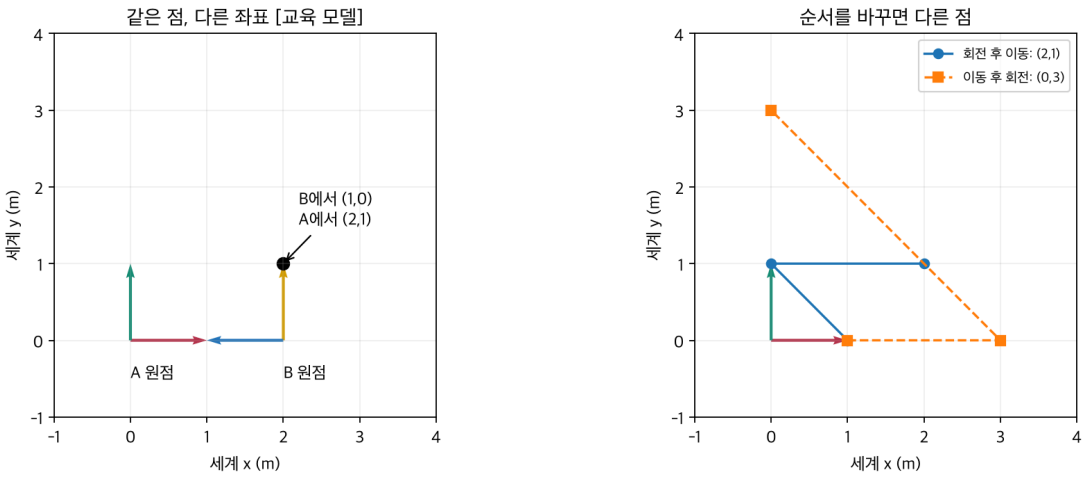


그림 A2. 왼쪽은 같은 점의 두 좌표, 오른쪽은 회전·이동 순서를 바꾸었을 때의 차이다. (2,1)과 (0,3)은 같은 변환이 아니다.

역변환은 단순히 t의 부호만 바꾸는 것이 아니다. p_B=R_AB^T(p_A−t_AB)이므로 역변환의 병진은 −R_AB^T t_AB다. 회전행렬은 열들이 서로 직교하고 길이가 1이어서 R^T R=I이며 역행렬이 전치와 같다. 합성은 T_AC=T_AB T_BC다. 오른쪽 변환을 먼저 적용한다. Modern Robotics: 동차변환

디버깅 계약과 한계

코드에서 행벡터 배열 N×3을 쓸 경우 `points @ R.T + t`가 위의 열벡터 수식과 같다. 전치가 있는 이유를 축 정의와 함께 확인한다. `frames()`는 역변환 후 원래 점 복원과 변환 순서 차이를 검사한다. 이 검사는 대수적 일관성만 보장한다. 잘못된 센서 보정값도 역변환 자체는 성립할 수 있으므로 실제 영상 overlay가 추가로 필요하다.

연습문제와 해설

문제 A2-1. 위 예제의 p_A=(2,1)를 B로 되돌려라.

해설 A2-1. p_A−t=(0,1), R^T(0,1)=(1,0)이다. `R^T p_A−t`로 계산하면 (-1,-2)가 되어 틀린다.

문제 A2-2. 센서 방향 d_B=(1,0)에 동차좌표 마지막 값을 1로 넣으면 무엇이 잘못되는가?

해설 A2-2. 올바른 방향은 R d_B=(0,1)이다. 1을 붙이면 위치 이동 t까지 더해 (2,1)이 된다. 위치와 방향의 타입을 구분해야 한다.

A3. 카메라 투영: 미터가 픽셀이 되는 과정

모델과 기호

핀홀(pinhole) 모델은 한 점인 카메라 중심을 지나가는 광선이 영상면에 닿는다고 본다. 카메라 좌표 X는 오른쪽, Y는 아래쪽, Z는 전방으로 두자. Z>0인 점에 대해 닮은 삼각형의 비율로 아래 식을 얻는다. 입력은 카메라 좌표의 3D 점과 내부 보정값, 출력은 영상의 픽셀 좌표다.

```
u = fx * X/Z + cx       v = fy * Y/Z + cy
K = [fx  0 cx]         fx,fy: 픽셀 단위 초점거리
    [ 0 fy cy]         cx,cy: 주점(principal point), px
    [ 0  0  1]         X,Y,Z: m; u,v: px
q = K [X,Y,Z]^T        u=q1/q3, v=q2/q3
```

주점은 광학 축이 영상면을 만나는 위치다. fx가 크면 같은 X/Z 비율이 더 큰 픽셀 이동이 된다. 실제 렌즈의 왜곡은 별도 모델이 필요하지만, 이번 입력은 KITTI의 보정된 영상과 제공된 투영행렬을 사용한다. 이미 보정된 영상에 왜곡 보정을 중복 적용하면 안 된다. OpenCV 카메라 모델

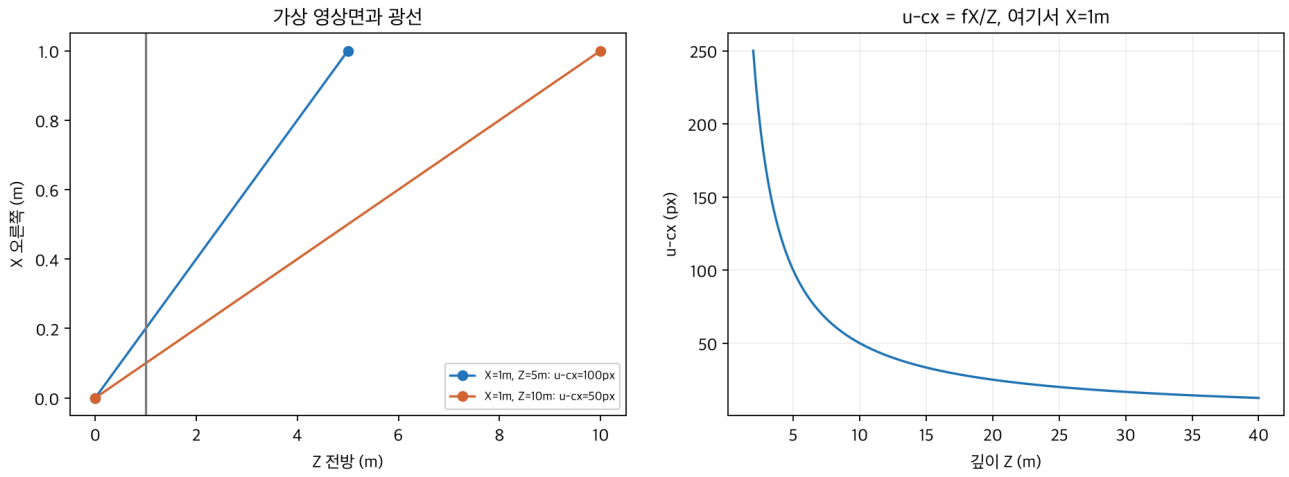


그림 A3-1. X=1m인 점도 깊이가 5m에서 10m가 되면 주점에서의 거리가 100px에서 50px로 줄어든다. 왼쪽 Z=1 선은 정규화된 가상 영상면이며 실제 초점거리 1m를 뜻하지 않는다.

단안의 scale 모호성. (X,Z)=(1,5)와 (2,10)은 X/Z가 같아 같은 픽셀이다. 한 이미지의 픽셀만으로 깊이를 결정할 수 없다. 여러 영상으로 구조와 이동을 계산하더라도 모든 거리와 이동을 같은 배율로 늘리면 같은 투영을 만들 수 있다. metric 기준이 없는 단안 VO의 이동을 미터라고 부를 수 없는 이유다.

실제 KITTI 점 하나를 끝까지 계산하기

00 시퀀스의 frame120을 사용한다. `.bin`은 little-endian float32의 (x,y,z,reflectance) 레코드이며 좌표변환에는 앞 세 값만 쓴다. reflectance는 동차좌표의 1이 아니다. `Tr`로 카메라 좌표에 옮긴 뒤 `P0`로 투영한다.

```
원시 점 index = 55553
p_L = (12.681147, −0.226356, −1.510892) m
p_C = Tr ∗ [p_L;1] = (0.232002, 1.363495, 12.399397) m
fx=fy=718.856 px, cx=607.1928 px, cy=185.2157 px
u = 718.856 ∗ 0.232002 / 12.399397 + 607.1928 = 620.6431 px
v = 718.856 ∗ 1.363495 / 12.399397 + 185.2157 = 264.2644 px
```

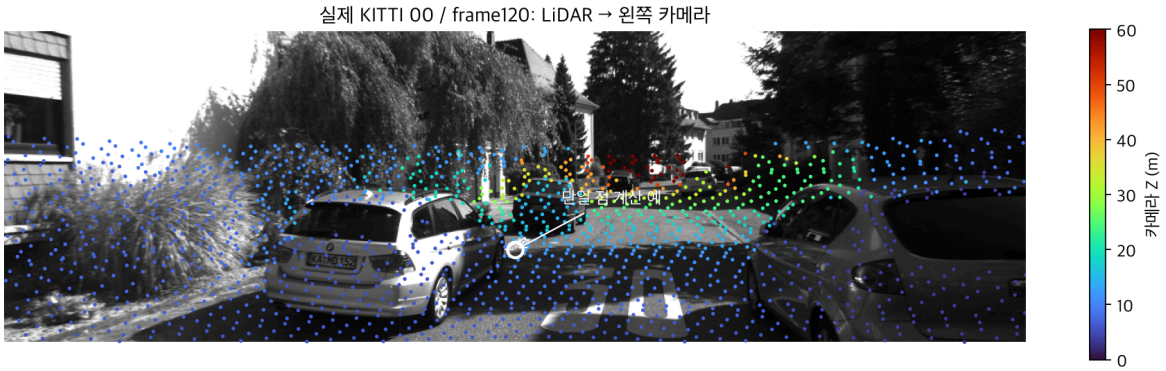


그림 A3-2. 흰 원이 계산한 단일 점이다. 색은 카메라 전방 깊이 Z이며 반사강도가 아니다. 원시 121,444점 중 전방이며 영상 경계 안인 점은 18,073개다. 보기 쉽게 그중 12개마다 하나씩 그렸다. 이는 가림까지 판정한 물리적 visibility가 아니라 투영 유효성 필터다.

`camera()`가 실제 Tr/P0, 선택 점, 전체 유효 투영을 `artifacts/kitti/values.npz`에 저장한다. 뒤쪽 점(Z≤0)은 나누기 전에 제외한다. 영상 밖 점도 제거하지만, 앞 물체에 가려진 뒤쪽 점을 제거하는 z-buffer는 여기서 구현하지 않았다. 따라서 overlay 일치만으로 시간 동기화나 모든 보정이 완벽하다고 결론내리지 않는다.

연습문제와 해설

문제 A3-1. fx=500px, cx=320px, X=1m, Z=5m라면 u는? 같은 광선 위에서 깊이를 두 배로 하면 X는?

해설 A3-1. u=500/5+320=420px. 같은 광선이면 X도 2m가 되어야 한다. X=1m를 유지한 채 Z만 두 배로 하면 u=370px로 달라진다.

문제 A3-2. LiDAR 레코드 (x,y,z,0.2)를 그대로 4×4 동차변환에 넣어도 되는가?

해설 A3-2. 안 된다. 마지막 0.2는 반사강도다. 점을 변환할 때는 (x,y,z,1)을 새로 만들어야 병진이 정확히 한 번 더해진다.

A4. 스테레오: 깊이와 거리 기준 얻기

단안에서 부족했던 정보

두 카메라의 간격 **baseline B**를 미터로 알면 길이 기준이 생긴다. 평행·정렬된 두 카메라가 같은 초점거리와 수평 주점을 가진 단순 모델에서 같은 점은 같은 영상 행에 나타난다. 왼쪽 좌표가 u_L, 오른쪽이 u_R이면 **시차(disparity)**는 d=u_L−u_R다. 입력은 대응점과 보정값이고 출력은 깊이 Z다. 주점이 다르면 그 차이를 반영해야 하며, 모든 데이터에 아래 단순식을 그대로 적용할 수는 없다.

```
u_L−cx = f X/Z
u_R−cx = f (X−B)/Z
d = u_L−u_R = f B/Z
Z = f B/d
예: f=500px, B=0.54m, d=27px → Z=10m
```

둘을 빼면 모르는 X가 사라진다. 이것이 삼각측량의 가장 간단한 모습이다. 일반적인 두 시점 카메라는 회전과 이동을 포함한 투영 행렬로 광선의 교차를 구한다. 광선이 거의 평행하면 작은 픽셀 오차가 큰 깊이 오차가 된다.

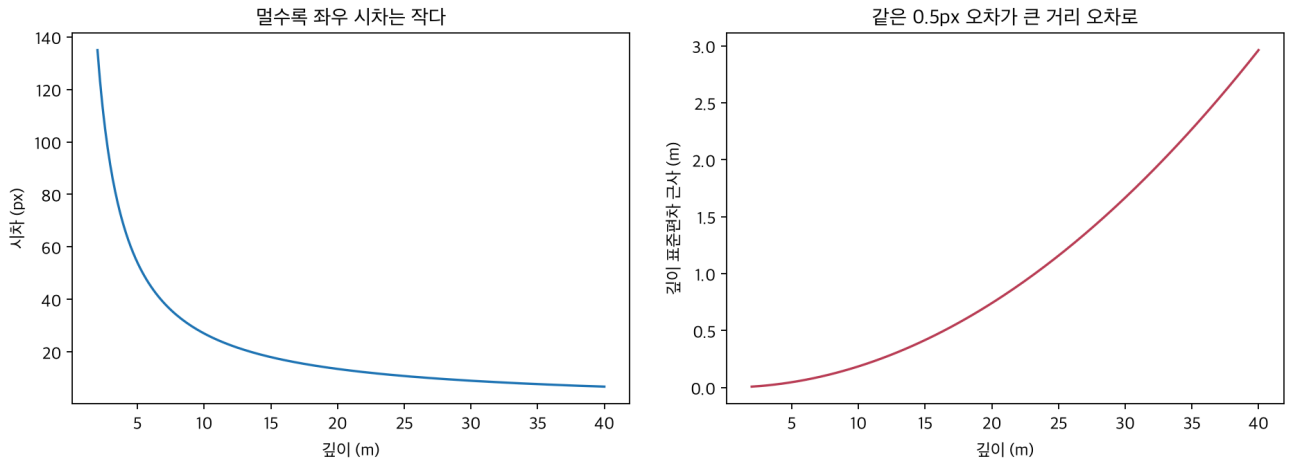


그림 A4. 왼쪽은 깊이에 반비례하는 시차, 오른쪽은 시차 표준편차 0.5px를 가정한 깊이 표준편차다. 실제 KITTI 보정값이 아니라 f=500px, B=0.54m인 교육용 설정이다.

오차를 계산하려면 함수의 기울기를 이용한다. $dZ/dd = -fB/d^2$이고 작은 시차 오차의 표준편차가 σ_d이면 $\sigma_Z \approx |dZ/dd| \sigma_d = Z^2 \sigma_d/(fB)$다. 10m에서 약 0.185m, 20m에서 약 0.741m다. 거리가 두 배면 불확실성은 이 근사에서 네 배가 된다. f와 B가 정확하고 오차가 작다는 가정이 포함된다. d가 0에 가까우면 이 선형 근사 자체도 불안정하다.

프로젝트 연결과 다음 한계

현재 VO는 왼쪽 영상 기반 단안 방식이다. 오른쪽 원본이 재생된다고 스테레오 추정을 쓰는 것은 아니다. 이 단원은 다음 구현에서 stereo matching→triangulation→3D-2D pose estimation을 도입해야 하는 이유를 설명한다. 컬러를 추가해도 baseline 같은 거리 기준이 자동으로 생기지 않는다. 반복 무늬·가림·반사·질감 없는 벽에서는 대응 자체가 틀릴 수 있다. 다음 A5가 그 잘못된 관측을 다룬다.

연습문제와 해설

문제 A4-1. 같은 f와 B에서 d=13.5px이면 깊이는? d=0이면?

해설 A4-1. Z=270/13.5=20m다. d=0에서는 유한한 깊이를 계산할 수 없다. 작은 양수로 강제로 나누어 거대한 “정답”을 만들지 말고 유효하지 않거나 먼 점으로 처리한다.

문제 A4-2. 동일 깊이에서 baseline을 두 배로 늘리고 σ_d를 유지하면 σ_Z는?

해설 A4-2. 식의 분모 B가 두 배이므로 절반이다. 그러나 실제로는 공통 시야와 가림, 대응 난이도가 변하므로 baseline을 늘리기만 하면 항상 좋아지는 것은 아니다.

A5. 잔차와 최소제곱: 무엇을 작게 만드는가

관측에서 목적함수로

상태 x로 예측한 관측을 h(x), 실제 측정을 z라 하면 **잔차(residual)**를 r(x)=h(x)−z로 정의한다. 부호를 반대로 써도 제곱 비용은 같지만 Jacobian과 업데이트는 같은 정의를 따라야 한다. 입력은 관측과 예측 모델, 출력은 잔차와 추정 상태다. 영상 재투영 잔차는 px, 공간 대응 잔차는 m일 수 있다. 단위가 다른 잔차를 그대로 더하면 큰 숫자의 단위가 결과를 지배한다.

```
상수 위치를 여러 번 측정: h(x)=x, r_i=x−z_i
F(x) = sum_i (x−z_i)²
dF/dx = 2 sum_i (x−z_i) = 0
x_hat = sum_i z_i / N
```

즉 단순 최소제곱(Least Squares, LS)은 평균을 만든다. z=[1,1.1,0.9,1,4]m이면 평균은 1.6m다. 4m가 잘못된 대응이라면 나머지 네 측정에서 멀어진다. 제곱은 큰 잔차에 매우 큰 벌점을 주므로 추정값이 이상치를 향해 움직인다.

Huber 손실의 의미

```
rho(r) = 0.5 r²                  if |r| <= delta
         delta (|r|−0.5 delta)   otherwise
```

작은 잔차에는 제곱, 큰 잔차에는 선형 벌점을 준다. **robust**란 일부 큰 오차의 영향에 덜 민감하도록 만든다는 뜻이지 오류를 모두 제거한다는 뜻은 아니다. delta=0.2m이면 이 예제의 해는 1.05m다. 그 근처에서 네 정상 측정의 기울기 합은 4x−4, 4m 관측은 기울기 −0.2를 주므로 4x−4−0.2=0, x=1.05다.

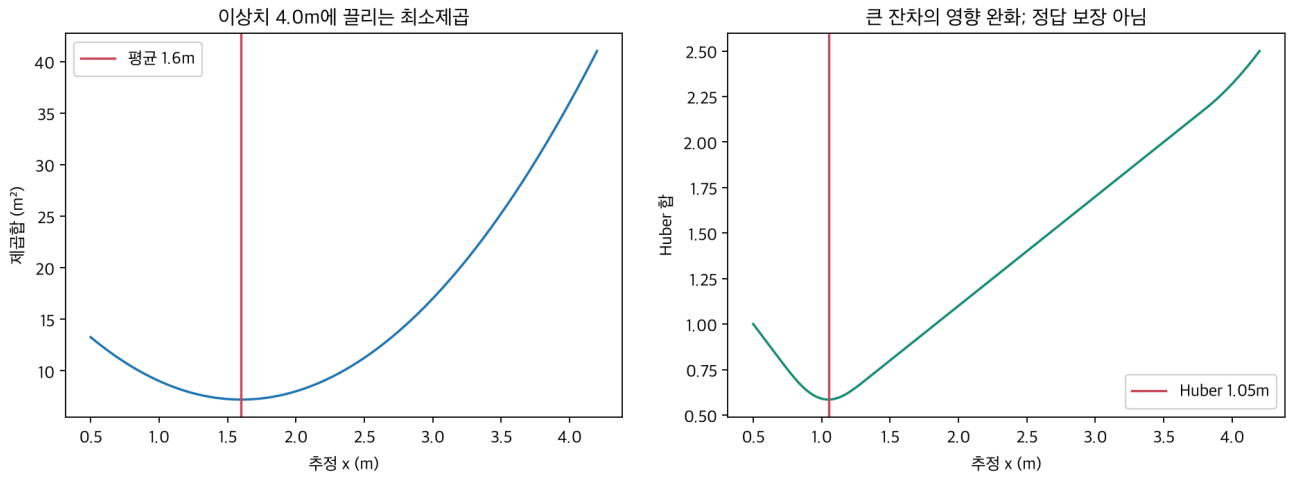


그림 A5. 같은 측정에 대한 두 목적함수와 최소점이다. 두 패널의 세로축 비용 정의가 다르므로 높이 자체를 성능으로 비교하지 않는다. 참값 1m를 가정한 교육 예제에서 추정 차이를 보여준다.

estimation()은 SciPy least_squares(loss='huber', f_scale=0.2)와 해석적 답을 비교한다. 현재 VO의 RANSAC(Random Sample Consensus, 무작위 표본 합의)은 작은 부분집합으로 모델을 만들고 그 모델과 맞는 관측을 찾는 별도 전략이다. Huber는 이미 선택한 잔차의 영향력을 줄이는 전략이다. 둘은 보완적으로 쓸 수 있지만 같은 알고리즘은 아니다. RANSAC의 threshold도 잔차의 단위와 정의에 맞춰야 한다.

연습문제와 해설

문제 A5-1. 마지막 측정 4m를 제외하면 평균은? 이것만으로 이상치를 자동 판정할 수 있는가?

해설 A5-1. 평균은 1m다. 그러나 결과를 보고 임의로 제거한 것이므로 자동 판정 근거가 아니다. 관측 모델·기하 검증·고정된 규칙과 평가가 필요하다.

문제 A5-2. Huber delta를 매우 크게 하면 어떻게 되는가?

해설 A5-2. 모든 잔차가 제곱 구간에 들어가 일반 최소제곱과 같은 최소점을 갖는다. delta가 너무 작으면 정상적인 큰 잔차까지 약하게 취급할 수 있다. 값은 노이즈 규모와 단위를 기준으로 정한다.

A6. Rank와 SVD: 왜 어떤 방향은 못 알아내는가

식이 많아도 정보는 적을 수 있다

선형 모델 A x=b에서 A의 행은 관측 조건, 열은 미지수 방향을 나타낸다. **rank(계수)**는 독립적인 열 또는 행의 수다. x1+x2=1을 백 번 반복해도 x1과 x2를 각각 결정할 수 없다. 관측 개수가 아니라 서로 다른 정보를 주는지가 중요하다.

SVD(Singular Value Decomposition, 특잇값 분해)는 A=U S V^T로 쓴다. V는 입력의 직교 방향, S의 비음수 대각값은 방향별 확대율, U는 출력 방향이다. 특잇값이 0이면 어떤 입력 변화가 출력에 전혀 나타나지 않는다. 아주 작으면 그 방향을 추정할 때 노이즈가 크게 증폭된다.

```
A = diag(2,0.2)
b = (2,0.2) → x=(1,1)
b'=(2,0.22) → x'=(1,1.1)
두 번째 관측의 0.02 변화가 상태의 0.1 변화로 증폭
condition number = 최대 특잇값 / 최소 특잇값 = 2/0.2=10
```

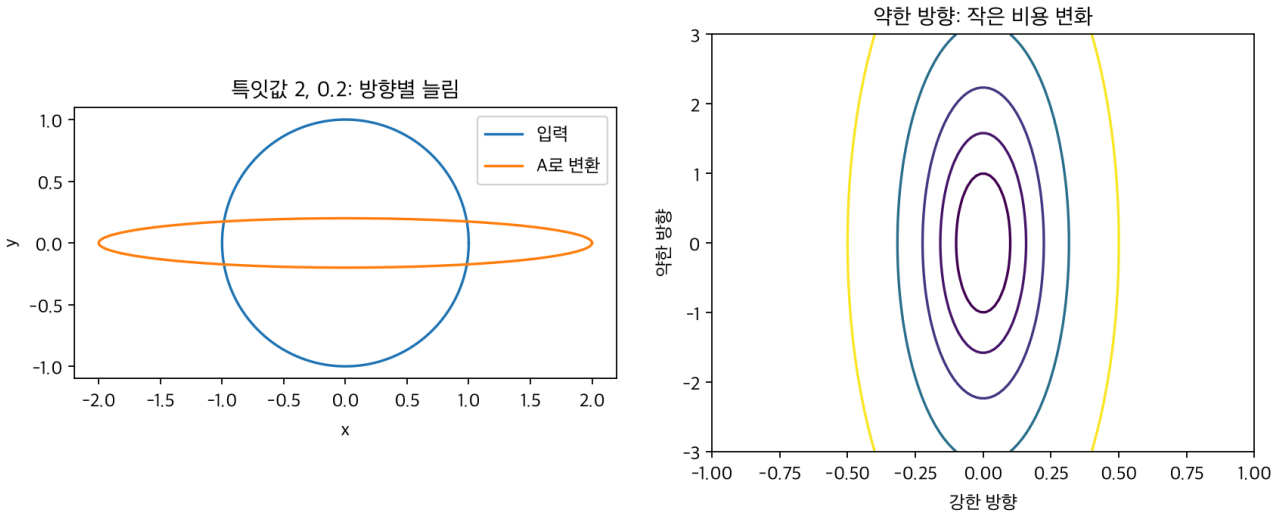


그림 A6. 왼쪽 단위원이 타원으로 바뀐다. 오른쪽 비용 등고선은 y 방향으로 길다. y를 많이 바꾸어도 비용 변화가 작으므로 그 방향의 추정이 약하다. 이 그래프는 무차원 선형 모델이다.

null space(영공간)는 A v=0을 만족하는 방향 집합이다. 벽까지 수직 거리만 잰다면 벽에 평행한 이동은 잔차를 바꾸지 않을 수 있다. 실제 3D LO의 관측 가능성은 장면 구조·움직임·잔차 모델 전체로 판단해야 하며, 이 작은 예제만으로 어느 주행 구간이 퇴화했다고 확정할 수 없다.

계산 관점의 주의

해가 불안정할 때 `inverse(A.T@A)`를 무조건 계산하지 않는다. 정규방정식은 A의 조건수를 제곱할 수 있다. QR 분해(직교행렬 Q와 상삼각행렬 R로 분해)나 SVD 기반 선형해법을 이용한다. 단, 미터와 라디안처럼 열의 단위가 섞인 행렬에서는 단순 조건수가 스케일 설정에 의존한다. 상태 정규화와 gauge를 먼저 검토한다. `estimation()`의 A와 조건수는 저장되어 있으나 대규모 SLAM의 관측성 분석을 수행한 것은 아니다.

연습문제와 해설

문제 A6-1. A=diag(2,0)이면 rank와 영공간 방향은?

해설 A6-1. rank는 1이고 v=(0,1)이 영공간 방향이다. 두 번째 상태를 아무리 바꿔도 출력은 같다. 반복 측정만 늘려서는 이 정보가 생기지 않는다.

문제 A6-2. A=diag(2,0.2)에서 A^T A의 조건수는?

해설 A6-2. 대각값이 4,0.04가 되므로 조건수는 100이다. A의 조건수 10보다 커져 수치적으로 불리해질 수 있다.

A7. 확률과 공분산: 오차에도 방향이 있다

노이즈를 숫자 하나로 표현할 수 있을까

확률변수는 값이 불확실한 양이다. 평균 μ는 중심, 분산 σ^2는 중심에서의 제곱 편차 평균, 표준편차 σ는 원래 단위의 퍼짐이다. Gaussian(정규) 분포의 1D 밀도는 다음과 같다. 밀도는 특정 점의 확률과 다르다. 연속 변수에서 구간 확률은 밀도를 적분한 값이며, 밀도 높이는 1을 넘을 수도 있다.

```
p(x) = 1/(sqrt(2*pi)*sigma) * exp(-(x-mu)²/(2*sigma²))
Cov(x,y) = E[(x-mu_x)(y-mu_y)]
Sigma = [[Var(x), Cov(x,y)], [Cov(x,y), Var(y)]]
```

E는 기대값, 즉 분포로 가중한 평균이다. 좌표가 m라면 공분산 성분 단위는 m^2다. 대각선은 각 축 분산, 비대각선은 두 축 오차가 함께 움직이는 정도다. 양의 상관이면 x 오차가 양수일 때 y 오차도 양수인 경향이 있다.

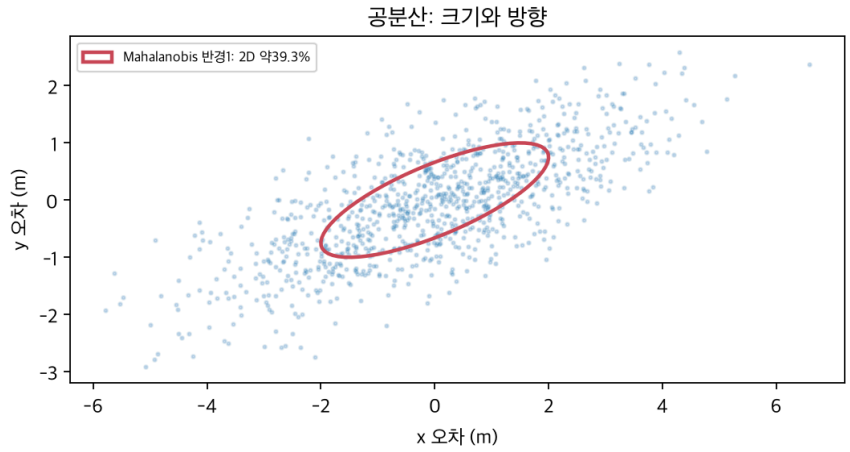


그림 A7. Σ=[[4,1.5],[1.5,1]]m^2의 Gaussian 표본 1,000개(seed=7)다. 타원은 Mahalanobis 반경 1 경계다. **2D에서 이 타원 안의 이론적 확률은 약 39.3%이며, 1D의 ±1σ 약 68.3%와 같지 않다.** 표본의 실제 포함률은 난수에 따라 달라진다.

Mahalanobis 거리는 d^2=e^T Σ^(−1)e다. 불확실한 방향의 오차는 덜, 정확한 방향의 오차는 더 크게 본다. Σ=diag(4,1)m^2에서 e=(2,0)m와 (0,1)m는 모두 d^2=1이다. 유클리드 거리는 다르지만 각 방향 표준편차로 보면 동일한 크기다. **정보행렬**은 Σ의 역행렬이며 높은 정보는 낮은 분산에 해당한다. 실제 계산은 명시적 역행렬보다 선형 시스템 풀이를 선호한다.

한계와 모델 검증

Gaussian은 계산이 편리한 모델이지 모든 센서 오차의 사실이 아니다. 가림에 의한 오대응은 평균 주변 작은 노이즈와 다르다. 공분산을 너무 작게 잡으면 틀린 관측을 과신한다. 또한 두 센서가 같은 원자료나 보정값을 공유하면 오차가 상관될 수 있다. 독립이라는 가정 없이 분산만 더하거나 정밀도를 더하면 안 된다.

연습문제와 해설

문제 A7-1. $\sigma = 0.2\,\mathrm{m}$의 분산은? 센티미터 단위로 바꾸면?

해설 A7-1. 분산은 $0.04\,\mathrm{m}^2$다. $\sigma = 20\,\mathrm{cm}$이고 분산은 $400\,\mathrm{cm}^2$다. 분산에 길이 환산계수 100만 곱하면 틀리며 100^2를 곱해야 한다.

문제 A7-2. $\Sigma = \mathrm{diag}(4,1)\,\mathrm{m}^2$, $\mathbf{e} = (2,1)\,\mathrm{m}$의 Mahalanobis 거리는?

해설 A7-2. $d^2 = 2^2/4 + 1^2/1 = 2$이므로 $d = \sqrt{2}$다. 두 성분을 표준편차로 나눈 뒤 길이를 구한 것과 같다.

A8. Bayes, MLE, MAP: 측정과 믿음을 합치기

likelihood는 상태의 확률인가

관측 z가 주어졌을 때 상태 x를 추정한다. `p(z|x)`는 x라는 상태에서 이런 관측이 나올 모델이다. 관측을 고정하고 x를 바꾸며 평가하면 **likelihood(우도)**라고 부른다. 이것만으로 x에 대한 확률분포가 되는 것은 아니다. **prior(사전분포)** p(x)는 관측 전 상태의 불확실성, **posterior(사후분포)** p(x|z)는 관측 후 상태의 분포다.

```
Bayes: p(x|z) = p(z|x) p(x) / p(z)
MLE (Maximum Likelihood Estimation, 최대우도추정): argmax_x p(z|x)
MAP (Maximum A Posteriori, 최대사후확률 추정): argmax_x p(x|z)
```

p(z)는 x와 무관한 정규화 상수라 최적점만 찾을 때 제외할 수 있다. 독립인 고정분산 Gaussian 측정이면 우도를 곱한다. 로그는 곱을 합으로 바꾸며, 음수를 붙이면 최대화가 최소화가 된다. 따라서 음의 로그우도는 상수항을 제외하고 $0.5 \sum_i r_i^2/\sigma_i^2$ 다. 최소제곱이 확률 모델과 연결되는 지점이다. 분산이 x에 따라 바뀌면 정규화항도 달라지므로 이를 무조건 버릴 수는 없다.

두 센서와 prior를 손으로 결합하기

```
z1=10m, sigma1=1m → 정밀도 w1=1
z2=12m, sigma2=2m → 정밀도 w2=0.25
MLE 평균 = (w1*z1+w2*z2)/(w1+w2) = 10.4m
측정 결합 분산 = 1/(1+0.25) = 0.8m²
prior: 평균 9m, sigma=0.5m → 정밀도 4
MAP = (10+3+36)/(1+0.25+4) = 9.333333m
사후 분산 = 1/5.25 = 0.190476m²
```

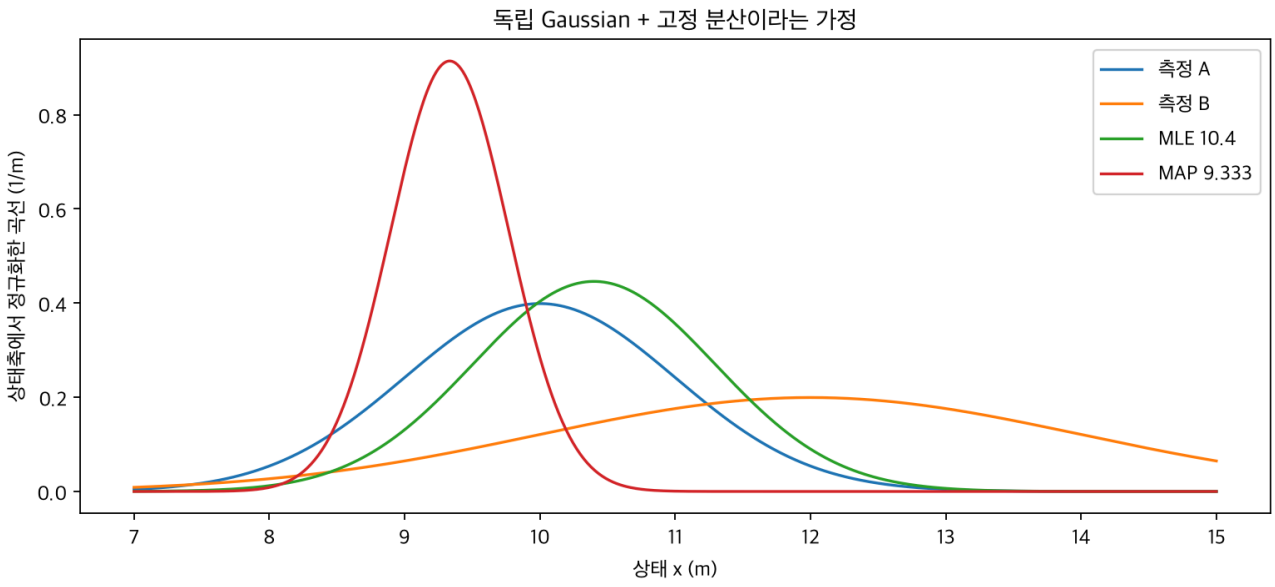


그림 A8. 상태축에서 비교하도록 정규화한 측정 곡선과 결합 결과다. MLE 곡선은 이 특별한 위치 관측 모델의 정규화된 우도이며, 모든 우도가 자동으로 상태의 확률밀도라는 뜻은 아니다. Gaussian인 이 예에서는 MAP가 사후 평균과도 일치한다.

과신의 위험. prior가 실제로 틀렸는데 σ=0.5m로 매우 정확하다고 선언하면 두 측정보다 prior 쪽으로 강하게 끌린다. MAP는 prior를 추가했으니 무조건 더 정확한 방법이 아니다. 두 센서의 노이즈가 상관되거나 bias가 있으면 위 계산의 독립·불편향 가정이 깨진다. `estimation()`은 수식과 수치가 일치하는지 검사할 뿐 실제 카메라와 IMU를 융합하지 않는다.

연습문제와 해설

문제 A8-1. 두 측정의 σ가 모두 1m이면 MLE와 결합 분산은?

해설 A8-1. 같은 가중치이므로 평균 11m, 분산 $1/(1+1)=0.5\text{m}^2$다. 두 번 독립 측정했다는 조건이 필요하다.

문제 A8-2. prior의 σ를 무한히 크게 하면 MAP는 어디로 가는가?

해설 A8-2. prior 정밀도가 0으로 가므로 이 Gaussian 모델에서 MLE=10.4m로 간다. 반대로 σ가 0에 가까워지면 prior 평균 9m로 강하게 고정된다.

A9. 미분과 Jacobian: 최적화의 한 걸음

비선형 함수를 잠시 직선으로 보기

미분은 입력을 조금 바꾸었을 때 출력이 얼마나 변하는지 나타낸다. 여러 입력과 여러 출력에 대한 미분을 모은 행렬이 **Jacobian(야코비안)**이다. 잔차가 m개, 상태가 n개이면 J는 m×n이다. `J_ij=∂r_i/∂x_j`이며 단위는 잔차 단위/상태 단위다. 카메라 투영의 X/Z 때문에 pose와 픽셀 관계는 일반적으로 비선형이다.

```
r(x+delta) ≈ r(x) + J delta
Gauss-Newton: delta = argmin ||r+J delta||²
정규방정식: J^T J delta = -J^T r
x_next = x + delta
```

Gauss-Newton(GN)은 잔차를 현재 추정 근처에서 선형화하고 이 작은 문제를 반복해서 푼다. 실제 구현은 J의 rank와 단위를 확인하고 안정적인 선형해법을 사용한다. 큰 delta는 선형 근사가 유효한 범위를 벗어날 수 있다. **Levenberg-Marquardt(LM)**는 감쇠항을 더해 과한 이동을 제어하는 접근이며, 감쇠를 어떻게 조절하는지도 중요하다. 이번 교육 코드의 단일 변수 GN에는 LM을 구현하지 않았다.

한 번의 업데이트를 계산하기

$r(x)=x^2-4$, J=2x인 무차원 예제에서 x=1이면 r=−3, J=2다. 2 `delta=3`이므로 delta=1.5, 다음 x는 2.5다. 다시 r=2.25, J=5이므로 delta=−0.45, x=2.05다. 이후 2.000609756, 2.000000093, 2.0으로 간다. x=−2도 해이며 x=0에서는 J=0이라 이 계산이 실패한다. 시작값에 관계없이 수렴한다는 예제가 아니다.

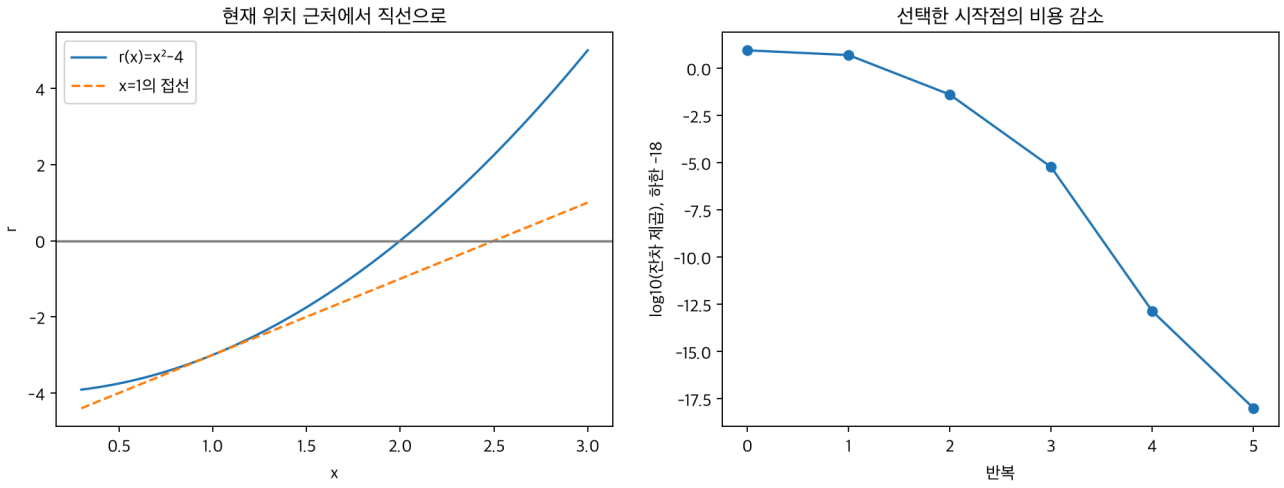


그림 A9. 왼쪽은 x=1에서의 접선, 오른쪽은 선택한 시작점에서의 비용 변화다. 작은 값의 표현을 위해 log10을 사용하고 -18에서 표시를 제한했다. 마지막 점은 실제 비용이 10^-18보다 작아도 그 아래로 그리지 않는다.

구현을 검산하는 방법

유한차분은 `J≈[r(x+ε)−r(x−ε)]/(2ε)`로 미분을 수치적으로 확인한다. `linearization()`은 ε=10^-6으로 다섯 반복의 J를 확인하고, `artifacts/linearization/values.npz`의 trace에 x,r,J,r^2를 저장한다. ε가 너무 작으면 반올림 오차, 너무 크면 곡률 오차가 생긴다. 수치 미분은 검증 도구이며 무조건 가장 정확한 기준이 아니다.

연습문제와 해설

문제 A9-1. x=3에서 이 예제의 GN 한 번을 계산하라.

해설 A9-1. r=5, J=6, delta=−5/6이므로 다음 x=13/6≈2.166667이다. 비용은 25에서 약 0.482253으로 줄지만 모든 비선형 문제에서 감소를 보장하지는 않는다.

문제 A9-2. 관측 픽셀 (u,v) 두 성분과 6차원 pose 증분을 쓴다면 점 하나의 J 크기는?

해설 A9-2. 2×6이다. 점 N개를 세로로 쌓으면 2N×6이다. 행 수가 6보다 크다는 것만으로 rank가 6이라고 보장되지는 않는다.

A10. SO(3)와 SE(3): 회전을 망가뜨리지 않고 갱신하기

아홉 숫자를 마음대로 바꾸면 안 되는 이유

3×3 회전행렬에는 아홉 숫자가 있지만 자유도는 셋이다. **자유도(Degrees of Freedom, DoF)**는 독립적으로 바꿀 수 있는 양의 수다. 회전은 `R^T R=I`, `det(R)=+1`을 만족해야 한다. determinant(행렬식)가 -1인 직교행렬은 반사를 포함하므로 올바른 3D 회전이 아니다.

SO(3)는 Special Orthogonal group in 3 dimensions, 3차원 특수직교군이다. **SE(3)**는 Special Euclidean group in 3 dimensions로 회전과 병진을 합친 강체변환 집합이다. 강체변환은 물체 내부 거리를 보존한다. "군"은 항등변환·합성·역변환이 집합 안에서 성립하는 구조를 뜻한다.

작은 회전을 세 숫자로 표현하기

회전벡터 w는 방향이 회전축, 길이가 회전각(rad)인 3차원 벡터다. `[w]x`는 임의 v에 대해 `[w]x v=w×v`가 되도록 만든 반대칭행렬이다. ×는 두 벡터에 수직인 방향을 만드는 외적이다. 행렬의 **지수함수 exp**는 원소마다 지수를 취하는 것이 아니라 `I+A+A²/2!+...`의 행렬 급수다.

```
w=(0,0,0.5) rad
[w]x = [[0,-0.5,0],[0.5,0,0],[0,0,0]]
1차 근사 R_bad=I+[w]x → (1,0,0)이 (1,0.5,0)으로, 길이 1.118034
올바른 R=exp([w]x) → (cos(0.5),sin(0.5),0), 길이 1
```

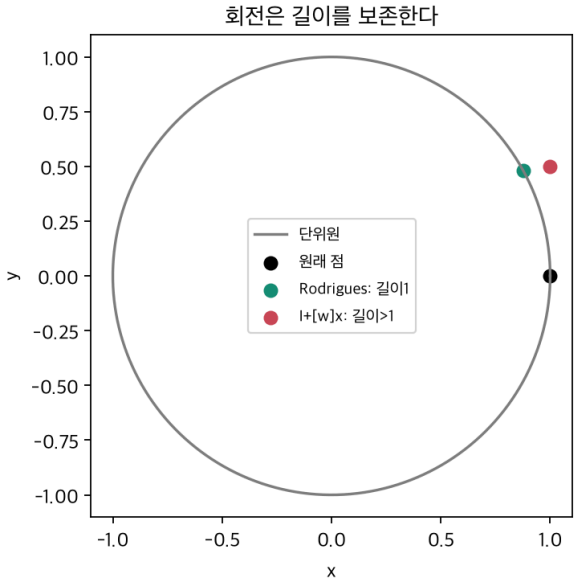


그림 A10. 초록 점은 단위원 위에 남지만 단순 1차 근사인 빨간 점은 원 밖으로 나간다. 작은 각도의 근사 자체는 유용하지만 이를 그대로 영구 회전행렬로 저장하면 제약이 깨질 수 있다.

`linearization()`은 OpenCV Rodrigues 변환을 사용해 회전벡터를 회전행렬로 바꾼다. Rodrigues 식은 이 3D 회전 지수함수를 계산하는 방법이다. 최적화에서는 `R_new=exp([delta_w]x)R`처럼 작은 증분으로 회전을 갱신할 수 있다. 오른쪽에 곱하는 `R exp(...)`도 가능하지만 증분이 표현된 좌표계와 Jacobian이 달라진다. 둘을 섞으면 안 된다.

SE(3)로 확장할 때의 주의

pose 증분 ξ는 보통 회전 3개와 병진 3개로 구성하지만 라이브러리마다 순서가 다르다. 엄밀한 SE(3) 지수지도에서는 회전과 병진 부분이 결합되므로 "회전은 exp, 병진은 그냥 더하기"가 언제나 동일한 연산은 아니다. 이번 예제는 SO(3)만 수치 검증했다. Lie algebra(리 대수)는 이런 국소 증분을 다루는 공간이며 전체 유도는 후속 학습 범위다. Solà·Deray·Atchuthan, A micro Lie theory

연습문제와 해설

문제 A10-1. diag(1,1,-1)은 R^T R=I를 만족한다. SO(3)인가?

해설 A10-1. 행렬식이 -1이므로 아니다. z축 반사가 포함된다. 직교성 검사만으로 회전 유효성을 모두 검사할 수 없다.

문제 A10-2. 90도를 그대로 회전벡터 길이에 넣으면 올바른가?

해설 A10-2. 라디안 API라면 $\pi/2 \approx 1.570796$을 넣어야 한다. 90을 넣으면 90rad다. 각도 단위 오류는 행렬의 직교성 검사로 잡히지 않는다.

A11. Factor Graph: 누적 궤적을 함께 고치기

오도메트리와 전역 수정의 차이

오도메트리는 이웃 시점의 상대 이동을 이어 붙인다. 나중에 과거 장소로 돌아왔다는 정보가 생기면 최신 위치만 바꾸는 대신 전체 상태가 관측을 함께 만족하도록 조정할 수 있다. **factor graph(인자 그래프)**에서 변수 노드는 미지 상태, factor는 일부 변수에 대한 관측 제약이다. pose graph는 pose들을 변수로 삼는 특수한 경우다. GTSAM 입문

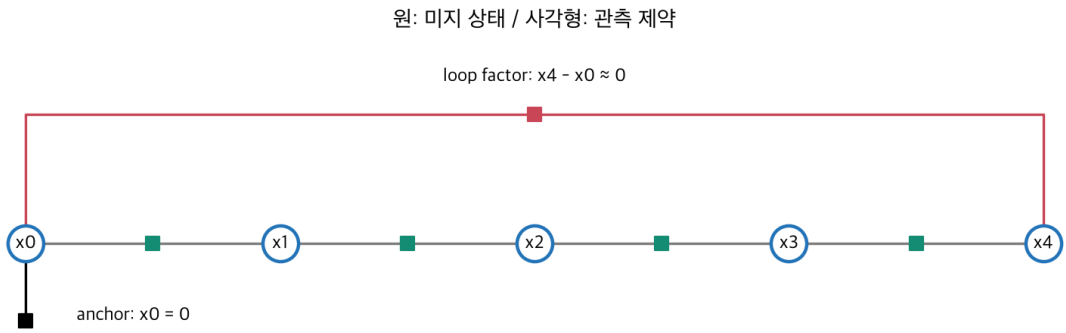


그림 A11-1. 원은 x0⋯x4, 초록 사각형은 이웃 이동 관측, 빨간 사각형은 시작과 끝의 상대 위치 관측이다. 검은 anchor는 기준 원점을 고정한다. 사각형 자체가 새 상태는 아니다.

1D 예제를 실제로 최적화하기

참고용 경로를 [0,1,2,1,0]m라 하고, 이동 관측은 [1.1,1.1,-0.9,-0.9]m로 만든다. 누적하면 [0,1.1,2.2,1.3,0.4]m다. 끝에서 시작점으로 돌아왔다는 상대 관측 0m를 추가한다. 이 loop 관측은 인식 알고리즘이 찾은 것이 아니라 교육자가 제공한 것이다.

```
x0=0 고정
odom residual_i = (x_(i+1)−x_i−z_i)/0.1
loop residual = (x4−x0−0)/0.05
최소화: 모든 정규화 잔차의 제곱합
보정 후: [0,1.005882,2.011765,1.017647,0.023529] m
```

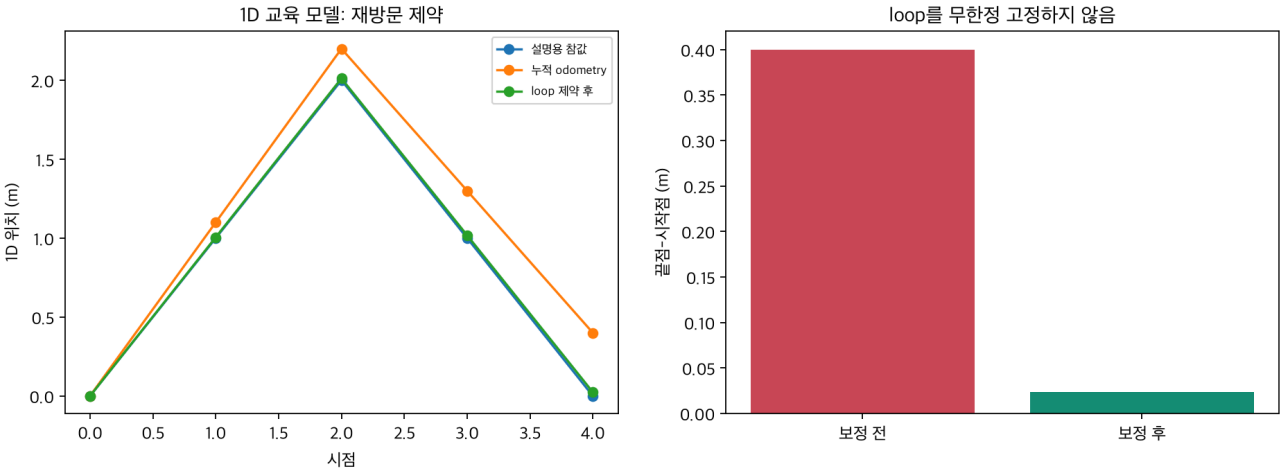


그림 A11-2. 끝점 잔차가 0.4m에서 0.023529m로 줄었다. loop 분산을 0이 아닌 값으로 설정했으므로 끝점이 강제로 정확히 0이 되지는 않는다. 교육용 1D 결과이며 실제 KITTI 루프 클로징 성능이 아니다.

gauge와 잘못된 루프

상대 이동만 있으면 모든 x에 같은 상수 c를 더해도 잔차가 같다. 이 기준의 자유를 **gauge freedom**이라고 한다. 차분 Jacobian은 4×5, rank=4이고 J(1,1,1,1,1)^T=0이다. 한 상태를 고정하거나 적절한 prior로 기준을 정해야 한다. 이것은 "센서가 나빠서 생긴 문제"와 달리 상대 측정의 구조적 성질이다.

실제 loop closing에는 장소 후보 검색, 기하 검증, 상대 pose 추정, 불확실성 설정, 그래프 최적화가 필요하다. GT 위치가 가깝다는 감사 결과는 자동 검출이 아니다. 틀린 loop는 전체 지도를 망가뜨릴 수 있다. `graph_imu()`는 알려진 제약을 SciPy로 푸는 교육 예제만 추가하며 현재 playback의 궤적을 보정하지 않는다. landmark까지 포함한 시각적 **Bundle Adjustment(BA, 묶음 조정)**는 여러 pose와 3D 점을 재투영 오차로 함께 조정하는 별도 확장이다.

연습문제와 해설

문제 A11-1. 보정 전 모든 pose에 100m를 더하면 이웃 이동 잔차는 변하는가?

해설 A11-1. `(x_(i+1)+100)-(x_i+100)`에서 100이 소거되므로 변하지 않는다. anchor를 적용한 문제에서는 x0=0 제약을 위반한다.

문제 A11-2. loop의 σ를 0.05m에서 0.5m로 늘리면 상대 영향력은?

해설 A11-2. 제곱 비용 가중치는 $1/\sigma^2$이므로 100분의 1이 된다. 같은 관측이어도 덜 믿으므로 odometry 누적 결과에 더 가까운 해를 만든다.

A12. IMU와 시간: 센서를 더하면 저절로 좋아질까

측정하는 것은 위치가 아니다

IMU(Inertial Measurement Unit, 관성측정장치)는 가속도계와 자이로스코프로 운동을 측정한다. 자이로는 각속도(rad/s), 가속도계는 비력(specific force, m/s²)을 측정한다. 비력은 단순한 세계 좌표 가속도와 다르며 자세와 중력을 고려해야 한다. `R_WB`가 body→world 회전이면 이상적인 모델은 다음과 같다.

```
a_m = R_WB^T (a_W − g_W) + b_a + noise
a_W = R_WB (a_m − b_a) + g_W      # noise를 생략한 복원
omega_m = omega_true + b_g + noise
```

정지한 센서에서도 가속도계가 0이 아닌 중력 관련 값을 읽는 이유다. 부호는 축·중력 정의에 따라 확인한다. **bias**는 지속적인 오프셋이며 시간에 따라 변할 수도 있다. 위치를 얻으려면 속도와 위치로 적분해야 하므로 작은 오차가 누적된다.

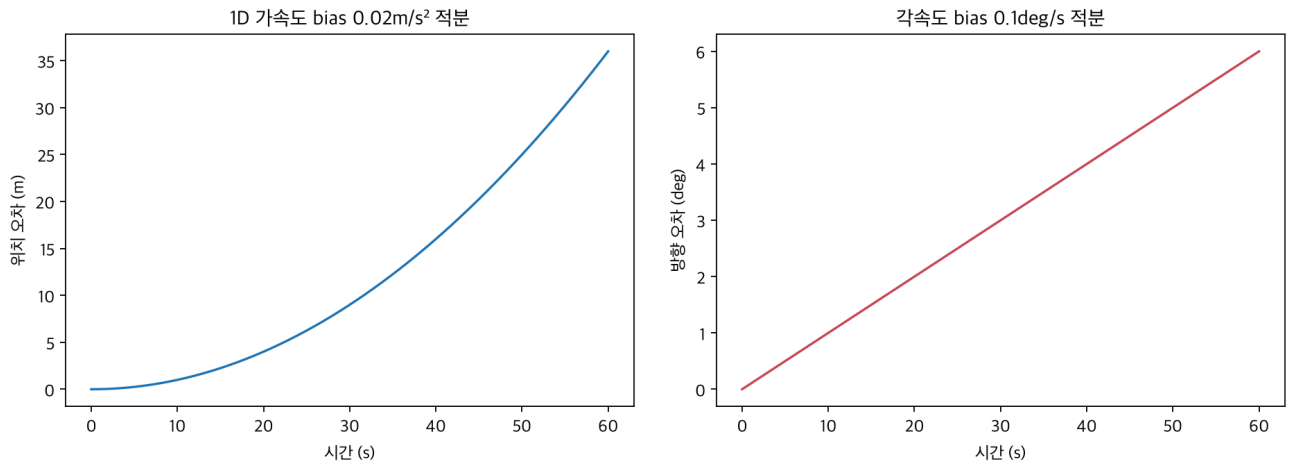


그림 A12-1. 중력·자세 결합을 제외한 독립 1D 모델이다. 가속도 bias 0.02m/s²는 위치 오차 `0.5*b*t²`를 만들어 60초 후 36m가 된다. 각속도 bias 0.1deg/s는 60초 후 6deg 오차를 만든다. 실제 3D IMU의 성능 예측값은 아니다.

시간 오차는 공간 오차가 된다

동일한 파일 번호가 동일 시각을 뜻한다고 가정하면 안 된다. 카메라와 IMU 사이 Δt가 있으면, 그동안 움직인 위치로 잘못 관측을 해석한다. 단순 가로 등속 v, 고정 깊이 Z, 회전 없음이라는 모델에서 영상 오차 크기는 `Δu≈f*v*Δt/Z`다.

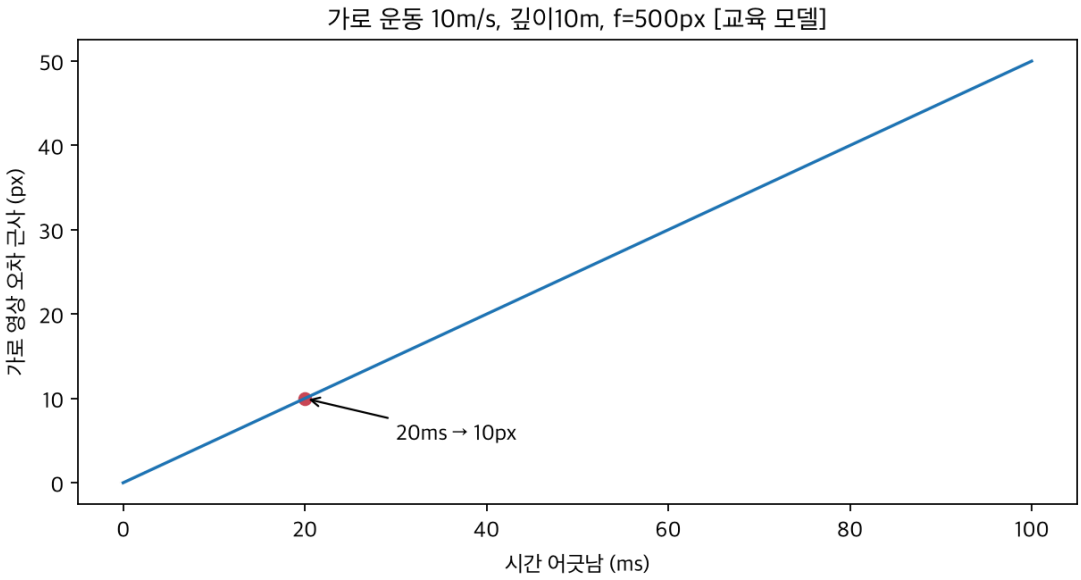


그림 A12-2. f=500px, v=10m/s, Z=10m일 때 20ms가 약 10px에 해당한다. 이는 KITTI 동기화 오차의 측정값이 아니다. 실제 회전·노출·LiDAR 스캔 시간은 더 복잡한 영향을 준다.

filtering, smoothing, preintegration의 역할

filtering(필터링)은 현재까지 관측으로 현재 상태의 분포를 순차 갱신한다. Kalman filter는 선형·Gaussian 모델에서 평균과 공분산을 전달한다. 예측에서는 운동 모델로 상태와 불확실성을 전파하고, 갱신에서는 새 관측과 예측의 차이를 가중해 수정한다. **EKF(Extended Kalman Filter, 확장 칼만 필터)**는 비선형 함수를 현재 추정 근처에서 선형화한다.

smoothing(평활화)은 여러 시점 상태를 함께 수정한다. 미래 관측으로 과거 상태도 바꿀 수 있다는 점이 filtering과 다르다. 고정 길이 구간만 유지하는 방식도 있다. **preintegration(사전적분)**은 두 카메라 시점 사이의 많은 IMU 측정을 상대 운동 제약으로 요약하여 최적화 중 반복 적분 비용을 줄이는 접근이다. bias와 회전의 처리까지 포함되므로 단순한 가속도 누적합과 같지 않다. Forster 외, On-Manifold Preintegration

현재 저장소에는 Raw OXTS 자료가 있지만 Odometry 프레임과의 시간·좌표 정렬이 검증되지 않았다. OXTS의 융합된 GPS/INS 위치·자세를 원시 IMU인 것처럼 입력하면 평가 정보가 섞일 수 있다. VIO(Visual-Inertial Odometry)나 LIO(LiDAR-Inertial Odometry)를 시작하기 전에 timestamp, 단위, 축, 외부 보정, bias, 초기화, GT 분리를 확인해야 한다. 이번 교재는 필터·VIO·LIO를 새로 구현하지 않는다.

연습문제와 해설

문제 A12-1. 같은 1D bias 모델에서 시간이 60초에서 30초가 되면 위치 오차는?

해설 A12-1. $0.5 \times 0.02 \times 30^2 = 9$m다. 시간이 절반이면 오차는 4분의 1이다. 초기 속도 오차 등이 없다는 가정이 필요하다.

문제 A12-2. 그림의 설정에서 시간차가 5ms라면 픽셀 오차는? 이 값으로 실제 동기화 오차를 확정할 수 있는가?

해설 A12-2. $500 \times 10 \times 0.005 / 10 = 2.5$px다. 단순 모델의 예상치일 뿐 실제 timestamp 오차를 추정한 것이 아니다. 실제 데이터의 속도·깊이·회전·잔차를 확인해야 한다.

기초 교재 실험 기록과 재현

이번 작업의 문제 정의. v1.1.1의 로드맵만으로는 수식·그림·연습을 연결하기 어렵다는 것이 문제다. 입력은 기존 문서, KITTI 00 frame120, 교육용 설정이다. 출력은 동일한 통합 PDF와 재현 가능한 그림·배열이다. [가정] 성공 기준은 12개 상세 단원, 24개 문제/해설, 15개 신규 그림, 25개 수치 검사와 PDF 검증이다. 실제 학습 성취도는 사용자 학습 후 평가해야 하며, 페이지 수가 이해도를 보장하지 않는다.

항목	기존 기준	이번 확장	해석
수학 학습	1장의 로드맵·짧은 예제	A1-A12 독립 단원	기존 응용 설명 보존
신규 단계 그림	이번 작업 전 없음	15개	실제 KITTI 1개, 교육 모델 14개
명시적 문제·해설 쌍	신규 기초 단원 없음	24쌍	각 단원 2쌍
계산 검산	신규 기초 실험 없음	25개 assertion	제한된 설정의 수치 검사
VO/LO 정확도	기존 연구 결과	재측정하지 않음	성능 향상 주장 없음

표 A13. 문서 완성도 기준의 baseline 비교다. 실제 주행 알고리즘의 정량 평가 표가 아니다.

코드 탐색 위치는 `research/foundations-textbook/src/generate.py:40`(벡터·좌표), `:55`(투영·스테레오·KITTI), `:75`(robust·SVD·확률), `:90`(Jacobian·회전), `:102`(그래프·IMU)다. 함수별 저장 배열을 먼저 살펴본 뒤 그림을 비교한다.

PDF 추가 환경 설치가 필요한 경우 아래 명령을 사용한다. 실제 사용 버전은 marked 18.1.3, Playwright 1.65.0, Mermaid 12.3.0이다. 시스템 Chrome도 필요하다.

```
npm install --prefix /tmp/kitti-report-pdf marked@18.1.3 playwright@1.65.0 mermaid@12.3.0
uv pip install --python research/20261031_vo-lo-study/.venv/bin/python \
  -r research/foundations-textbook/configs/pdf-requirements.txt
```

모듈 흐름과 계약. `configs/lessons.json → generate.py → artifacts/*/values.npz 및 summary.json → figures/*.png → docs/report.md → docs/report.pdf`. frames는 점·회전·병진, camera는 깊이·시차·표준편차, kitti는 원시 점·변환·픽셀, estimation은 비용·공분산·MLE/MAP, linearization은 반복 기록·회전, graph는 전후 상태·Jacobian, imu는 시간과 오차를 저장한다. 입력·출력의 차원과 좌표계는 각 단원에 정의했으며 상세 계약은 `research/foundations-textbook/notes/architecture.md`에 둔다. 새로운 상태 추정 모듈을 기존 playback에 연결하지 않았다.

```
# 저장소 루트에서 실행. 기존 KITTI 데이터와 연구 venv 필요.
env MPLCONFIGDIR=.cache/matplotlib XDG_CACHE_HOME=.cache \
  research/20261031_vo-lo-study/.venv/bin/python \
  research/foundations-textbook/src/generate.py
node tools/render_report.mjs
research/20261031_vo-lo-study/.venv/bin/python \
  research/foundations-textbook/src/check_pdf.py
```

수치 설정은 seed=7, frame120, 교육용 f=500px, B=0.54m, σ_d=0.5px다. 실제 투영 그림에는 교육용 f 대신 KITTI P0/Tr를 읽는다. 생성 당시 기준 커밋은 178ec59724069e60b991f1f4b967b1d63fc3617c이며 실행 시 HEAD와 설정 SHA-256을 `artifacts/summary.json`에 자동 기록한다. Python 3.11 환경에서 OpenCV 5.0.0, NumPy 2.4.6, SciPy 1.17.1, Matplotlib 3.11.1을 사용했다. 의존성은 기존 연구 lock 파일을 사용하고 PDF 검사는 신규 `configs/pdf-requirements.txt`의 pypdfium2 5.14.0을 추가한다. 한글 그림은 macOS Apple SD Gothic Neo를 사용하므로 다른 OS에서는 한글 글꼴 설정이 필요하다.

검증 결과. 변환 역복원, 순서 차이, 벡터 길이, 투영 scale 모호성, 스테레오 깊이, 실제 영상 내 점 수, 평균·Huber·MLE·MAP, 공분산 양의 고윳값, 다섯 유한차분 Jacobian, GN 해·선택 시작점의 비용 감소, SO(3) 직교성·잘못된 덧셈, gauge rank·공통 이동, loop 잔차, IMU·시간차 예제의 총 25개 검사가 통과했다. 이는 교재의 제한된 예제를 검산한 것이며 모든 입력에 대한 알고리즘 안정성 검증은 아니다.

강점·약점과 다음 단계. 원시 LiDAR는 미터 거리와 실제 투영을 확인하기 좋지만 가림·스캔 시간 문제를 이 단원에서 해결하지 않았다. 이미지 대응은 장면의 풍부한 정보를 쓰지만 단안 scale과 오대응이 남는다. 공분산 가중치는 서로 다른 신뢰도를 반영하지만 모델이 틀리면 과신한다. loop는 누적 오차를 재분배하지만 잘못된 제약은 전체 궤적을 망가뜨린다. IMU는 빠른 운동 정보를 주지만 bias와 시간 정렬을 필요로 한다. 이는 각 단원의 수식·그림으로 설명한 한계이지 이번 작업의 feature 성능 ablation 결과가 아니다.

다음 학습 선택지는 ① A1-A4의 실제 투영 검산, ② 기존 ORB 장의 실제 단계 이미지와 A5-A9 연결, ③ A11에서 실제 loop 검증·BA로 확장, ④ A12 이후 시간 정렬과 IMU 사전적분이다. 이번 책은 기초 1차 확장이며 BA 전체 유도, EKF 전체 구현, Lie Jacobian, 실제 SLAM 통합은 아직 남아 있다. 사용자의 선택 없이 해당 구현으로 넘어가지 않는다.

가정·미확인 목록. [가정] 교육용 Gaussian 독립·고정분산, 알려진 stereo baseline, 작은 시차 오차, 1D bias, 등속 가로 운동, 올바른 합성 loop를 사용했다. [미확인] 실제 OXTS 시간 정렬, 실제 VIO/LIO 성능, 신규 설명의 학습 효과, 다른 OS의 글꼴·PDF 동일성은 검증하지 않았다. 신규 원문 출처는 각 단원의 링크와 `notes/prior_work.md`에 모았으며 그림은 모두 자체 생성했다.

2. ORB: 픽셀부터 descriptor와 매칭까지

기존 요약보다 상세한 설명을 우선한다. 피라미드 그림은 공통 픽셀 축으로 보완한 최신 자료다.

원 실험 디렉터리: `research/20261101_orb-explained/`

기본 용어와 사용 가능한 리소스

약자와 이름을 먼저 풀기

이름	풀어 쓴 말 / 유래	이 프로젝트에서 하는 일
ORB	Oriented FAST and Rotated BRIEF	방향이 있는 코너와 회전 보정 이진 descriptor 생성
FAST	Features from Accelerated Segment Test	원 둘레 밝기를 비교해 코너 후보 검출
Harris	Chris Harris의 성; 약자가 아님. Mike Stephens와 공동 제안	여러 방향으로 밝기가 바뀌는 정도를 평가
NMS	Non-Maximum Suppression, 비최대 억제	가까이 중복된 응답 중 약한 것을 제거
BRIEF	Binary Robust Independent Elementary Features	여러 픽셀 쌍의 밝기 비교를 비트열로 표현
rBRIEF	ORB 논문의 회전 대응·선택된 BRIEF 검사 패턴을 가리키는 명칭	분산이 높고 상관이 낮은 비교를 선택
BF	Brute Force	후보 descriptor들을 직접 비교
Hamming	Richard Hamming의 성; 약자가 아님	두 비트열에서 서로 다른 자리 개수
RANSAC	RANdom SAmple Consensus	일부 잘못된 대응을 견디며 기하 모델 추정
VO / SLAM	Visual Odometry / Simultaneous Localization and Mapping	영상 이동 추정 / 위치와 지도 동시 추정
ROI	Region of Interest, 관심 영역	피라미드에서 동일 장면 일부를 확대 비교
MAE	Mean Absolute Error, 평균 절대 오차	축소 후 복원 영상과 원본의 평균 밝기 차이

ORB 이름·주요 구성은 OpenCV ORB 설명, FAST와 BRIEF의 이름은 각각 FAST 설명, BRIEF 설명에서 확인했다.

자주 섞이는 데이터 타입

- **픽셀 intensity I(x,y):** 흑백 밝기. 이번 영상은 uint8, 즉 0~255 정수다. 색 이름이나 거리 값이 아니다.
- **patch:** 한 위치 주변을 잘라낸 작은 이미지. 예를 들어 31×31 픽셀이다.
- **keypoint:** 고른 위치와 부가 정보. x/y, 크기 size, 방향 angle, 검출 레벨 octave, 반응 response 등이 있다. angle과 response는 서로 다르다.
- **descriptor:** keypoint 주변 외형의 수치 요약. 이번 ORB는 32바이트=256비트다. keypoint 좌표가 descriptor 안에 그대로 저장되는 것은 아니다.

- **match:** 이전 영상 descriptor의 인덱스와 현재 영상 descriptor의 인덱스, 그 사이 거리다. 비슷하다고 검색된 후보이지 동일 물체라는 보장은 아니다.
- **inlier:** 지정한 기하 모델과 오차 허용 범위에 맞는 대응. 반대는 outlier다. 영상의 초록색 점이라는 이유만으로 진실이라고 단정할 수 없다.

실제 보유 리소스

리소스	위치와 사용
원본	data/kitti/dataset/sequences/00/image_0/000120.png 및 000121.png
이전 교육 코드	research/20261031_vo-lo-study/src/orb_stages.py:7~13
이전 피라미드	같은 study의 artifacts/orb/pyramid.npz
실제 ORB 호출	같은 study의 src/vo.py; 새 src/analyze.py에도 설정 명시
새 중간 출력	artifacts/*.npz, levels.csv, summary.json, harris_toys.json
실행 환경	기존 연구 venv, OpenCV 5.0.0, 단일 CPU thread

접근 선정: 단계마다 직접 확인하기

새 검출기를 개발할 문제가 아니라, 기존 알고리즘 설명과 그림의 정보 손실을 수정할 문제다. 따라서 기존 데이터를 사용하는 작은 재현 실험을 택했다. 이하 설명에서 **실제 ORB 출력**, **독립 교육 계산**, **가상의 수치 예제**를 구분한다. 원 논문 전체를 그대로 재구현했다고 부르지 않는다.

피라미드: 왜 같은 사진을 여러 크기로 만드는가

차량이 물체에 가까워지면 그 물체가 사진에서 커진다. 한 크기의 작은 patch만 비교하면 같은 물체라도 주변 모양이 다르게 들어온다. 입력을 여러 해상도로 만들어 비슷한 크기로 보이는 레벨끼리 대응할 기회를 주는 것이 피라미드의 역할이다.

다른 장면을 만드는 것이 아니다. 동일 장면을 더 적은 픽셀로 표현한다. 따라서 큰 윤곽이 비슷한 것은 정상이다. 차이는 배열 크기, 작은 무늬의 소실, 같은 patch가 덮는 원본 영역이다.

이번 scaleFactor=1.2, 레벨 l=0~7이면 명목 축소 배수 s(l)=1.2^l이다. 폭은 대략 W/s(l), 높이는 H/s(l)이며 정수 반올림이 있다. 영상 길이 비율은 1/s(l), 픽셀 수 비율은 대략 1/s(l)^2다. '1.2배 scale'을 영상을 1.2배 키운다는 뜻으로 읽으면 반대가 된다.

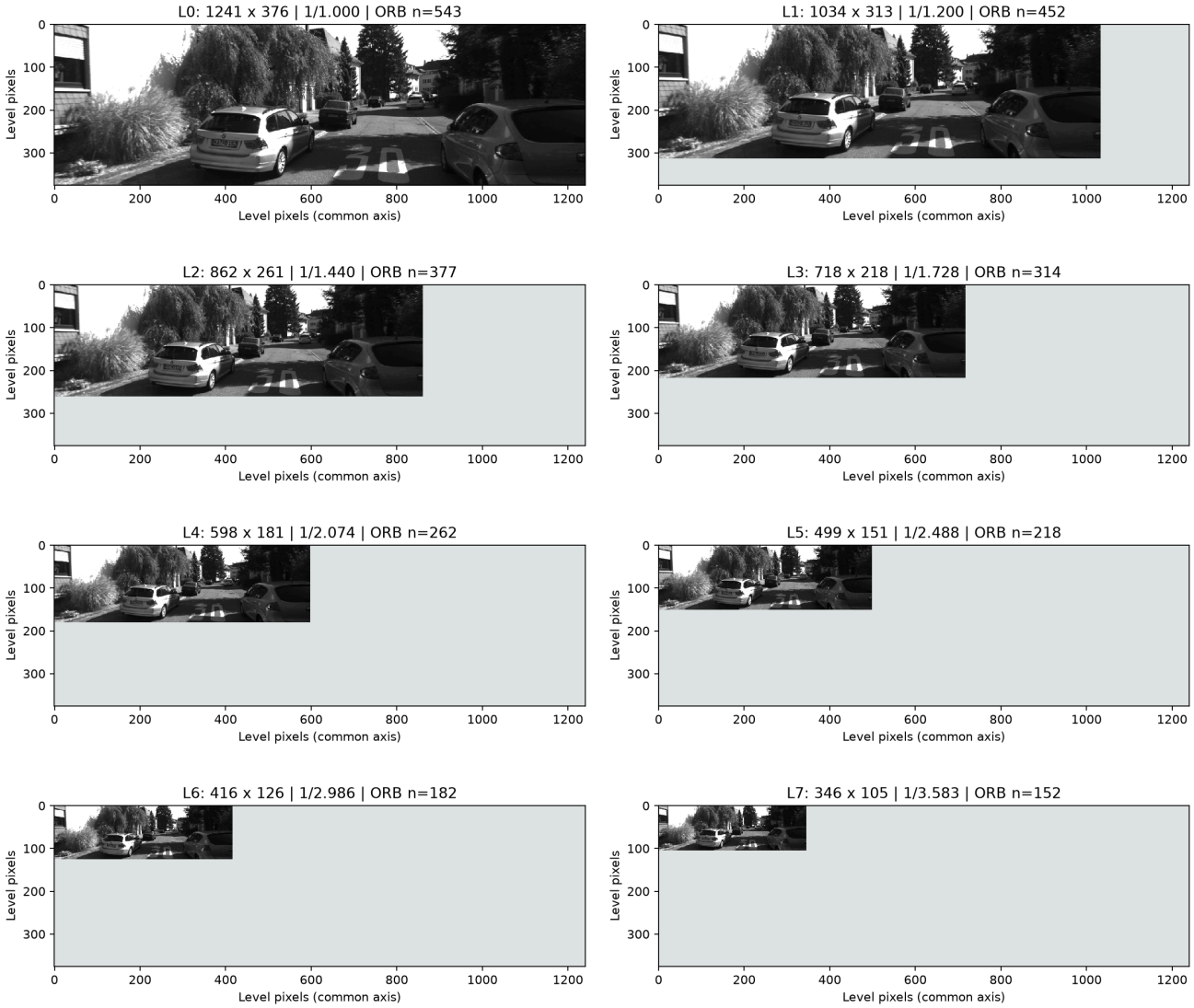
가상의 예: 어떤 물체가 첫 영상에서 20px, 두 번째에서 24px라면 두 번째의 레벨1에서는 약 20px로 줄어든다. 이때 비슷한 크기의 모양을 비교할 수 있다. 실제 장면은 원근·가림·조명도 함께 변하므로 피라미드만으로 완전한 scale invariance가 생기지는 않는다.

피라미드 scale과 VO metric scale은 별개다. 전자는 영상 크기 비율이다. 후자는 '이동 벡터 1이 실제 몇 m인가'라는 물리 거리 문제다. ORB 레벨을 늘려도 단안 VO의 절대 거리 문제가 해결되지 않는다.

기존 그림이 거의 같아 보였던 정확한 이유

이전 코드에서는 각 subplot의 `imshow(p)`가 해당 영상 크기에 맞춰 좌표 범위를 자동으로 잡았다. L0도 L3도 자기 칸을 거의 가득 채워 원래 줄어든 크기가 지워졌다. 제목에 shape만 적어 놓는 것으로는 충분하지 않았다.

기존 저장 배열과 같은 방식으로 재생성한 L0~~L3를 `np.array_equal`로 비교했고~~ **~~네 배열 모두 동일~~**했다. ~~이는 이전 배열 생성이 실제 축소였다는 증거다. 이제 모든 subplot의 x축을 0~~1241, y축을 0~376으로 고정했다. 같은 level pixel이 같은 도식 크기를 차지하므로 축소를 직접 볼 수 있다. PDF 화면의 실제 모니터 픽셀과 1:1이라는 뜻은 아니다.



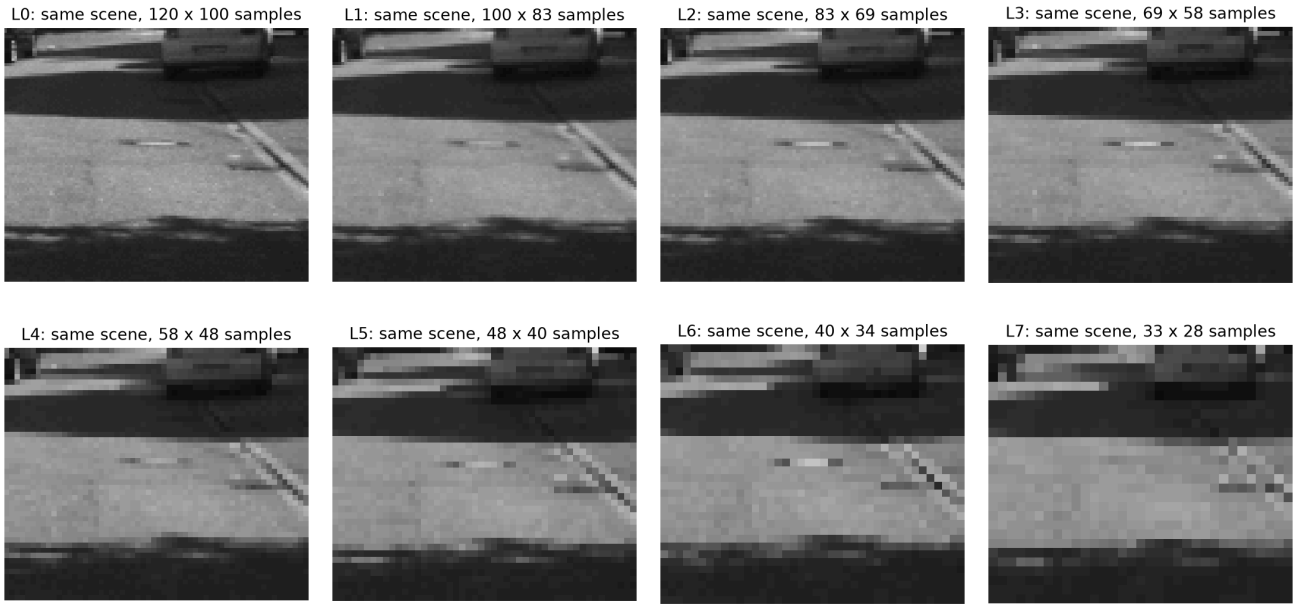
읽는 법: 회색 여백은 데이터가 없는 부분이다. L0는 전체 칸을 채우고 L7은 작은 부분만 채운다. 제목의 ORB n은 실제 ORB의 레벨별 개수이며 배경은 독립 교육용 resize다.

레벨	실제 배열 폭×높이	원본 대비 길이	31px patch의 원본 폭	실제 ORB 점 수	복원 MAE
0	1241×376	100.00%	31.00px	543	0.000
1	1034×313	83.33%	37.20px	452	3.138
2	862×261	69.44%	44.64px	377	3.312
3	718×218	57.87%	53.57px	314	4.442
4	598×181	48.23%	64.28px	262	5.277
5	499×151	40.19%	77.14px	218	5.348
6	416×126	33.49%	92.57px	182	6.339
7	346×105	27.91%	111.08px	152	7.170

MAE는 각 축소 영상을 원본 크기로 bilinear 확대하고 mean(abs(original-restored))로 계산했다. 단위는 0~255 밝기다. 0이 아니라는 것은 원본 복제본이 아니며 축소·복원 과정에서 정보가 바뀌었다는 뜻이다. VO 정확도 지표는 아니다. 레벨별 검출 개수도 품질 점수가 아니라 설정과 검출 결과다.

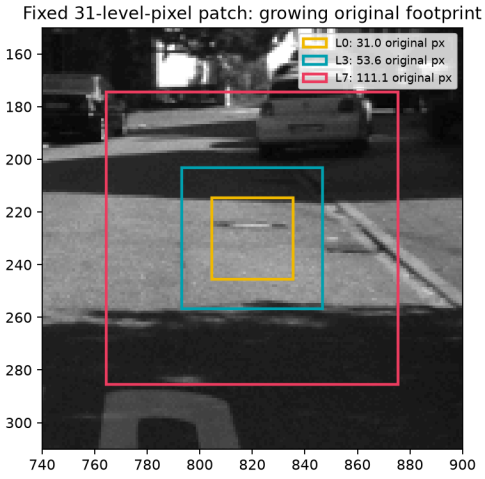
같은 장면의 확대 비교와 patch 범위

bilinear interpolation은 정수 격자 사이 밝기를 주변 네 픽셀의 가중 평균으로 계산하는 보간이다. 줄어든 영상에서 모든 원본 픽셀을 독립적으로 보존할 수 없다. 미세 패턴이 사라지거나 다른 패턴처럼 보이는 aliasing이 생길 수 있다. 정밀한 축소 필터와 완벽한 영상 복원은 이번 실험 범위가 아니다.



이번에는 의도적으로 같은 크기로 확대했다. 목적은 배열 전체 크기가 아니라 **정보 손실을 확대해 보는 것**이다. nearest 표시를 사용해 새 부드러운 픽셀을 만들어 숨기지 않았다. 같은 도로 부분이 L0에서는 120×100표본, L7에서는 33×28표본으로 표현된다. 작은 점과 경계가 뭉개지는 부분을 비교한다.





왼쪽 빨간 상자가 위 확대의 위치다. 오른쪽 상자들은 각 레벨에서 31픽셀인 patch가 원본에서 얼마나 큰 영역을 덮는지 보여준다. L7의 31px는 원본 약111px 범위다. 상자 중앙은 설명용 고정 위치이지 같은 keypoint가 모든 레벨에서 검출된다는 주장이 아니다.

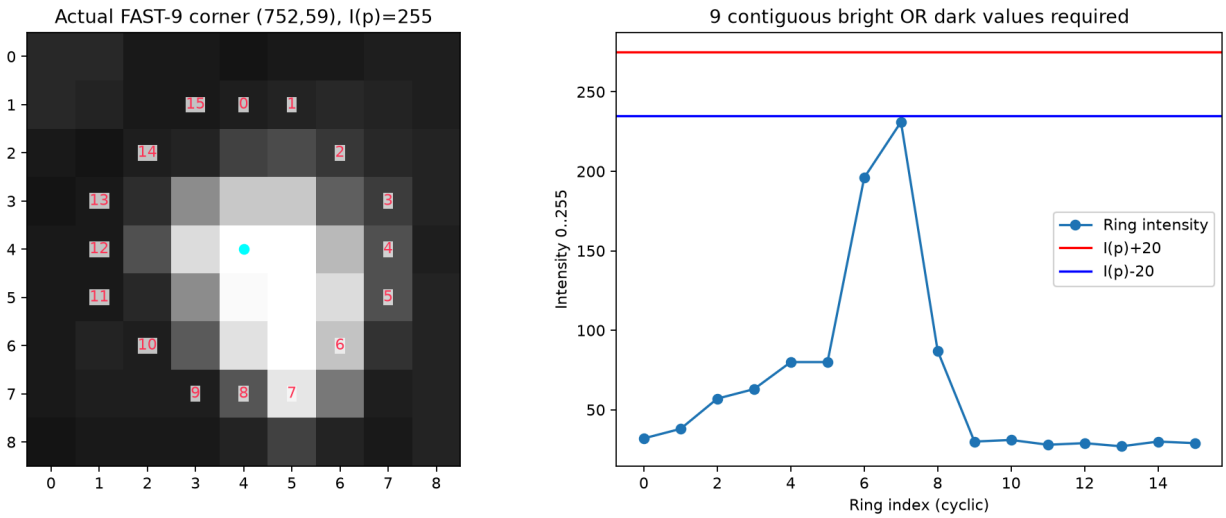
OpenCV keypoint의 `octave`는 여기서 피라미드 레벨 인덱스다. 이름이 octave여도 매 레벨이 2배라는 뜻은 아니다. `pt`는 보통 원본 영상 좌표로 반환되고 `size`에는 레벨 scale이 반영된다. 그래서 서로 다른 레벨의 점도 원본 위에 그릴 수 있다. 이번 실험은 레벨별 출력 필드를 저장한다. OpenCV ORB API

FAST: 원 둘레만 보고 후보를 빠르게 찾기

입력은 한 레벨의 grayscale 영상이다. 후보 중심 p의 밝기를 I(p), 밝기 문턱을 t라고 하자. 중심에서 반경 약3픽셀인 원 둘레의 16개 위치를 본다. 이번 FAST_9_16에서는 그 원을 따라 **연속 9개 이상**이 모두 I(p)+t보다 밝거나, 모두 I(p)-t보다 어두우면 통과한다. 밝고 어두운 것을 섞어 9개 세는 것이 아니다. 원이므로 끝 인덱스에서 처음으로 이어지는 구간도 검사한다.

FAST는 이름처럼 segment test의 구조를 이용해 많은 위치를 빨리 탈락시키려는 방법이다. 원래 FAST 논문의 FAST-12 설명과 이번 OpenCV FAST-9 설정을 혼동하면 안 된다. 학습된 decision tree는 검사 순서를 효율화하는 별도 요소이며 이번 코드는 그것을 재학습하지 않는다. Rosten·Drummond, 2006 원문

왜 단순한데 코너를 찾을까? 균일한 영역은 원 둘레도 중심과 비슷해 탈락한다. 경계의 꺾임이나 국소적으로 독특한 밝기 분포에서는 긴 밝은/어두운 호가 생길 수 있다. 하지만 '기하학적으로 건물 모서리'만 검출하는 것은 아니다. 점광원·노이즈·텍스처도 통과할 수 있다.



실제 선택 위치는 (752,59), 중심 밝기는 255다. t=20이므로 어두운 판정 경계는 235다. 원 둘레 밝기는 아래와 같고 모두 235보다 작아서 연속9 조건을 만족한다. 밝은 판정 경계 275는 uint8 범위 밖이라 이 위치는 밝은 호로 통과할 수 없다.

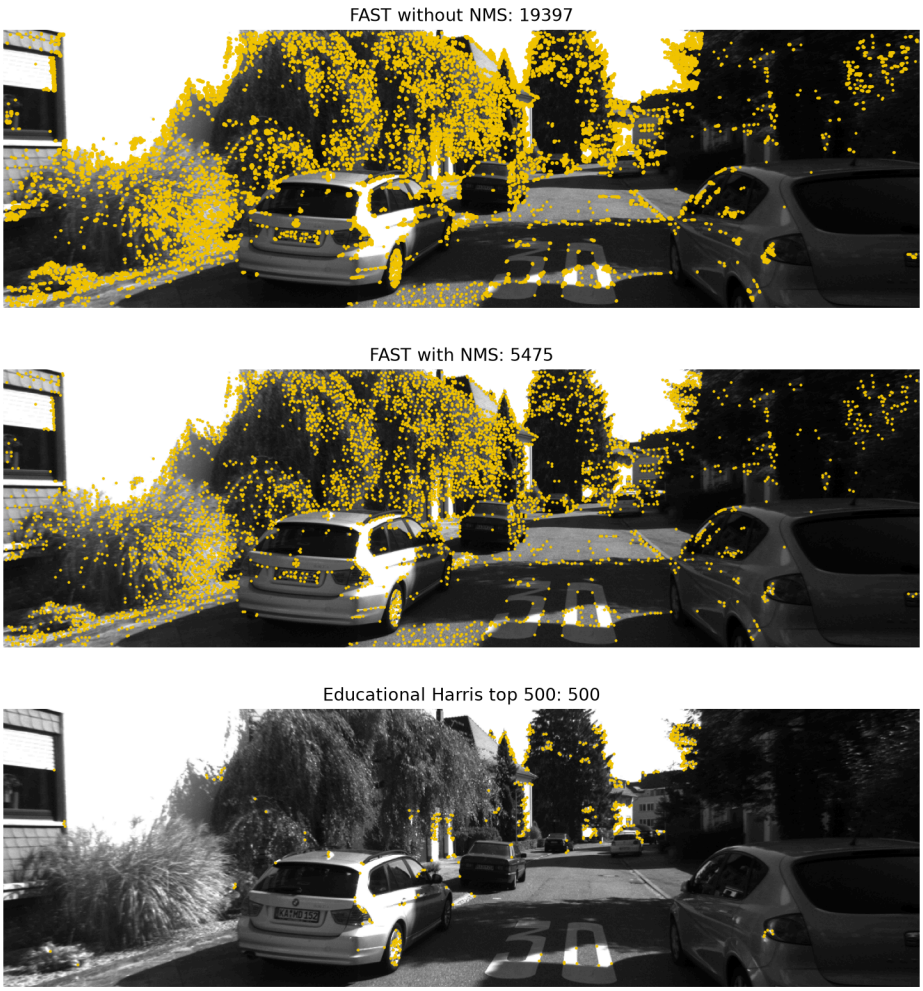
```
ring index:  0  1  2  3  4  5   6   7  8  9 10 11 12 13 14 15
intensity:  32 38 57 63 80 80 196 231 87 30 31 28 29 27 30 29
state:      전부 dark (<235)
```

이 사례는 오히려 중요한 한계를 보여준다. FAST의 corner는 '물리적인 모서리'라는 의미와 동일하지 않다. 포화된 중심 밝기를 가진 작은 구조도 높은 응답이 가능하다. 이 점 하나를 대표적인 좋은 추적점이라고 일반화하지 않는다.

NMS: 이웃한 후보들을 왜 줄이는가

좋은 구조 주변에서 여러 픽셀이 동시에 통과할 수 있다. 이들을 모두 쓰면 한 군데에 점이 몰리고 비교 비용이 늘어난다. NMS는 가까운 후보 사이에서 응답이 더 큰 것을 남기는 지역 경쟁이다. 전체 영상에서 상위500개를 고르는 연산과는 다르다.

이번 영상에서 FAST nonmaxSuppression=False는 19,397개, True는 5,475개였다. 이 차이는 실제 detector를 두 설정으로 실행한 결과다. 점 개수가 줄었다고 recall이나 VO 성능이 좋아졌다고 결론내리지는 않는다. nonmax 점수의 정확한 내부 구현은 OpenCV 버전에 의존하므로 이 보고서는 임의의 간단한 점수를 같은 내부 점수라고 부르지 않는다.



위에서 아래로 후보가 어떻게 줄어드는지 본다. 마지막 500개는 교육용 Harris 순위다. 실제 다중 레벨 ORB의 2,500개와 같은 집합이 아니다.

Harris: FAST를 썼는데 왜 또 다른 코너 점수가 필요한가

FAST의 역할은 빠른 후보 생성이다. 제한된 개수만 남길 때는 다른 관점의 코너 강도로 순위를 매길 수 있다. Harris는 patch를 어느 방향으로 조금 움직여도 모양이 달라지는지 평가한다. 이름은 약자가 아니라 제안자 이름에서 왔다. Harris·Stephens, 1988 학회 기록

평평한 벽 patch는 좌우·위아래로 조금 이동해도 비슷하다. 긴 수직 경계는 가로로 이동하면 크게 달라지지만 세로로 이동하면 비슷하다. 코너는 두 방향 모두 변화가 크다. 이것이 코너가 2D 위치를 정하기에 유리한 이유다.

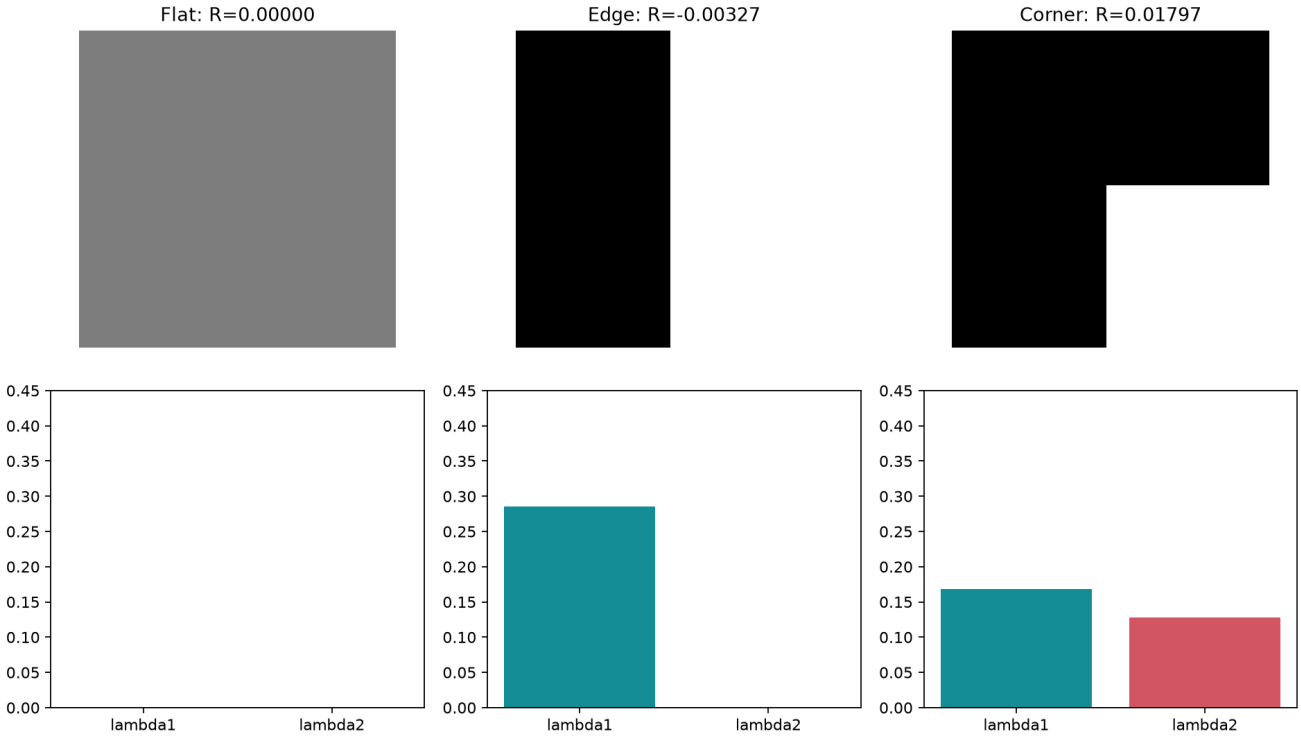
수식의 기호를 하나씩 쓰면 다음과 같다. I_x, I_y는 각각 가로·세로 밝기 변화량(영상 미분)이다. w는 patch 안의 픽셀에 주는 가중치다. sum은 patch 전체 합이다.

```
M = [ sum(w * Ix²)      sum(w * Ix * Iy) ]
    [ sum(w * Ix * Iy)  sum(w * Iy²)     ]
```

```
R = det(M) − k ∗ trace(M)²
  = lambda1 ∗ lambda2 − k ∗ (lambda1 + lambda2)²
```

M은 2×2 구조 행렬이다. trace는 대각 원소 합, det는 행렬식이다. lambda1/lambda2는 고유값으로, patch의 두 주된 방향에서 밝기가 얼마나 변하는지 요약한다. k는 점수 모양을 조절하는 상수이며 여기서는 0.04다. R이 큰 양수면 두 방향 변화가 모두 큰 경향, 음수면 한 방향만 큰 경계 경향, 0 근처면 평탄한 경향이다. 계산 구조와 해석은 공식 Harris 설명에 근거한다.

실제 교육 계산은 `cornerHarris(img, 7, 3, .04)`다. 7은 주변 집계 영역 크기, 3은 미분을 구하는 Sobel 커널 크기다. Sobel은 이웃 밝기의 가중 차분으로 미분을 근사하는 필터다. FAST 후보의 위치에서 R을 읽고 높은 순으로 500개를 골랐다. 이 dense Harris 계산을 OpenCV ORB 내부의 모든 최적화와 동일하다고 주장하지 않는다.



이 그림만은 해석을 쉽게 하려고 만든 41×41 합성 영상이다. 밝기는 0~1 float다. 위는 영상, 아래는 중심 부근 두 고유값이다. 실제 도로 사례와 합성 예제를 혼동하지 않는다.

합성 patch	lambda1	lambda2	R	해석
평탄면	0	0	0	어느 방향도 변화 없음
수직 경계	0.285714	0	-0.003265	한 방향만 변화
코너	0.168367	0.127551	0.017973	두 방향 모두 변화

Harris의 약점은 blur·noise·밝기 대비에 따라 응답이 달라지고, 코너 점수가 높아도 다른 장소의 비슷한 코너와 구분된다는 보장은 없다는 것이다. 위치 선택과 식별력은 다른 문제다.

방향: Oriented FAST의 Oriented는 어디서 생기는가

FAST만으로는 '이 patch가 어느 방향을 향하는가'를 얻지 못한다. 카메라가 회전하면 같은 물체 주변도 돌아가기 때문에 descriptor의 비교 방향을 맞출 기준이 필요하다.

교육 예에서는 중심을 (0,0)으로 놓고 반지름15인 원 안의 밝기를 무게처럼 사용한다. 좌표 x는 오른쪽, y는 아래로 증가한다. 모멘트(moment)는 '위치에 밝기를 곱해 더한 값'이다.

```
m00 = sum(I)        : 밝기 총합
m10 = sum(x * I)    : 좌우 밝기 분포
m01 = sum(y * I)    : 상하 밝기 분포
centroid = (m10/m00, m01/m00)
theta = atan2(m01, m10)
```

centroid는 밝기 무게중심이다. 중심에서 무게중심으로 향하는 방향을 theta로 삼는다. atan2는 x/y 부호를 함께 고려해 사분면까지 구분하는 각도 함수다. m00=0인 검은 patch, m10/m01이 모두0인 대칭 patch는 방향이 의미 없거나 불안정해질 수 있다. 구현에서 반환하는 숫자가 있더라도 방향 정보가 충분하다는 뜻은 아니다.

이번 실제 patch의 m00=29662, m10=-1100, m01=-21548이다. 무게중심은 (-0.0371,-0.7265)px, 각도는 -92.92도다. 이미지 y가 아래로 증가하므로 화면상 위를 향한다. 0~360도로 표현하면 약267.08도지만 **이 독립 교육 계산이 실제 ORB keypoint.angle과 반드시 같다는 뜻은 아니다.** 경계/원형 표본 규칙이 다를 수 있다.

BRIEF: 작은 그림을 어떻게 0과 1로 바꾸는가

BRIEF는 검출기가 아니라 descriptor다. 이미 정해진 patch 안에서 위치 a와 b를 고르고 밝기를 비교한다. I(a)<I(b)이면 1, 아니면 0이다. 이런 질문 256개를 고정된 순서로 묶으면 256비트가 된다. 8비트가1바이트이므로 32바이트다. 픽셀을 비교하기 전에 smoothing을 해 작은 노이즈의 영향을 줄인다. OpenCV BRIEF

Gaussian smoothing은 이웃 밝기를 종 모양 가중치로 평균하는 것이다. 이번 교육 예는 7×7 커널과 sigma=2를 사용한다. sigma는 평균에 참여하는 공간 범위의 퍼짐을 조절한다. smoothing 때문에 작은 밝기 차이는 완화되지만 세부 정보도 줄어든다.

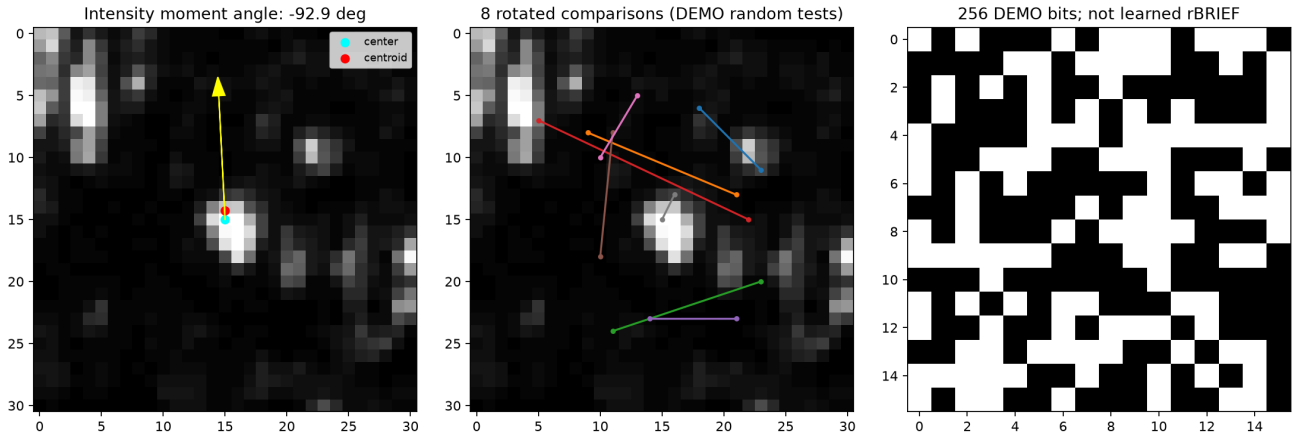
Rotated BRIEF: 고정된 비교 위치를 그대로 쓰면 patch가 회전할 때 다른 물체 부분을 읽는다. 앞에서 구한 theta만큼 비교 위치 좌표를 회전시키면 물체 기준으로 비슷한 위치를 비교할 수 있다. 여기서 이미지 전체를 회전시키는 것이 아니라 비교 좌표를 돌린다.

```
[ x' ] = [ cos(theta)  -sin(theta) ] [ x ]
[ y' ]   [ sin(theta)   cos(theta) ] [ y ]
```

실제 영상에서 뽑은 교육용 첫8개 질문은 다음과 같다. 이것은 seed7로 뽑은 무작위 패턴이며 실제 ORB의 학습된 패턴이 아니다.

질문	I(a)	I(b)	I(a)<I(b)
0	36	55	1
1	38	29	0
2	29	51	1
3	111	37	0
4	27	27	0
5	28	25	0
6	28	29	1
7	135	87	0

위 예의 앞8비트는 10100010이다. 어느 질문인지의 순서까지 같아야 두 descriptor를 비교할 수 있다. 프레임마다 새 random pattern을 뽑으면 대응 비교가 의미 없어진다.



왼쪽은 원 중심·무게중심·방향, 가운데는 실제 patch에 그린8개 회전 비교, 오른쪽은256개 질문의 참/거짓이다. 화살표 길이10px는 각도를 보기 위한 표시 길이이며 무게중심 거리0.73px가 아니다. 비트의 16×16 배열은 표시용이며 실제 물체의 2D 모양 지도가 아니다.

rBRIEF: 무작위 질문만 있으면 충분한가

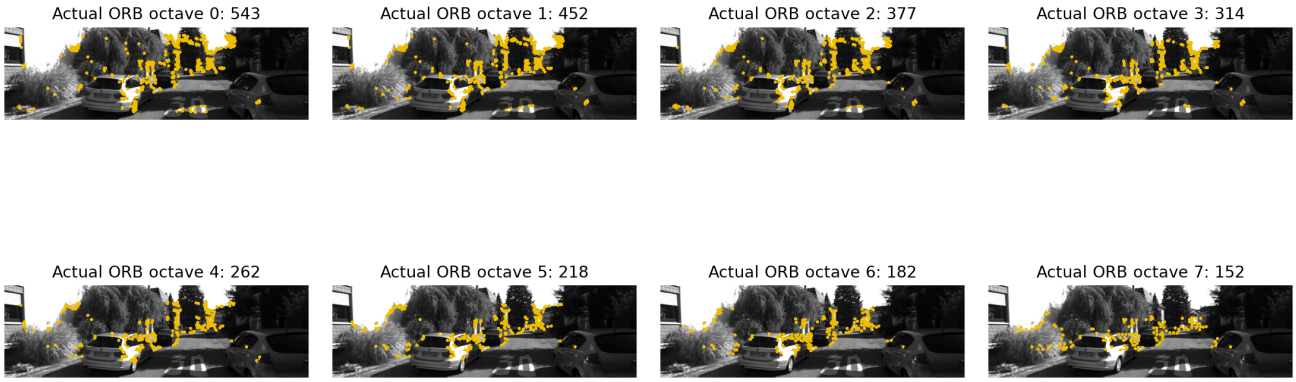
어떤 질문의 답이 거의 항상0이면 서로 다른 patch를 구분하는 정보가 적다. 반대로 0/1이 자주 바뀌는 질문은 더 다양한 경우를 나눌 수 있다. 또 두 질문의 답이 항상 함께 바뀌면 비슷한 정보를 중복해서 저장한다.

ORB의 rBRIEF 선택은 다양한 patch에서 비트의 평균이 약0.5이고 분산이 크며 질문 사이 상관이 낮은 패턴을 찾는 방향이다. Bernoulli 비트의 분산은 p(1-p)이므로 p=0.5일 때0.25로 최대다. 상관은 두 질문의 답이 함께 변하는 정도를 뜻한다. 완전한 독립을 보장한다는 뜻은 아니다. OpenCV ORB의 rBRIEF 설명

이것은 '현재 frame120에서 매번 패턴을 다시 학습한다'는 말이 아니다. 이번 OpenCV 호출은 제공된 패턴으로 descriptor를 계산한다. 학습 데이터·선택 과정은 재실행하지 않았다. 새 교육 스크립트의 랜덤256질문을 rBRIEF 재현이라고 부르지 않는 이유다.

실제 OpenCV ORB 출력과 교육 그림의 경계

실제 `detectAndCompute` 출력은 keypoint 2,500개와 uint8 descriptor 배열 (2500,32)다. 레벨별 개수 합도2,500이다. 각 행은 한 keypoint의256비트를32바이트에 담은 것이다. 코드에서 그림에 보이는 byte 값 하나를 하나의 bit로 해석하면 틀린다. `unpackbits`는 바이트를 비트들로 펼쳐 표시하는 연산이다.



모든 배경이 같은 원본 크기인 것은 의도적이다. 이 그림의 질문은 '각 레벨에서 찾은 점이 원본의 어디에 있는가'다. 피라미드 크기를 보여주는 첫 그림과 목적이 다르다. 같은 물체가 여러 레벨에서 검출될 수도 있으며 점 하나하나의 3D 동일성은 이 그림만으로 알 수 없다.

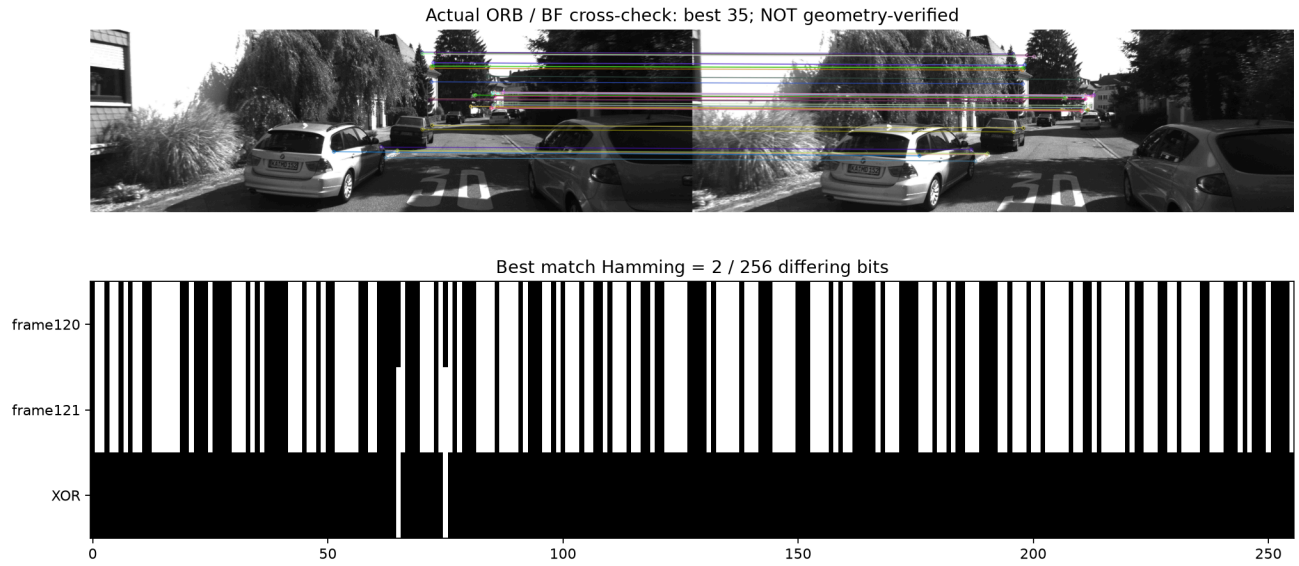
우리 교육 피라미드는 원본에서 각 레벨을 독립 resize했다. 실제 ORB는 버전별 내부 경계 확장·보간·반복 축소·blur 등의 세부가 다를 수 있다. OpenCV4.13 소스를 참고했으나 실행 binary는5.0.0이고 정확한 빌드 소스 일치까지 확인하지 못했다. 따라서 그림을 'ORB 내부 buffer 그대로'라고 표시하지 않는다. 참고 소스

Hamming, BF, cross-check, ratio: 무엇을 기준으로 같은 점을 찾는가

Hamming distance는 같은 길이 비트열에서 다른 자리 수다. 예를 들어101100과100110은 세 번째와 다섯 번째가 달라 거리가2다. XOR는 다를 때1, 같을 때0을 내는 비트 연산이다. XOR 결과에서1의 개수를 세는 popcount로 Hamming을 계산한다.

BF는 이전 descriptor마다 현재 descriptor들을 직접 비교해 가장 작은 거리를 찾는다. cross-check는 A가 고른 최선이 B이면서 B가 고른 최선도 A일 때만 남기는 상호 최근접 조건이다. 반복된 무늬에서는 상호 최근접이어도 오매칭일 수 있다. OpenCV matching

ratio test는 최근접 거리 d1과 두 번째 거리 d2를 비교해 d1<0.75*d2 같은 조건을 쓴다. 최선과 차선이 비슷하면 애매하다고 보는 것이다. 예를 들어20과22는 탈락,20과50은 통과한다. 0.75는 이전 실험의 설정이지 보편적 정답이 아니다. cross-check와 ratio는 서로 다른 필터이며 이번 기본 VO는 cross-check다.



120→121의 실제 ORB/BF cross-check 대응은1,611개다. 위에는 거리순35개를 보여주고, 아래에는 최저 거리 대응의 descriptor 두 개와 XOR를 표시했다. XOR의1이2개이고 OpenCV distance도2로 일치한다. **이 그림은 기하 검증 전**이다. 작은 Hamming이 올바른 3D 대응을 보장하지 않는다.

여기서 VO까지: ORB가 끝나는 곳

현재 baseline은 매칭을 거리순으로 최대1,000개 사용한다. 이후 `findEssentialMat`의 RANSAC으로 정적인 장면의 두 시점 기하를 추정한다. RANSAC은 일부 대응으로 후보 모델을 만들고 많은 대응이 그 모델에 맞는지 반복 확인하는 방식이다. 모델에 대한 오차가 작으면 inlier로 분류한다.

카메라 내부 파라미터 K는 초점거리와 주점을 담는다. 픽셀(u,v,1)에 K의 역행렬을 곱해 카메라 광선 방향에 대응하는 정규화 좌표 x를 만든다. Essential matrix E는 두 정규화 관측 사이의 관계 $x2^T$ E x1=0을 표현한다. 실제 대응에는 노이즈가 있어 이 값 또는 관련 거리 오차가 완전히0일 필요는 없다.

`recoverPose`는 E에서 회전 R과 이동 방향 t를 복원하고, 삼각측량된 점이 두 카메라 앞에 놓이는지 검사해 가능한 해를 고른다. 삼각측량은 두 광선으로3D 위치를 추정하는 계산이고, 앞쪽 여부 검사를 cheirality check라고 한다. OpenCV 기하 API

단안 영상만으로 전체 장면과 이동을 함께 같은 배수로 키워도 같은 투영이 가능하다. 따라서 이 단계의 t 길이를 실제 m로 알 수 없다. 현재 코드는 단위 이동 방향을 누적하므로 속도 변화·프레임 간 거리 차이를 복원하지 못한다. 회전이 좋아도 경로 길이는 틀릴 수 있다. 피라미드·Harris·BRIEF는 이 물리적 scale 모호성을 해결하는 방법이 아니다.

이후의 발전 방향도 목적이 다르다. stereo는 알려진 카메라 간 거리로 깊이 scale을 얻는 접근, BA(Bundle Adjustment)는 여러 카메라 pose와3D 점을 영상 재투영 오차가 작도록 함께 조정하는 최적화, loop closure는 재방문 제약으로 전역 누적 오차를 줄이는 기능이다. 이들을 ORB라는 한 단어 안에 포함시키지 않는다. 이번 보고서에서 새 구현한 단계는 아니다.

시스템 아키텍처와 산출물 계약

모듈	입력	출력 및 저장	검증
피라미드	frame120 grayscale	artifacts/pyramid.npz, levels.csv, roi_level_*.npy	기존4배열 동일, shape/hash, 복원 MAE

모듈	입력	출력 및 저장	검증
FAST/Harris	원본 픽셀, threshold20	artifacts/fast_harris.npz, harris_toys.json	cyclic9 조건, 고유값/응답 부호
방향/BRIEF 교육	실제 patch, seed7 패턴	artifacts/orientation_brief.npz	모멘트·회전 좌표·픽셀 비교 저장
실제 ORB	원본, 명시 설정	artifacts/actual_orb.npz	8레벨 합2500, descriptor shape
매칭	120/121 descriptor	artifacts/matching.npz	XOR1 개수=OpenCV distance
출판	위 출력	figures/*.png, report.pdf	이미지 decode 및 레이아웃 검토

모든 영상 좌표는 x오른쪽/y아래, 단위pixel이다. 방향 교육 배열은 radian, 표시값은 degree다. pi radian=180도다. level 좌표와 원본 좌표의 scale을 구분하며 이번 작업에는 새로운 센서 시간 정렬이 없다.

결과와 baseline 비교

검사	이전 자료	이번 결과
배열 구분	shape만 표시	기존4배열 동일 재현, 8레벨 shape/hash 기록
축소 크기 시각화	subplot 자동 fit	모든 축 동일, 여백으로 축소 표현
정보 손실	확인 어려움	같은 ROI 표본 수와 nearest 확대
patch 범위	설명 부족	31px→원본31~111.08px 표시
FAST	후보 overlay만	실제16픽셀 수치와9연속 조건 통과
Harris	순위 결과만	합성 평탄/경계/코너 고유값과 R 비교
방향/BRIEF	이름 위주	모멘트 값·각도·비교 밝기·비트 저장
실제 매칭	선 그림	실제 descriptor XOR와 distance2 일치

이 표는 교육적 관찰 가능성의 개선이다. VO RMSE, detector recall, 실행 지연 성능을 새로 측정한 것이 아니다. 코드 assert가 통과했고 그림9개를 생성했다. 사람이 이해했는지는 이 자동검사만으로 판정할 수 없다.

강점, 한계, 실패 사례

Feature별 강점과 약점

구성	강한 조건	약한 조건	이번 근거
피라미드	제한된 영상 크기 변화	극단적 축소, 원근 변형, 가림	L7 ROI 세부 소실, MAE7.170
FAST	대비가 뚜렷한 국소 구조	노이즈, blur, 포화 점	중심255인 실제 통과 예
NMS	이웃 중복 후보 제거	강한 소수 영역 편중 해결 불충분	19397→5475, 위치 그림
Harris	두 방향 변화 구분	반복 코너의 식별 문제	합성 고유값 표
밝기 중심 방향	비대칭 밝기 분포	대칭/어두운 patch, 조명 변화	실제 centroid가 중심에서0.73px밖에 떨어지지 않음

구성	강한 조건	약한 조건	이번 근거
회전 BRIEF	방향 추정이 일관된 경우	방향 오차, 비교 밝기 차이가 작은 경우	27/27,28/29 같은 민감한 질문
Hamming/BF	가볍고 해석 가능한 거리	반복 무늬, 모든 점 비교 비용	거리2도 기하 정답 아님

숨기지 말아야 할 한계

이번 frame120 하나와121의 대응만으로 일반적인 불변성·강건성을 평가하지 않았다. 밝기 변화·회전·blur 조건별 성능 곡선은 없다. FAST가 찾은 포화점의 실제 추적 안정성도 이 실험으로 확인하지 않았다. 실제 ORB 패턴과 random 교육 패턴의 성능 비교를 한 것이 아니다.

이전 연구의 단일30도 합성 회전 ablation은 방향의 영향을 보여주는 별도 진단이다. 그 수치를 KITTI 전체나 실제 주행 회전에서의 정확도로 읽으면 안 된다. 이번 보고서는 이해를 위해 입력과 계산을 눈으로 추적하는 데 집중했다.

성공 기준 달성도

배열 동일성, 실제 shape 기록, L1~7 비영 복원 오차, FAST9, XOR 거리 일치와 주요 용어 설명은 완료했다. OpenCV 내부 buffer 완전 재현, rBRIEF 재학습, 모든 주행 상황의 성공률은 범위 밖이며 완료했다고 표시하지 않는다.

한계를 다루는 후속 접근

SuperPoint: 검출 위치와 descriptor를 데이터로 학습

SuperPoint는 FAST/Harris의 수작업 규칙과 BRIEF식 외형 표현 대신, 영상에서 위치 점수와 descriptor를 함께 계산하는 신경망 접근이다. Homographic Adaptation은 알려진 평면 투영 변환으로 영상을 바꿔 얻은 검출 결과를 모아 학습 신호를 보강하는 과정이다. homography는 평면 또는 특정 카메라 운동에서 두 영상의 점을 연결하는3×3 투영 변환이다.

이 방향은 '어떤 점이 다시 찾기 좋고 어떤 표현이 구분에 좋은가'를 학습한다. 하지만 가림으로 사라진 관측을 항상 복원하거나 단안 metric scale을 해결하는 방법은 아니다. 학습된 weight, 호환 런타임, 우리 데이터에서의 검증이 필요하며 이번에는 실행하지 않았다. DeTone·Malisiewicz·Rabinovich, SuperPoint, 2018

LightGlue: 점 하나의 가장 가까운 descriptor만 보지 않기

LightGlue는 두 이미지의 local feature들을 받아 주변 대응의 관계까지 활용하는 학습 기반 matcher다. attention은 다른 feature의 정보 중 무엇을 더 참고할지 가중하는 계산이다. 쉬운 쌍과 어려운 쌍에 계산량을 다르게 쓰는 적응적 설계가 포함된다.

이것은 FAST 자체를 만드는 검출기가 아니라 주로 매칭 단계의 대안이다. 임의의 ORB 32바이트를 기존 모델에 그대로 넣으면 된다는 뜻은 아니다. 모델이 지원하는 descriptor 종류·차원·학습 조건을 맞춰야 한다. 반복 패턴의 애매함을 줄이는 방향이지만 실제 차량 이동, 가림, 잘못된 카메라 모델까지 모두 해결한다고 볼 수 없다. Lindenberger·Sarlin·Pollefeys, LightGlue, 2023

지금 선택할 때의 구분

다음 방법	해결하려는 한계	필요한 것	예상 공수
설정 ablation	threshold/level에 대한 민감도	현재 데이터로 가능, 통제된 변환	낮음
SuperPoint	검출·표현의 반복성/식별력	weight, 비교 기준, 모델 실행 환경	중간
호환 feature+LightGlue	독립 최근접 매칭의 애매함	호환 descriptor와 weight	중간
stereo+깊이+pose 추정	단안 metric scale 모호성	오른쪽 영상과 calibration	높음

후속 논문은 '모든 단점의 만능 후계자'가 아니라 서로 다른 부분을 바꾸는 선택지다. 이번 작업에서 자동 적용하지 않는다.

부록: 재현 경계와 참고문헌

- [가정] 교육 목적의 대표 frame120과 ROI(760,180,120,100)를 선택했다. 장면 전체 대표성을 보장하지 않는다.
- [미확인] 현재 OpenCV5.0.0 binary와4.13 참고 소스의 정확한 구현 일치. 원 ORB PDF 전체 직접 열람. 실제 rBRIEF 선택 과정 재현.
- [구분] 교육 Harris/원형 moment/random BRIEF는 실제 ORB의 내부 단계 trace가 아니다. 실제 ORB의 kp/descriptor/octave 결과는 별도 저장했다.
- [출처] 도표 수치는 이 작업의 artifacts에서 생성했다. 일반 알고리즘 정의는 본문 가까이에 링크한 원저자 논문·학회 기록·공식 OpenCV 자료를 사용했다. 옛 논문의 속도 비교 숫자와 특허 상태는 현재 사실로 재사용하지 않았다.
- ORB 서지: Ethan Rublee, Vincent Rabaud, Kurt Konolige, Gary Bradski, *ORB: An efficient alternative to SIFT or SURF*, ICCV2011. 공식 OpenCV 자료로 서지 확인.
- BRIEF 서지: Michael Calonder, Vincent Lepetit, Christoph Strecha, Pascal Fua, *BRIEF: Binary Robust Independent Elementary Features*, ECCV2010. 공식 OpenCV 자료로 서지 확인.
- 실제 실행 코드와 결과의 연결: src/analyze.py, configs/experiment.json, artifacts/summary.json. 새 연구 코드는 이전 생산 VO/LO를 수정하지 않는다.

3. VO 기하와 LiDAR Odometry: 구현·비교·한계

첫 200프레임 실험이다. 화면 구성은 5장의 통합 재생이 최신이다. 이 장의 실행 당시 커밋·측정 조건은 역사적 실험 출처로 보존한다.

원 실험 디렉터리: `research/20261031_vo-lo-study/`

영상 기하와 통제 실험

방향 보정은 실제로 도움이 되는가: 통제 실험

frame 120을 정확히 30도 회전한 합성 영상을 만든다. 알려진 affine 변환으로 매칭의 정답 pixel 위치를 계산하고 오차 3px 미만을 올바른 대응으로 정의했다. GT는 이 교육 실험의 label에만 사용했다.

같은 검출 위치에서 ORB 방향을 사용한 경우와 angle을 0으로 강제한 경우를 비교했다. 방향을 제거한 방법은 실제 baseline 개선안이 아니라 구성 요소의 역할을 보는 ablation이다.

설정	상호 최근접 매칭 수	알려진 변환 기준 정확 대응 비율
실제 ORB 방향 사용	1280	89.14%
같은 위치, descriptor angle=0	388	1.03%

표 1. 단일 합성 회전에서 방향 보정 효과. 실제 도로의 원근·가림·독립 물체 운동·조명 변화를 포함한 전체 성능으로 해석하면 안 된다.

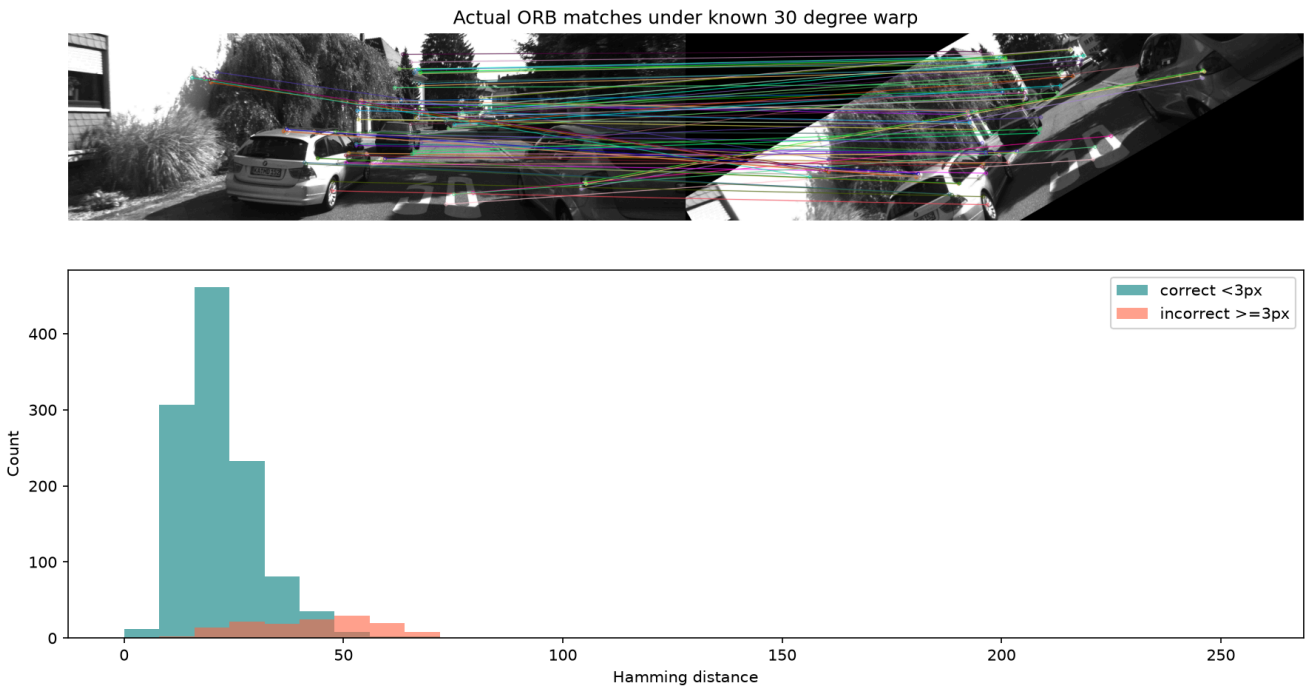


그림 6. 위는 실제 ORB 매칭 일부, 아래는 정답/오답별 Hamming 거리 분포. 겹치는 분포가 있으므로 거리 하나만으로 완벽한 판별이 불가능하다. 원시 자료는 `synthetic_rotation.csv`다.

단계 5: BF Hamming, cross-check, ratio test

Hamming 거리는 두 descriptor의 XOR에서 다른 비트 수다. BFMatcher는 후보를 비교해 작은 거리를 찾는다. cross-check는 A가 선택한 B가 반대로도 A를 선택할 때 유지한다. 이는 상호 선택이지 같은 3D 점이라는 증명은 아니다.

ratio test는 가장 가까운 거리 d1이 두 번째 거리 d2에 비해 충분히 작은지 본다. 이번 비교는 `d1 < 0.75 d2`로, cross-check baseline을 KNN ratio 방식으로 **대체**한 ablation이다. 둘을 동시에 적용한 것으로 해석하지 않는다. 두 방법 모두 거리순 상위 1000개를 사용한다.

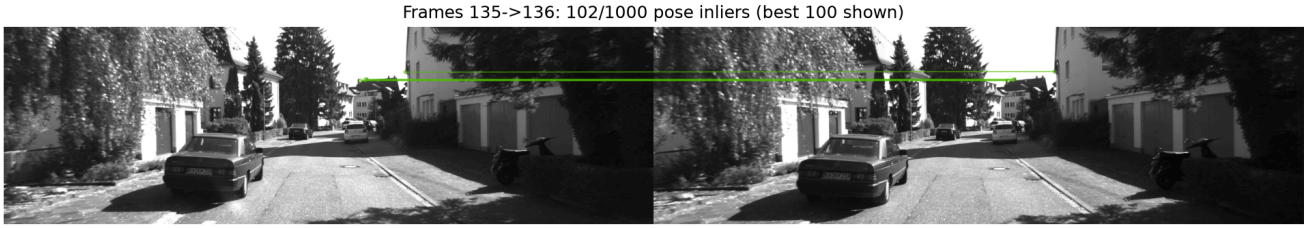


그림 7. frame 135→136의 pose inlier 중 거리순 상위 100개 범위만 표시했다. 전체 대응과 거리·mask는 `artifacts/vo/baseline/000136.npz`에 있으며 이 프레임의 pose inlier는 102/1000개다.

단계 6: Essential Matrix와 RANSAC

매칭에는 오답이 섞인다. RANSAC은 작은 대응점 집합에서 기하 모델을 가정하고 많은 관측과 일관되는 모델을 찾는다. 카메라 K로 정규화한 대응 x1,x2에는 $x2^{\top} E x1 \approx 0$라는 제약이 있고 $E=[t]_{\times}R$이다. 한 점의 상대 영상 위치가 한 직선 위에 놓여야 한다는 뜻이다.

이번 구현은 OpenCV `findEssentialMat`, RANSAC prob=0.999, threshold=1 pixel을 사용한다. 출력 E, Essential inlier mask, 두 영상 pixel 좌표를 모두 NPZ로 저장했다. 실제 API는 `vo.py`에 있다.

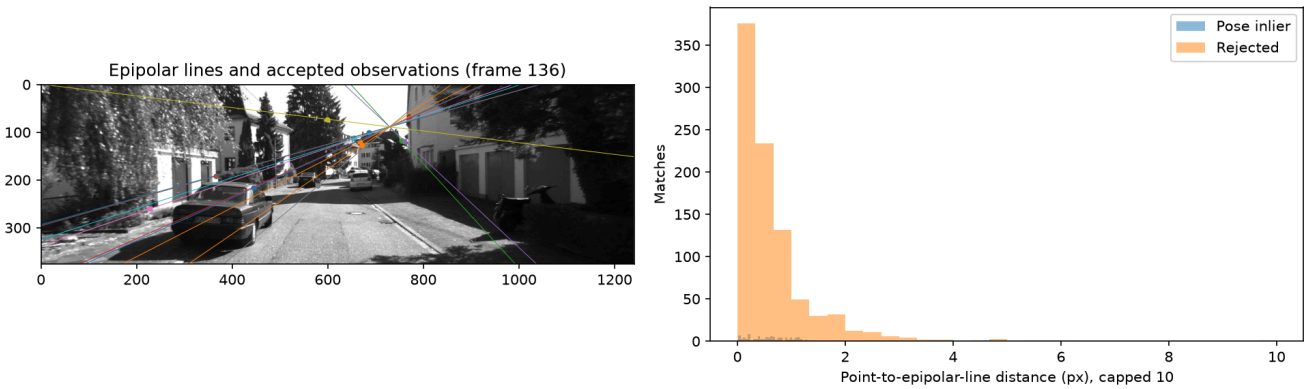


그림 8. frame 136의 epipolar line과 대응점, 점-직선 거리 분포. **pose에서 거절된 점의 중앙 거리가 더 작을 수도 있다:** 이번 inlier 중앙값 0.574px, 거절점 0.441px다. recoverPose는 epipolar 잔차만이 아니라 양의 깊이 등 기하 조건도 검사하므로 거절을 곧 오매칭 label로 취급하면 안 된다.

단계 7: recoverPose와 경로 누적

E에서 가능한 R,t 후보를 복원하고 점이 카메라 앞쪽에 놓이는 해를 고른다. t는 방향만 알려준다. `X_current=R X_previous+t`이면 카메라 world pose는 상대 변환의 역행렬을 곱해 누적한다.

출력은 full 4×4 pose 배열이다. 개별 R,t와 E도 저장했다. 여기서 모든 프레임의 이동 길이를 1로 두면 차가 감속해도 추정 이동량은 1이다. 이것이 현 baseline의 구조적 scale 한계다. 좋은 descriptor만 넣어도 사라지지 않는다.

LiDAR Odometry: 같은 깊이로 단계별 보기

LO는 무엇을 최적화하는가

카메라는 물체의 투영을 본다. LiDAR는 이미 meter 단위 3D 점을 제공한다. LO는 현재 점들을 이전 점군/지도에 맞추는 회전과 이동을 찾는다. 가장 단순한 목적은 대응 q에 대해 $\Sigma \ ||R\ p + t - q||^2$를 줄이는 것이다. 대응이 알려져 있지 않아 대응 찾기와 변환 갱신을 반복한다.

이번 실행은 **공식 KISS-ICP 1.3.0의 핵심 pipeline**이다. 자체 신규 LO나 LOAM의 재현이 아니다. 별도로 이해를 위한 fixed-gate nearest-neighbor/SVD ICP를 저장된 실제 scan-map 쌍에 구현했다. 이 교육용 반복을 KISS C++ 내부 iteration log로 부르지 않는다.

단계 1: XYZI 읽기와 범위 필터

각 .bin은 Nx4 float32의 x,y,z,intensity다. XYZ는 LiDAR 좌표계에서 x 전방/y 왼쪽/z 위쪽이다. intensity는 이번 정합 비용에 사용하지 않는다. 너무 가까운/먼 관측은 불안정하거나 비용 대비 효과가 낮을 수 있어 이번 실험은 5~100m를 사용했다.

입력 전체 스캔과 실제 KISS preprocessor 출력은 대표 프레임의 `artifacts/lo/000120.npz`에 저장된다. timestamp 기반 deskew는 **비활성화**했다. Odometry bin에 점별 시간 필드가 없고, 임의 시간 복원을 넣지 않기 위해서다.

단계 2: Voxel 표본화

공간을 격자로 나눠 점 수를 줄인다. 그대로 모든 점을 사용하면 가까운 표면의 많은 점이 계산과 목적함수를 지배할 수 있다. 이번 KISS 설정의 map voxel은 1m, 내부 map 추가용 downsample은 0.5m, registration source는 1.5m다. 설치된 `KissICP.voxelize`에서 확인했다.

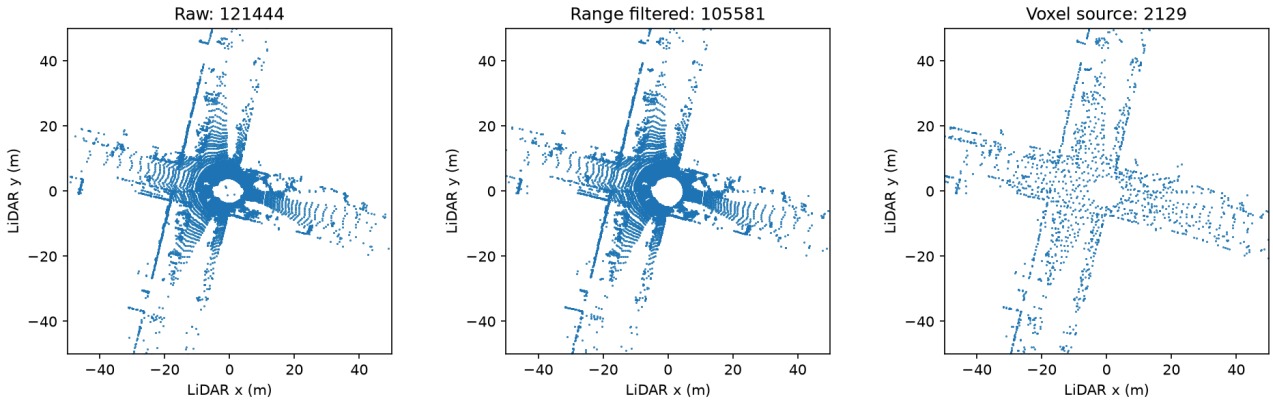


그림 9. frame 120의 원본→범위 필터→registration source. 각 점 개수는 그림 제목과 frames.csv에 있다. 같은 XY 범위를 그려 시각적 밀도를 비교한다. 큰 voxel은 비용을 줄이지만 작은 구조를 없앨 수 있다. voxel-size ablation은 미수행이다.

단계 3: 운동 예측과 대응점

정합은 초기값에 민감하다. KISS는 직전 상대 운동을 다음 시점에도 적용하는 예측을 초기 pose로 사용한다. 이 예측은 IMU를 사용한 예측이 아니라 이전 LO 결과에 기반한다.

현재 source를 예측 pose로 변환해 local map과 대응시킨다. 너무 먼 대응은 배제하고 residual이 큰 관측의 영향은 robust kernel로 줄인다. KISS의 threshold σ는 예측과 실제 추정의 차이에 따라 갱신되며, 이번 설치 코드의 correspondence gate는 3σ, kernel 인자는 σ다. σ의 실제 시계열은 `artifacts/lo/frames.csv`에 저장된다.

단계 4: 정합 반복과 pose 갱신

교육용 ICP는 다음 순서를 20회 실행한다: 현재 pose로 source 변환 → KD-tree 최근접 대응 → 1m gate → 중심을 뺀 대응의 cross-covariance → SVD로 R → 중심 차이로 t → pose 누적. reflection이면 det(R)=+1이 되도록 수정한다. 각 반복의 residual/대응 수/회전·이동 갱신을 CSV에 남긴다.

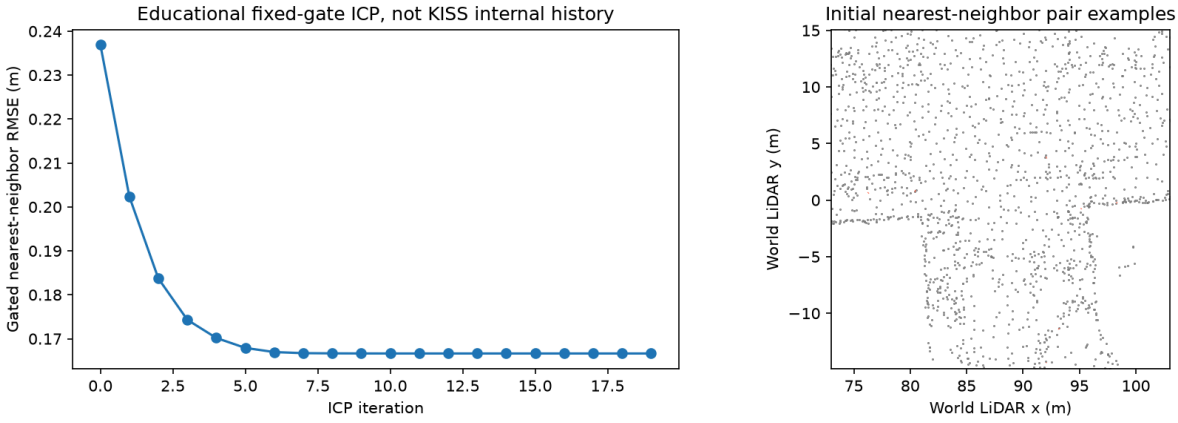


그림 10. frame 120 실제 scan-map에서 교육용 ICP의 gated 최근접 RMSE가 0.237m에서 0.167m로 줄었다. 매 반복 대응 집합이 달라지므로 고정 대응 비용의 단조 감소 보장은 아니며, 낮은 잔차 자체가 올바른 pose의 증명도 아니다.

실제 KISS는 robust registration backend를 사용한다. 보고서의 다음 거리 분포는 저장한 pose 전후에 SciPy KD-tree로 **독립 재계산한 진단**이고 KISS 내부 correspondence/가중치의 직접 dump가 아니다.

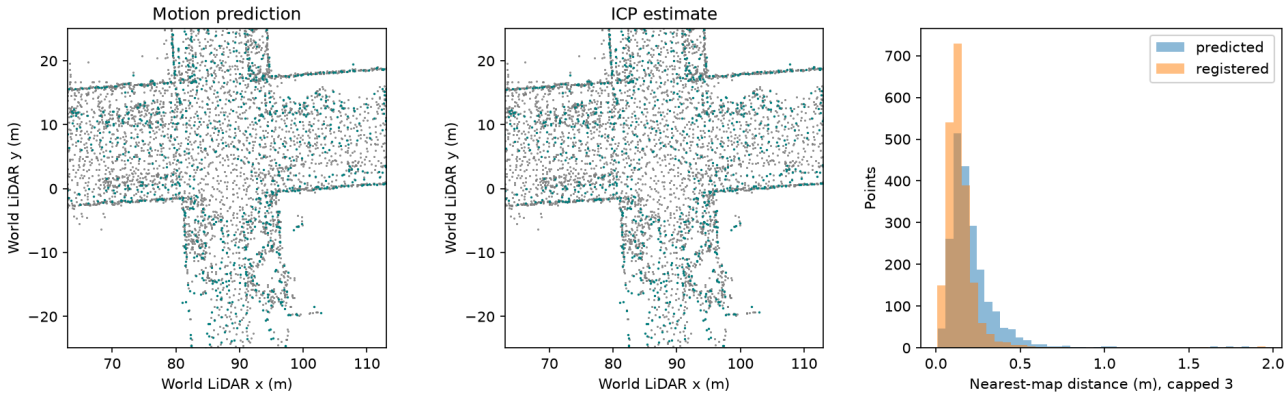


그림 11. 회색은 과거 local map, 청록은 현재 source. 예측과 최종 pose를 같은 공간에서 비교한다. 오른쪽은 최근접 거리 분포로 큰 거리는 3m에 모아 표시했다. 이전 source/target/pose와 거리 배열은 NPZ에 있다.

단계 5: 로컬 지도 갱신

최종 pose로 downsample 스캔을 map에 추가하고 센서에서 멀어진 점을 관리한다. 모든 과거 점을 영구 보관하는 전역 지도와 다르다. 그림은 200프레임 후 rolling local map이며 loop closure로 보정된 완전한 SLAM 지도가 아니다.

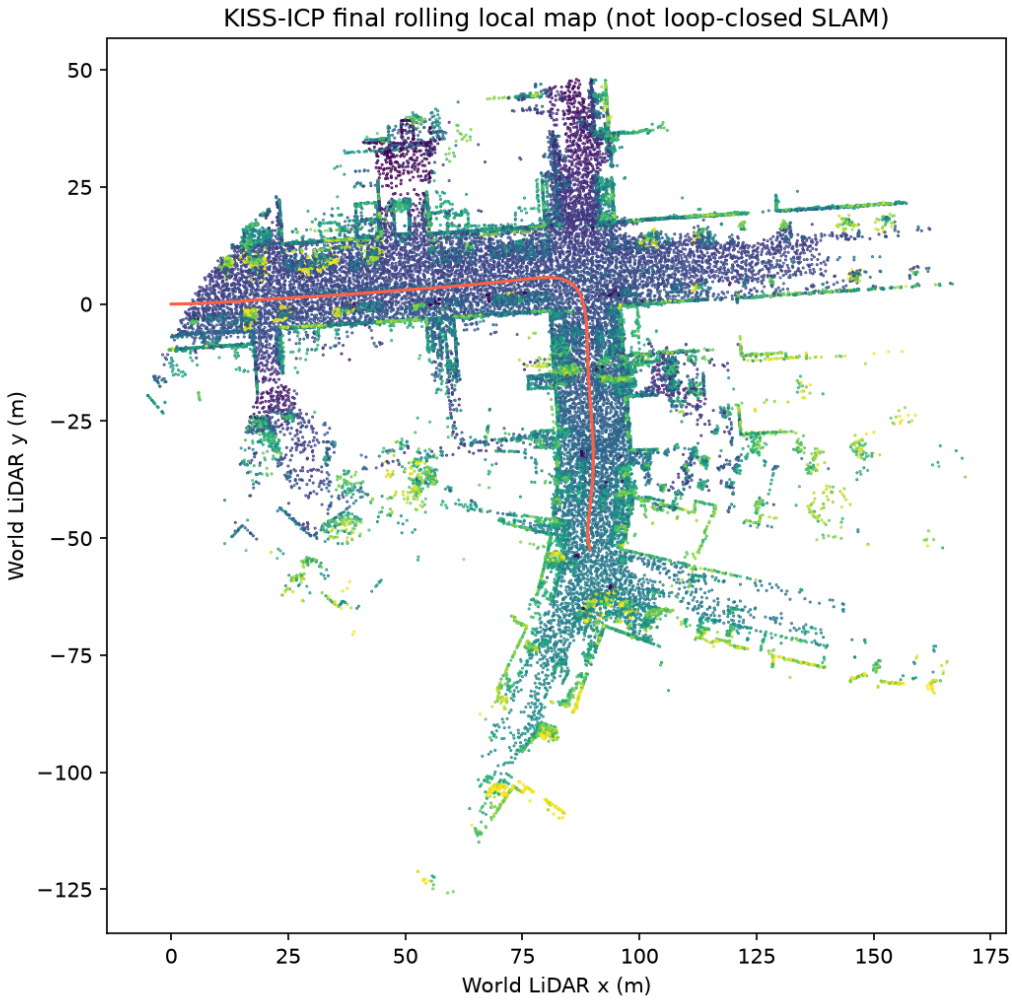


그림 12. 높이로 색칠한 LO 로컬 점군과 센서 경로. `final_local_map.npy`로 실제 3D 점을 제공한다. 색은 높이이지 카메라 RGB가 아니다.

LO의 한계와 관련 후속 연구

한계	관련 연구 / 접근	얻는 것	새로 필요한 것과 한계
전체 점 정합의 비용·geometry 활용	LOAM(Zhang/Singh, RSS 2014): edge/plane 특징과 odometry/mapping 분리	구조적 제약 활용	feature quality/센서 가정; 이번 미실행
단순 ICP의 outlier·초기값 민감도	KISS-ICP(Vizzo 외, RA-L 2023): 표본화·적응 gate·robust 정합·운동 예측 조합	순수 LiDAR baseline	geometry 부족/동적 물체/나쁜 초기값에서 실패 가능
빠른 운동과 스캔 시간차	FAST-LIO2(Xu 외, 2021 preprint): LiDAR와 IMU 결합, 직접 map 정합	고주파 운동 예측과 융합	정확한 시간/외부 보정/bias 모델 필요
장기 drift	loop detection + pose graph/global optimization	재방문 제약으로 일관성 개선	false loop 위험; 이번 미구현

표 3. LOAM은 KISS보다 오래된 다른 설계 갈래다. “KISS 다음에 나온 논문”이라는 뜻이 아니다. 이번 KISS 실행도 공식 KITTI loader의 별도 scan correction을 적용하지 않은 core 실험이며, 공식 논문/벤치마크 수치 재현으로 주장하지 않는다.

실험 설정과 재현

항목	설정
데이터	KITTI 00, 0~199, 동일 인덱스 이미지/LiDAR
VO	ORB 2500, match limit1000, RANSAC1px/prob0.999, seed7+frame
ratio variant	cross-check 대신 KNN d1<0.75d2
LO	KISS-ICP1.3.0, voxel1m, range5~100m, deskew false, C++ threads4
GT	추정에는 미사용, 평가에만 사용
Metric transform	T_camera(t)=Tr × T_lidar(t) × inverse(Tr), 첫 pose 정규화
Shape 진단	각 방법에 전체 구간 GT 사용한 global Sim(3) least squares
환경	Python3.11 arm64, 연구 전용 .venv, pinned lock 저장

표 4. 기준 설정. 상세 설정 `configs/experiment.json`, 실제 KISS 설정 `artifacts/lo/effective_config.json`. GT 위치로 fit한 scale은 추정 입력이 아니라 사후 진단이며, 그 결과를 metric VO로 부르지 않는다.

기준 Git HEAD는 `c1b5ff7e3af1525061166953afb6caab855f11c8`이다. 이번 연구 코드는 아직 별도 커밋되지 않았다. 환경/설정·calibration hash는 `artifacts/evaluation/environment.json`에 저장한다. full dependency는 `configs/requirements-lock.txt`다.

레포 루트에서 실행한다. 기존 제품 .venv는 변경하지 않았다.

```
uv venv --python 3.11 research/20261031_vo-lo-study/.venv
uv pip install --python research/20261031_vo-lo-study/.venv/bin/python \
  -r research/20261031_vo-lo-study/configs/requirements-lock.txt
source research/20261031_vo-lo-study/.venv/bin/activate
export MPLCONFIGDIR="$PWD/.cache/matplotlib"
```

```
export XDG_CACHE_HOME="$PWD/.cache"
python research/20261031_vo-lo-study/src/orb_stages.py
python research/20261031_vo-lo-study/src/vo.py
python research/20261031_vo-lo-study/src/lo.py
python research/20261031_vo-lo-study/src/evaluate.py
python research/20261031_vo-lo-study/src/geometry_stages.py
python research/20261031_vo-lo-study/src/playback.py
```

PDF 생성에는 기존 `research/20261030_progress-review/src/render_pdf.mjs`에 이 연구 폴더 경로를 인자로 전달한다. 이 렌더러는 Chrome과 `/tmp/kitti-report-pdf/node_modules`의 marked/playwright/mermaid가 필요하다. PDF 생성 환경은 알고리즘 환경과 분리된다.

모듈별 산출물 계약

표의 NPZ/CSV는 실제 저장된 값이다. 모든 대표 프레임은 configs의 목록으로 고정했다.

모듈	타입·단위·좌표	저장 경로	검증
ORB 교육	images uint8, xy pixels, angles radians, bits bool	artifacts/orb/*	실제 descriptor shape와 synthetic warp label
VO	Nx2 pixel, E/R/t, masks, 4x4 camera0 arbitrary pose	artifacts/vo/baseline,ratio	finite/SE3/frame count, 실패 상태
LO	Nx3 m LiDAR/world, 4x4 pose, sigma m	artifacts/lo/*	finite/SE3, scan0 pose 정규화
Geometry	epipolar distance px, ICP residual m	artifacts/geometry/*	기록된 입력으로 직접 재계산
Evaluation	full pose, 위치 m, 각도 degree	artifacts/evaluation/*	GT=self zero error, 알려진 similarity 복원 테스트
Playback	embedded JPEG/points/trajectories	artifacts/playback/index.html	브라우저 재생/seek/mode/canvas 검사

결과: VO와 LO를 어떻게 비교했는가

Metric LO와 shape 진단을 구분

단안 VO에 실제 scale이 없으므로 raw 숫자를 미터로 간주한 RMSE를 발표하지 않는다. 먼저 LO만 시작 좌표계가 같은 GT와 직접 비교하고, 다음에는 VO·LO 모두에 동일한 전역 Sim(3) fit을 적용해 경로 모양을 비교한다. 후자는 GT를 보고 유리하게 정렬한 **사후 진단**이다.

방법 / 평가 정책	위치 RMSE m	끝점 오차 m	1-frame 상대 이동 RMSE m	상대 회전 RMSE deg
LO, 시작 좌표계만 일치	2.299	2.964	0.0387	0.0905
VO baseline, GT Sim(3)	5.336	3.772	0.2772	0.6554
VO ratio, GT Sim(3)	5.208	5.845	0.2854	0.9836
LO, GT Sim(3)	0.209	0.312	0.0385	0.0905

표 5. 같은 200프레임의 실측. Sim(3) fitted scale은 VO 0.73054, ratio0.76298, LO1.001800이다. LO의 2.299→0.209 변화는 scale만의 효과가 아니라 전역 회전/이동 정렬도 포함한다. 자세 원점 정렬 외 GT 보정을 하지 않은 수치는 첫 행이다.

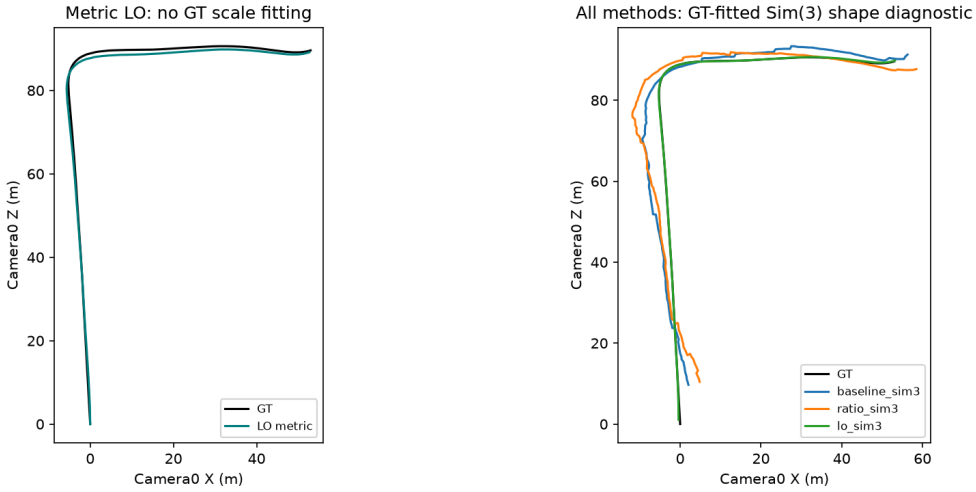


그림 13. 왼쪽은 metric LO, 오른쪽은 모든 방법의 사후 shape 정렬. 이번 LO가 이 구간에서 더 작은 오차를 보였지만 정교한 stereo/VIO/SLAM보다 낫다는 실험은 아니다.

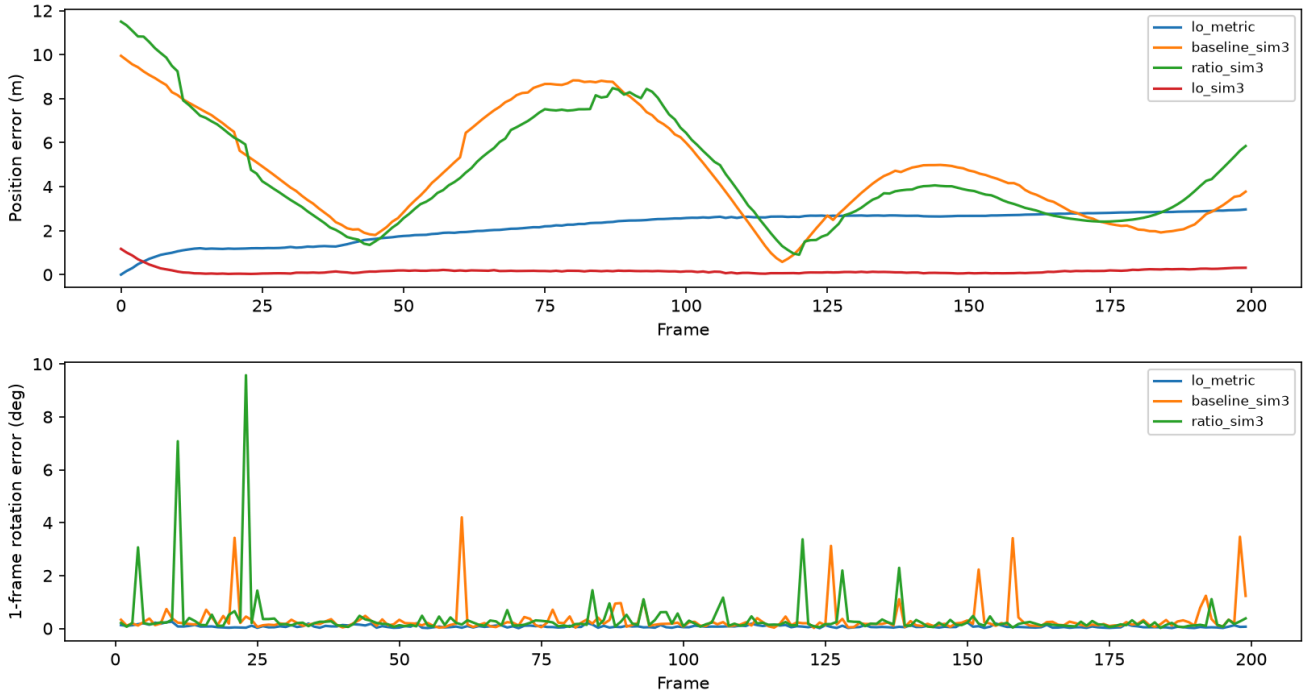


그림 14. 위치 오차와 1-frame 상대 회전 오차. 짧은 translation RPE와 전체 누적 위치 오차는 다른 성질이다. KITTI 공식 100~800m segment 평균 benchmark를 실행한 것은 아니다.

작은 해결책: ratio test의 결과

VO baseline과 ratio 모두 199개 전이에서 pose를 반환했다. pose inlier 중앙 비율은 baseline58.2%, ratio56.6%; 최저 비율은 둘 다 frame136에서 10.2%, 7.0%였다. ratio가 이 구간에서 더 좋은 관측만 골랐다고 결론 낼 근거가 없다.

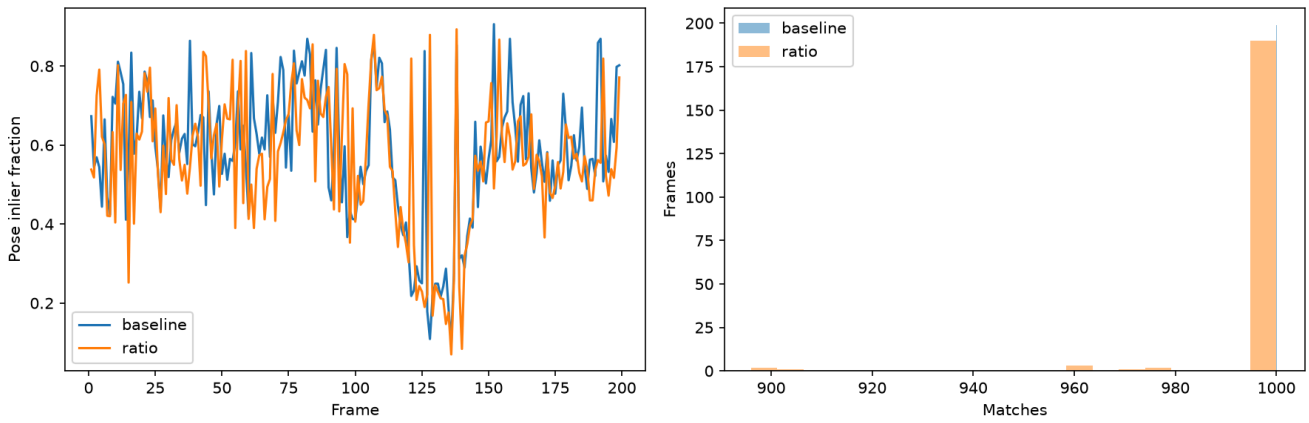


그림 15. pose inlier 비율과 매칭 수 분포. inlier 비율은 실제 정답 precision이 아니다. ratio의 위치 RMSE는 약 2.4% 낮지만 끝점/상대 이동/회전 지표는 악화됐다. 따라서 전체 개선으로 채택하지 않는다.

시간은 코드 내 추정 구간의 관측값으로 VO baseline 중앙64.1ms, ratio37.0ms, LO9.7ms였다. 영상/스캔 읽기·artifact 저장·그리기는 제외했고, 최초 실행에서 다른 연구 프로세스가 일부 겹쳤으며 thread 수와 구현 언어도 다르다. **공정한 latency benchmark나 실시간 보장은 아니다.** 시간 로그는 제공하지만 성능 순위의 근거로 사용하지 않는다.

LiDAR와 영상은 같은 것을 보고 있는가

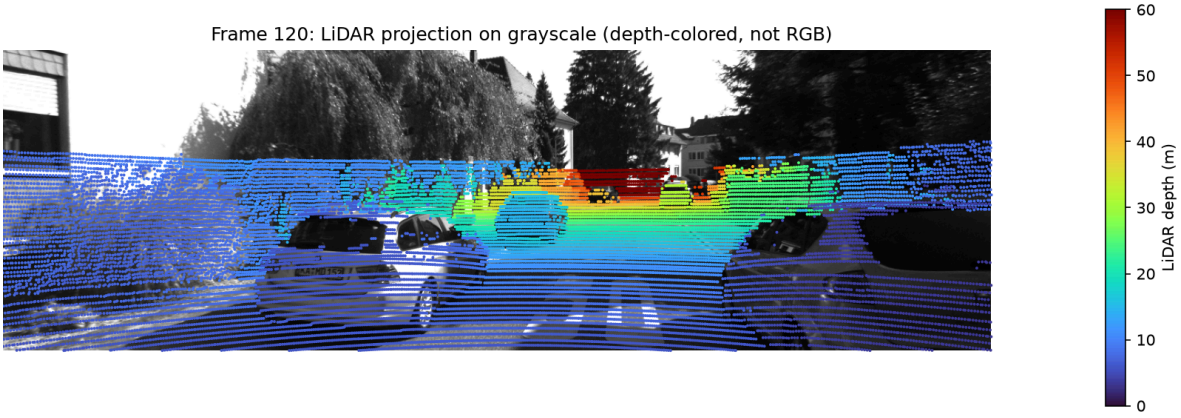


그림 16. frame120의 XYZ를 Tr/K로 영상에 투영했다. 색은 깊이이며 컬러 영상이 아니다. 이 그림은 대략적인 extrinsic 일관성 점검이고 동적 물체의 세밀한 시간 정렬을 보증하지 않는다.

VO·LO 동기 비교 재생 열기

플레이백은 raw grayscale/LiDAR 투영/ORB 대응점, 현재 스캔 BEV, GT·VO·LO 누적 궤적을 같은 frame index로 갱신한다. 재생/일시정지/이전·다음/seek/속도/metric·shape 전환을 제공한다. 브라우저에서 파일을 직접 열 수 있고 서버·인터넷이 필요 없다. 사전 계산 결과를 재생하는 것이며 online estimation UI는 아니다.

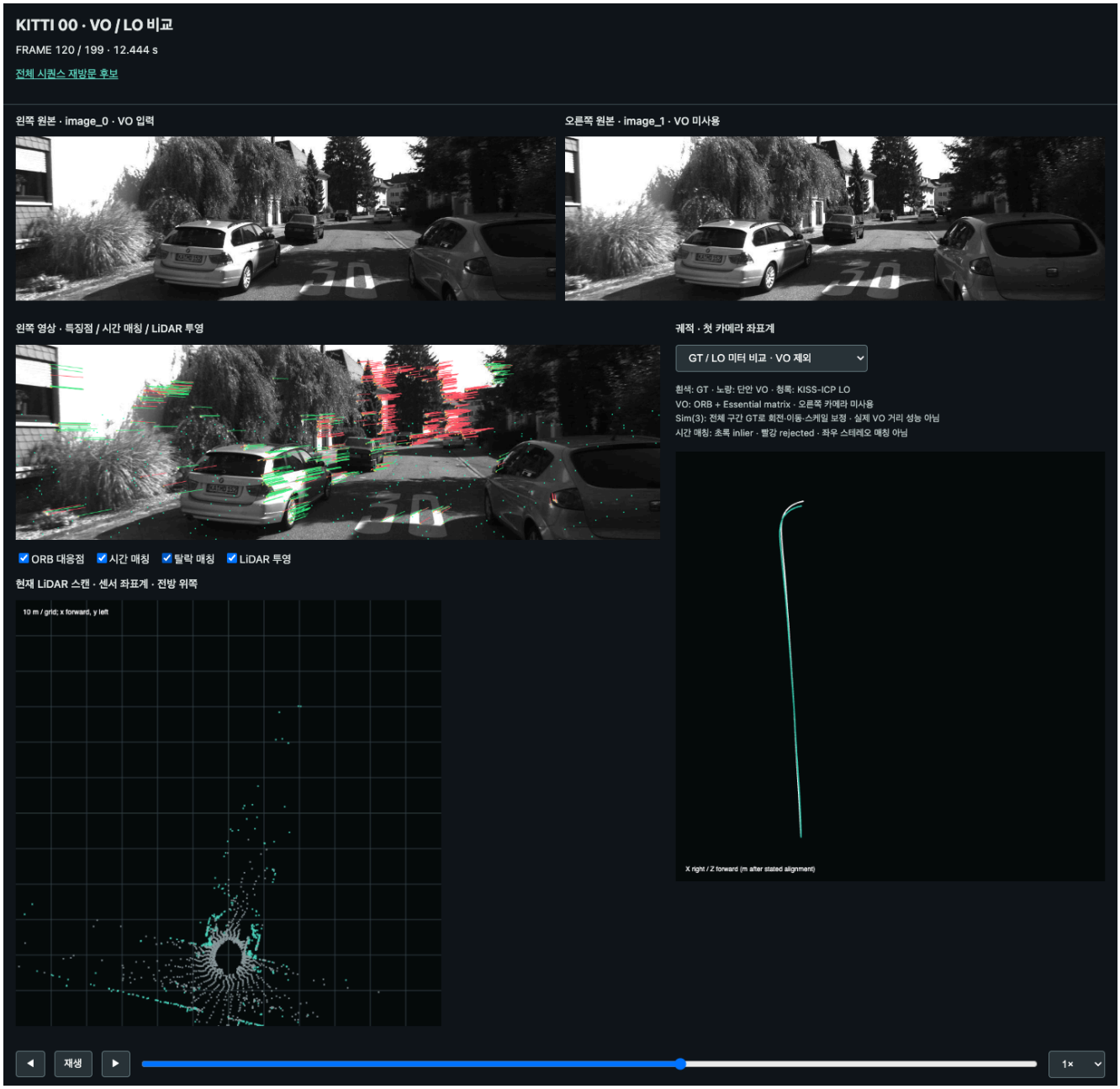


그림 17. frame120의 비교 재생 화면. 점군은 표시용으로 균일 stride 표본화했고 실제 정합 입력은 별도로 저장했다.

분석: 강점·약점과 이번에 알 수 없는 것

Feature별 강약점

관측/표현	확인한 강점	약점·한계	근거
ORB 방향+binary	합성 회전에서 정확 대응89.14%	단일 warp 실험; 3D 시점/조명/가림 일반화 불가	표1/그림6
Hamming/cross-check	작은 descriptor로 많은 대응 생성	반복 구조와 모호한 후보; 기하 검증 필요	frame136/거리 분포
ratio gate	위치 RMSE 약간 감소	상대 회전 및 끝점 악화	표5/그림15

관측/표현	확인한 강점	약점·한계	근거
LiDAR XYZ	metric 공간 제약으로 작은 상대 오차	긴 평면/동적 물체/초기값 실패 가능성 [가정]	이번 수치와 미시험 조건 구분
voxel source	원본보다 작은 정합 집합	세부 구조 소실과 voxel 편향 가능성 [가정]	그림9; size ablation 없음
intensity	파일에 존재	이번 cost에 사용하지 않아 효과 미확인	lo.py XYZ만 입력
IMU	고주파 보조 관측 가능	시간축 이상/bias 확인 전	기존 회고 보고서, 이번 미사용

실패·한계 사례

- VO의 frame136은 많은 대응이 pose 복원에 기여하지 못했다. 단순 Hamming 또는 epipolar 거리만으로 원인을 설명할 수 없다. 동일 장면에서 ratio도 악화됐다.
- 고정 단위 이동량은 차량 속도 변화를 표현하지 못한다. 전역 scale fit으로도 모든 구간의 왜곡은 없어지지 않는다.
- LO도 GT 보정 없이 누적 위치 RMSE2.299m이며 완벽하지 않다. deskew off, 공식 KITTI scan correction 미적용, 초기 회전/정합 편향 등이 후보이나 기여도를 분리한 ablation은 하지 않았다.
- KISS C++의 모든 iteration/correspondence를 dump하지 않았다. 교육용 ICP와 독립 nearest-map 잔차를 대체 관찰로 제공했고 이를 실제 내부 출력이라고 표현하지 않았다.
- 연구 VO의 실패 hold 정책과 seed/thread 설정은 과거 exporter와 완전히 동일하지 않다. 이번 비교 표는 이번 재실행끼리 비교한다.
- 순수 frame index 대응은 Odometry camera/LiDAR에 한정한다. Raw OXTS를 같은 숫자로 연결하지 않았다.
- 200프레임만으로 장거리 loop closure/재위치추정/야간/비·안개/급격한 운동을 평가할 수 없다.

해결 정도

사용자의 핵심 요청인 알고리즘 설명→단계 산출물→한계→후속 접근을 구현/그림/평가와 연결했다. VO/LO의 비교와 실제 local map도 제공한다. 완전 ORB/후속 learned 논문 재현이나 full SLAM의 구현까지 완료한 것은 아니다. 후속 연구를 설명한 뒤 자동으로 모두 구현하지 않고 다음 선택지로 남긴다.

질문에 대한 답: 컬러, IMU, SLAM

Gray 대신 컬러면 더 좋아지는가

자동으로 좋아지지 않는다. 현재 ORB는 밝기 패턴 기반이므로 컬러를 받아 grayscale로 바꾸면 색 정보를 이용하는 새 알고리즘이 되지 않는다. 같은 밝기의 다른 색을 구분하는 관측이나 semantic masking, 색을 이용한 map 정합을 설계해야 색의 효과가 생긴다.

컬러의 장점 후보는 객체 의미 분류·동적 객체 masking·색이 있는 지도다. 단점 후보는 처리량/메모리/노출·white balance 변화이며, 이 프로젝트에서 그 효과는 측정하지 않았다. KITTI의 컬러 영상은 별도 센서와 calibration을 사용하므로 grayscale 비교를 단순 채널 수 차이로만 통제하기도 어렵다. 현재 로컬에 컬러가 없어 ablation을 했다고 말할 수 없다.

다음 개선의 우선순위는 컬러 다운로드보다 **기존 우측 영상의 stereo 깊이**다. 이 판단은 현재 구조의 scale 미관측과 이미 보유한 데이터에 근거한다. 이후 실제 task가 semantic map이라면 컬러가 다른 가치를 갖는다.

IMU 데이터는 안 써도 되는가

순수 VO/LO는 IMU 없이 가능하며, 이번 실험도 사용하지 않았다. KISS-ICP의 운동 예측은 직전 LO의 변환을 활용한다. IMU가 없다는 이유로 LO가 meter scale을 잃지는 않는다. LiDAR가 이미 거리를 측정하기 때문이다.

IMU는 빠른 회전·짧은 시각 실패·scan 내 움직임을 다룰 보조 제약을 제공할 수 있지만 bias와 적분 drift가 있고 calibration/time offset/중력 처리가 필요하다. 센서를 하나 더 붙인다고 무조건 오차가 줄지 않는다. 현재 OXTS의 비양수 시간 간격5개와 긴 gap을 해결하지 않은 채 preintegration에 입력하면 오히려 잘못된 모델이 된다.

VIO/LIO는 별도 연구 단계다. 입력은 가속도/각속도이며 OXTS의 GPS/INS 위치·자세를 섞으면 GT 누출 우려가 있다. MLE/MAP, 공분산, residual weighting은 그 융합에서 관측의 신뢰도를 표현하는 도구가 된다.

KITTI Odometry로 SLAM 연구는 못하는가

SLAM 연구도 할 수 있다. 데이터셋 이름이 알고리즘을 odometry로 제한하지 않는다. 연속 영상/스캔으로 pose와 지도를 만들고 keyframe/BA/loop closure/재위치추정을 연구할 수 있다. 공식 페이지 이름도 Visual Odometry / SLAM Evaluation이다.

다만 평가에서 주어지는 주된 GT는 궤적이다. 지도 정확도를 정량 평가하려면 별도 기준 map·평가 방법이 필요하다. loop closure 효과를 보려면 실제 재방문이 있는 충분히 긴 구간이 필요하며 이번 0~199프레임은 그 실험이 아니다. SLAM에 loop closure가 정의상 항상 필수인 것은 아니지만 전역 일관성·장기 drift를 다루는 대표 기능이다.

이번 rolling local map은 LO의 scan-to-map 추정을 지원한다. 지도 그림이 있다는 사실만으로 loop-closed SLAM을 구현했다고 하지 않는다. KITTI 전체 00을 활용한 장기 지도/재방문 연구는 다음 단계로 분리한다.

개선 방향과 다음 단계 선택

우선순위	다음 연구	기대 효과 [가정]	공수 [가정]
1	Stereo 깊이→PnP VO, 같은 LO/GT와 metric 비교	단안의 scale 한계를 직접 해소	중간
2	LO deskew/scan correction/voxel 및 긴 구간 ablation	이번 LO의 잔여 오차 원인 분리	중간
3	ORB vs learned feature/matcher 비교	실제 모호한 관측에서 개선 여부 확인	중간~높음
4	IMU 시간축/축/노이즈 검증 후 VIO 또는 LIO	고속·관측 약화 조건의 융합 준비	높음

표 6. 모두 사용자 선택을 위한 제안이다. 실험하지 않은 효과를 보장하지 않는다. 기존 src 통합이나 자동 리팩터링은 하지 않았다. 교육 순서는 관측→대응→잔차→최적화→누적 오차→추가 센서/지도 제약으로 이어진다.

부록

가정과 미확인

- [가정] 정량 목표가 지정되지 않아 동일 구간 baseline 비교를 사용했다. production latency 목표는 없다.

- [가정] 컬러·IMU·grid·stereo·learned matcher 개선 효과는 본 실험 밖이다.
- [미확인] 장거리/다른 시퀀스 일반화, 동적 객체별 GT, scan 내부 time 및 motion distortion의 기여도.
- [미확인] 공식 KITTI scan correction/deskew 적용 시 이 결과가 얼마나 바뀌는지.
- [미확인] multi-thread KISS의 다른 실행에서 bit-identical 재현. 환경/설정은 고정했으나 알고리즘 수치 결과가 bit-exact하다는 주장은 하지 않는다.

참고문헌과 읽는 순서

1. Rublee, Rabaud, Konolige, Bradski. **ORB: An efficient alternative to SIFT or SURF**, ICCV 2011. DOI: 10.1109/ICCV.2011.6126544. 공식 구현 설명. 본 보고서 4.2~4.6의 실험과 함께 읽는다.
2. OpenCV Feature Matching, calib3d. 매칭과 pose 추정의 책임을 구분한다.
3. Vizzo, Guadagnino, Mersch, Wiesmann, Behley, Stachniss. **KISS-ICP: In Defense of Point-to-Point ICP**, RA-L 2023. 저자 PDF, 공식 코드. 이번 LO 실험과 연결한다.
4. DeTone, Malisiewicz, Rabinovich. **SuperPoint: Self-Supervised Interest Point Detection and Description**, CVPR workshop 2018. 논문. feature 반복성과 descriptor 학습의 문제를 읽는다.
5. Lindenberger, Sarlin, Pollefeys. **LightGlue: Local Feature Matching at Light Speed**, ICCV 2023. 논문. 추출기와 matcher가 다른 모듈임을 염두에 둔다.
6. Engel, Koltun, Cremers. **Direct Sparse Odometry**, 2016 preprint. 논문. descriptor 대응 대신 광도 잔차를 쓰는 다른 갈래다.
7. Mur-Artal, Tardos. **ORB-SLAM2**, T-RO 2017. 논문. 특징점 이후 지도/최적화 시스템을 읽는다.
8. Zhang, Singh. **LOAM: Lidar Odometry and Mapping in Real-time**, RSS 2014. 저자 논문. edge/plane 기반의 다른 LO 설계를 비교한다.
9. Xu 외. **FAST-LIO2: Fast Direct LiDAR-inertial Odometry**, 2021 preprint. 논문. IMU와 map 정합을 결합할 때의 상태와 제약을 읽는다.
10. KITTI Odometry 공식 설명, Raw 설명. 원시 데이터/GT/동기화 범위를 확인한다.

제출 점검

- 각 설명에 실행 산출물 또는 미수행 표기를 연결했다. 단일 링크 나열로 대체하지 않았다.
- 학습 패턴을 재훈련하지 않았고 실제 OpenCV 출력과 교육용 근사를 구분했다.
- 코드·설정·중간 산출물·그림·baseline ablation·실패 사례·좌표/단위·재현 명령을 제공했다.
- Sim(3) 진단과 metric 성능을 분리했고 GT를 estimator에 넣지 않았다.
- 추가 연구와 제품 통합은 자동 진행하지 않는다.

4. SLAM과 재방문: 데이터가 있는가, 무엇이 아직 없는가

GT를 사용한 후보 분석이며 자동 loop closure 구현 결과가 아니다.

원 실험 디렉터리: `research/20261031_loop-readiness/`

문제 정의

SLAM은 위치와 지도를 함께 추정하는 문제다. 루프 클로징은 재방문으로 누적 오차를 줄이는 구성 요소이지 SLAM의 필수 정의는 아니다. KITTI Odometry라는 이름이 SLAM 연구를 금지하거나 데이터 사용을 오도메트리로 제한한다는 뜻은 아니다.

현재 baseline은 단안 ORB VO와 KISS-ICP LO의 00 시퀀스 첫 200프레임 결과다. VO에는 지도·BA·루프가 없고, LO의 rolling local map은 전역 루프 최적화 지도와 다르다. 이번 입력은 00 전체 GT poses/times, 실제 왼쪽 영상, 기존 VO/LO 산출물이다. 출력은 후보 CSV/JSON, 영상쌍, 비교 재생 HTML, 본 PDF다.

성공 기준: [가정] 500프레임 이상 떨어지고 중간 이동 100m 이상, GT 거리 5m 이하, 상대 회전 30도 이하인 후보가 최소 1쌍 존재할 것. 재생/탐색/정렬 모드 전환과 두 화면 이동에 브라우저 오류가 없어야 한다. 조건은 공식 벤치마크 규격이 아니라 이번 데이터 탐색용 설정이다.

범위 밖: 전체 구간 odometry 재실행, 자동 place recognition, pose graph 최적화, VIO/LIO, 운영 파이프라인 통합. 실제 루프 성능 기준은 이후 별도 정의해야 한다.

사용 가능한 리소스

리소스	확인 내용	부족한 것
데이터	00 grayscale stereo와 LiDAR 각각 4,541프레임, GT	자동 루프 정답 라벨
기존 VO	ORB, Hamming 매칭, Essential RANSAC, recoverPose	metric scale, BA, 지도
기존 LO	KISS-ICP 1.3.0, metric 상대 이동 및 local map	전역 그래프, 루프 검출
Feature	ORB descriptor, LiDAR XYZ/intensity	장소 검색 descriptor 및 검증 파이프라인
환경	연구용 Python venv, SciPy, OpenCV, Matplotlib, Chrome 검사	일반화 평가 및 전체 구간 실험

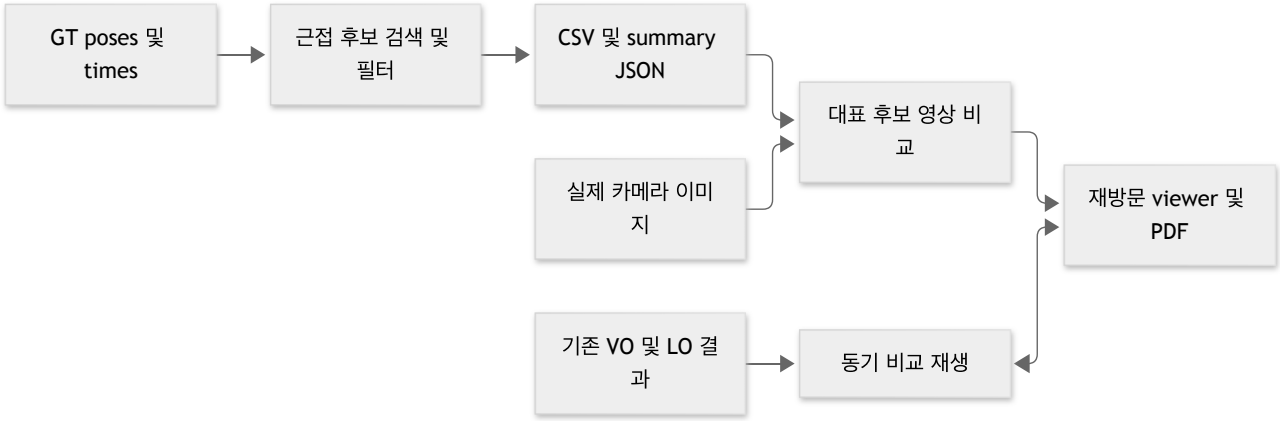
상세 인벤토리는 원 실험의 notes/inventory.md에 있다. VO/LO 알고리즘 단계 설명과 이미지는 이 문서의 2~3장을 참조한다.

접근 방법 선정

Quick-win으로 GT 위치의 cKDTree 근접 검색을 사용했다. 이는 '이 데이터에 재방문이 있는가'에 답할 수 있으므로 새 검출 알고리즘이나 선행연구 비교 실험은 수행하지 않았다. GT를 사용했기 때문에 실제 로봇의 loop detector로 사용해서는 안 된다.

근거 코드: src/audit.py:28 근접 검색, :31 시간·이동·회전 필터, :38 대표 선택, :63 실제 영상 읽기. 후보 수는 인접 프레임 조합을 포함하므로 11,772개의 독립 루프를 의미하지 않는다.

시스템 아키텍처



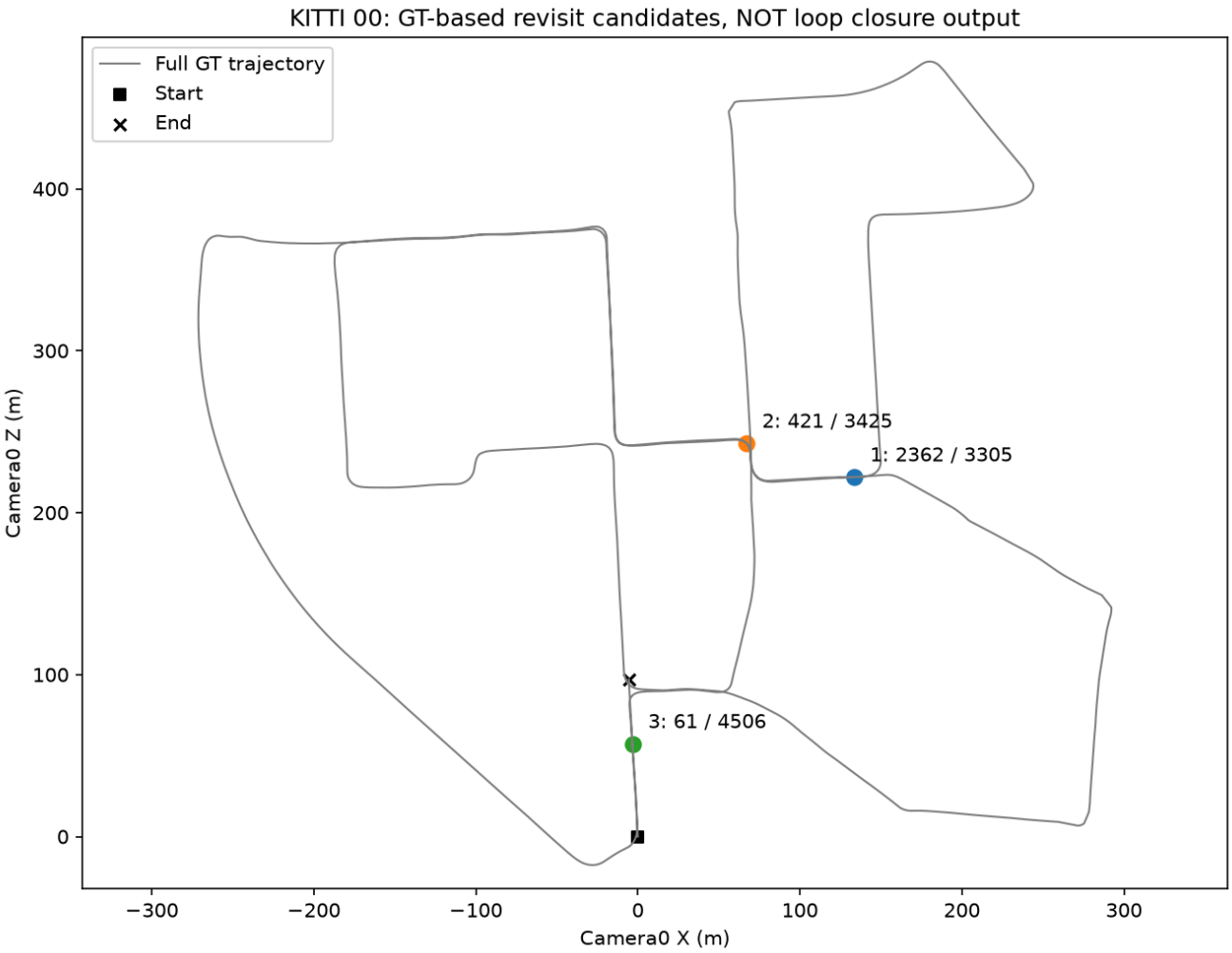
모듈	입력 / 좌표계 / 단위	출력 계약	검증
후보 검색	GT Nx3 위치, 첫 카메라 좌표계, m	artifacts/candidates.csv	유한값, 시간 단조 증가, 필터
대표 선정	후보와 GT 회전, 초/도	artifacts/summary.json	40m 이상 공간 분리
영상 비교	uint8 image_0 PNG, 프레임 인덱스	figures/pair_*.png 및 artifacts/loops/index.html	이미지 decode와 프레임 표시
동기 재생	기존 200프레임 영상·LiDAR·poses	별도 study의 artifacts/playback/index.html	play/seek/mode, canvas, 모바일

LiDAR pose는 기존 연구에서 Tr로 카메라 좌표계로 변환했다. 본 후보 검색은 GT 좌표만 사용하며 추가 센서 동기화나 단위 변환은 없다. 생산 코드 변경 없이 연구 디렉터리와 VSCode 실행 설정을 추가했다.

결과

후보 프레임	GT 거리(m)	시간 간격(s)	중간 이동(m)	상대 회전(도)
2362 / 3305	0.031	97.7	800.4	0.94
421 / 3425	0.119	311.4	2348.8	6.11
61 / 4506	0.233	460.7	3627.0	0.37

표는 GT 근접 후보이며 검출 정확도나 보정 성능이 아니다. 전체 이동은 3724.2m, 시작-끝 거리는 97.2m다.



전체 경로 중 재방문 후보의 위치를 표시한다. 추정 궤적이 아니다.



2362/3305의 실제 영상으로 건물·차량·도로 구조가 반복되는 장면을 확인할 수 있다.



421/3425를 나란히 표시하여 시점 및 장면 차이를 확인한다.



61/4506의 실제 영상 비교다. 첫 200프레임만으로는 이 재방문을 관측할 수 없다.

기존 첫 200프레임 평가 비교:

방법 / 정렬 정책	위치 RMSE(m)	해석
ORB VO / GT 전역 Sim(3)	5.336	scale을 GT로 맞춘 사후 궤적 형상
ratio VO / GT 전역 Sim(3)	5.208	작은 개선이나 다른 지표 악화, 우월성 결론 불가

방법 / 정렬 정책	위치 RMSE(m)	해석
KISS-ICP LO / GT 전역 Sim(3)	0.209	동일 형상 비교 조건
KISS-ICP LO / 시작점만 정렬	2.299	metric 오차, 위 Sim(3) 수치와 구분

이 표는 이전 연구 산출물을 재사용하며 전체 KITTI 평가나 loop closure 효과가 아니다. 방법·평가 코드는 ../20261031_vo-lo-study/src/lo.py 및 evaluate.py다.

분석

Feature 강점/약점

Feature	강한 조건	약한 조건	근거
GT 위치 근접	재방문 데이터 존재 확인	실사용 시 GT 없음	위 후보 3쌍
상대 회전 필터	비슷한 시점 영상 선별	반대 방향 루프 제외	30도 제한
실제 grayscale 영상	반복 구조 정성 확인	조명, 동적 물체, 유사 장소	pair_1~3
LiDAR 기하	metric odometry 비교	반복 구조, 움직이는 물체	이전 LO 보고서; 이번에는 loop 검증 미실시

알고리즘의 약점 및 실패 사례

GT 기준의 좋은 사례만 선정했으므로 자동 검색의 false positive/false negative는 측정하지 않았다. 가까운 GT 위치라도 가림이나 시선 차이로 정합이 실패할 수 있다. 프레임 간 연관으로 후보 개수는 크게 부풀 수 있다. 공식 KISS KITTI loader의 특수 scan correction과 deskew를 적용하지 않은 연구 설정이라는 한계도 유지된다.

문제 해결 정도

데이터 존재 기준은 대표 3쌍으로 달성했다. Chrome 검사에서 재생, 120번 프레임 탐색, 정렬 모드 변경, 후보 선택, 양방향 화면 이동을 통과했고 JavaScript 오류는 0건이었다. 390px 모바일에서 가로 넘침이 없고 영상 3개가 정상 decode되었다. camera/BEV/path canvas의 비영 픽셀은 각각 466611/34435/3076개였다. 단, 실제 loop 검출·보정의 성공은 미평가다. 비교 재생은 원시 센서와 추정 궤적을 관찰하는 도구이며 실시간 온라인 계산기가 아니다.

개선 방향

보완 항목	기대 효과	예상 공수
전체 00 LO baseline	루프 전 누적 오차 측정	중간
장소 검색 descriptor + 기하 검증	GT 없이 후보 검색 및 오검출 배제	높음
pose graph 최적화	검증된 루프를 통한 전역 보정	높음
추가 시퀀스 / 반대 시점 평가	선택 편향 확인	중간

다음 단계 제안

1. 00 전체 LO와 지도 누적을 실행해 baseline을 확정한다.
2. LiDAR 루프 검출 방법을 조사·선정하고 후보 검색과 정합 검증을 각각 평가한다.
3. 검증된 루프 제약으로 pose graph를 최적화하고 보정 전후 재생을 비교한다.

부록

- [가정] 5m/500프레임/100m/30도/40m 분리 조건은 탐색 목적의 임시 조건이다.
- [미확인] GT 없는 자동 루프 precision/recall, 전체 구간 LO 정확도, loop 최적화 개선량.
- 근거: 로컬 GT 및 실제 영상, src/audit.py, artifacts/summary.json, artifacts/candidates.csv. 본 작업은 새로운 논문 성능 수치를 인용하지 않는다.
- 관련 알고리즘 원문과 단계별 실험은 이 문서 2~3장의 참고문헌 및 본문 참조.

5. 좌우 카메라·VO·LO 통합 재생

오른쪽 영상은 표시용이다. 단안 VO를 stereo VO로 바꾸지 않았다.

원 실험 디렉터리: `research/20261101_unified-playback/`

문제 정의

입력: KITTI 00 image_0/image_1/velodyne 및 기존 200프레임 VO/LO 산출물. 출력: 하나의 offline HTML. [가정] 성공 기준은 200개 오른쪽 영상의 존재/크기 검증, 다섯 canvas의 비영 픽셀, 재생/탐색/모드 전환 성공, 모바일 가로 넘침 0건, JavaScript 오류 0건이다. 알고리즘 개선 및 전체 시퀀스 추정은 범위 밖이다.

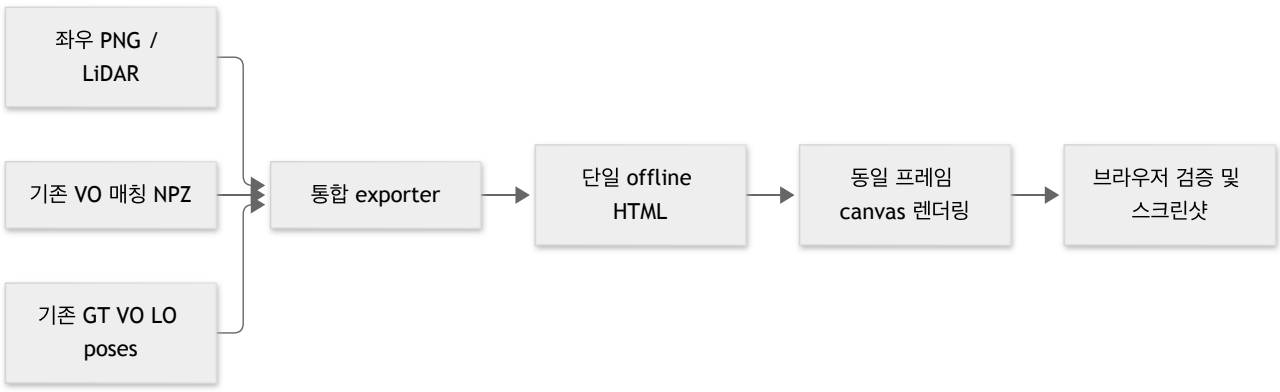
리소스

기존 src/viz/export_playback.py는 VO 전용이고, research/20261031_vo-lo-study/src/playback.py는 VO/LO 결과를 조합한다. 후자를 확장했다. image_1은 이미 로컬에 있다. 시간 매칭의 points_previous/current 및 pose_mask도 기존 NPZ에 존재한다. 새 센서 다운로드나 추정 실행은 필요 없다.

접근 선정

Quick-win: 기존 산출물을 재사용해 renderer/exporter만 확장한다. 알고리즘 개발이 아니므로 선행연구 조사와 신규 알고리즘 실험은 수행하지 않았다. 기존 생산 VO 코드는 변경하지 않는다.

아키텍처



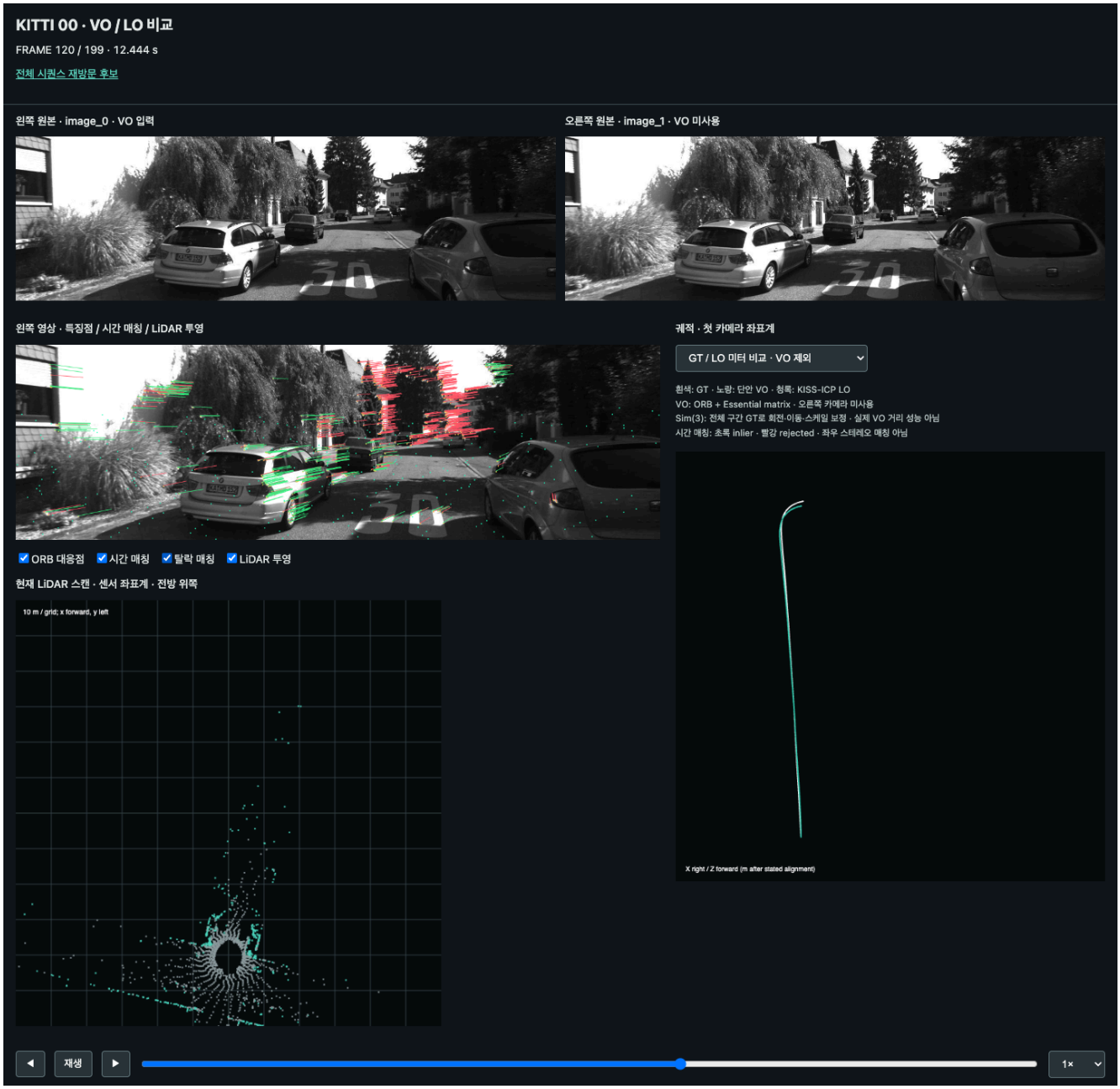
원본 영상은 uint8 grayscale이며 JPEG로 HTML에 내장한다. 매칭은 이전/현재 왼쪽 영상의 pixel 좌표와 pose inlier flag다. LiDAR는 센서 좌표 m, 궤적은 기존 평가 좌표계를 그대로 사용한다. 동일 프레임의 두 영상 decode가 끝난 후 전체 화면을 갱신하고 이전 비동기 렌더 요청을 version으로 폐기한다. HTML 산출물은 ../20261031_vo-lo-study/artifacts/playback/index.html, 스크린샷은 해당 study의 figures/playback_*.png다.

결과

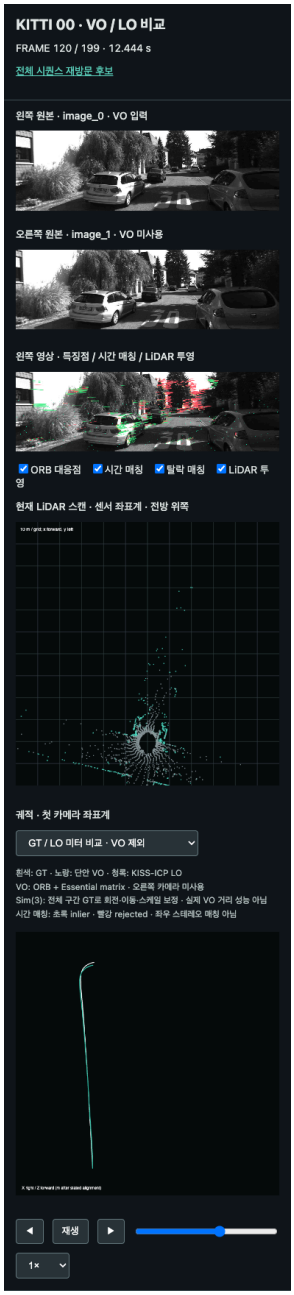
항목	기존 연구 화면	통합 화면
카메라 원본	왼쪽 overlay만	왼쪽/오른쪽 원본 + 별도 왼쪽 overlay
시간 매칭	현재 좌표 점만	이전→현재 선, inlier/rejected 선택
LiDAR	투영/BEV	유지
GT/VO/LO	기본 Sim(3)	기본 metric GT/LO, 선택형 Sim(3)

표는 표시 기능 차이이며 VO 정확도 향상을 의미하지 않는다. 기존 baseline VO의 GT-fitted 위치 RMSE 5.336m도 그대로다. ratio 실험의 5.208m 결과가 기본 VO로 교체된 것이 아니다. 실제 오른쪽 영상을 읽지 못하거나 크기가 다르면 export는 실패한다.

검증 결과: 200프레임 export 성공. 120번 프레임에서 좌/우/overlay/BEV/궤적의 비영 픽셀은 466610/466609/466611/34435/3076개로 모두 비어 있지 않았다. 좌우 픽셀 배열이 다름을 검사하여 복제 영상이 아님을 확인했다. 재생, 탐색, 양쪽 궤적 모드, 후보 화면 왕복, 모바일 390px 가로 넘침 검사 통과. JavaScript 오류 0건. 이 검사는 센서 시간 offset의 독립 검증이나 알고리즘 정확도 검사가 아니다.



좌우 원본, 특징점/매칭 overlay, LiDAR, 궤적을 동일 프레임으로 표시한다.



좁은 화면에서는 좌우 영상을 세로로 배치한다.

분석

관측	강점	한계
좌우 원본	실제 시차 비교	stereo depth/VO 미구현
시간 매칭	움직임과 outlier 관찰	3D 지도 아님
Sim(3) 궤적	형상 비교	GT 이용, metric 성능 아님
metric LO	GT scale fitting 없는 비교	현재 200프레임만

기존 VO 전용 화면은 단위 이동량과 GT 미터를 그대로 비교했다. 연구 화면은 전역 GT 정렬을 하므로 둘의 시각적 차이를 알고리즘 개선으로 해석하면 안 된다. 두 코드 버전의 모든 차이에 대한 controlled benchmark는 이번 범위가 아니다. 전체 HTML 크기가 증가하며 저사양 장치에서 메모리 사용은 미측정이다.

개선 방향

추후 stereo VO를 연구하면 오른쪽 영상을 실제 깊이 추정에 사용할 수 있다(공수 높음). 장시간 재생에서는 파일 분할/지연 로딩이 필요할 수 있다(중간). 이번에는 구현하지 않았다.

다음 단계

1. 현재 통합 화면에서 매칭과 GT 정렬 방식 차이를 학습한다.
2. 별도 연구로 stereo VO와 단안 VO의 metric 성능을 비교한다.

부록

[가정] 200프레임과 기존 추정 유지가 이번 UI 통합 범위다. [미확인] stereo 추정 성능, 장시간 메모리 사용. 출처는 로컬 exporter, evaluate.py의 sim3(), 기존 metrics.json이며 외부 논문 수치를 추가 인용하지 않는다.

6. 초기 데이터·IMU 감사 기록과 과거 baseline

이 장만은 초기 상태의 보관 기록이다. 여기의 LO 미구현·ATE 미측정·옛 main 코드 지적은 당시 상태이며 현재 상태가 아니다. 현재 LO 구현 및 측정값은 3장, 현재 재생은 5장이 우선한다. 과거 실패·미확인 사항을 삭제하지 않기 위해 보존했다.

원 실험 디렉터리: `research/20261030_progress-review/`

사용 가능한 리소스

표는 이번에 확인한 리소스 요약이다. 전체 목록과 코드 위치는 인벤토리에 있다.

리소스	현재 상태	제약
Stereo grayscale	00~21, 00 좌우 각 4,541장	VO는 image_0만 사용
GT	00~10 pose 파일	평가용, 추정 입력으로 사용하지 않음
LiDAR	Odometry 00의 4,541개 XYZI 스캔	다른 시퀀스 미추출; 추정 알고리즘 미구현
OXTS	45,700행 × 30열, 전 값 finite	IMU 외 GPS/INS 항목도 포함; 시간축 이상 존재
Calibration	P0~P3/Tr, Raw 세 calibration 파일	센서간 변환 체인 실측 검증 전
플레이백	원본/ORB/매칭/GT·VO 동기화, seek/step/speed	계산 후 재생; 실시간 VO 처리 성능 아님
Feature	ORB 위치/descriptor, Hamming distance, inlier mask	3D landmark나 semantic label 아님
부족한 것	공통 평가기, full pose log, stereo/VIO backend	ATE/RPE/latency 결과 아직 없음

데이터는 `data/kitti/dataset`, `data/kitti/raw`, 원본 ZIP은 `data/kitti/archives`에 있다. 이전 단계에서 Downloads의 세 ZIP을 이동하고 LiDAR 00과 OXTS·timestamp·calibration만 추출했다. Raw 영상/Raw LiDAR 중복 추출은 하지 않았다. 이 이력은 `data/kitti/SENSOR_IMPORT.md`와 이전 실행 기록에 근거한다.

결과

아래 표는 **기존 baseline 보관 결과의 재집계**이며 개선 방법과의 A/B 실험은 아니다.

지표	기존 결과 재집계	개선 알고리즘
프레임 / 전이	200 / 199	미구현
tracked 상태	199 / 199 전이	미평가
ORB 개수	모든 프레임 2,500	미평가
pose inlier 비율 최소/중앙/최대	10.2% / 58.2% / 90.6%	미평가
GT 누적 이동 거리	144.879m	비교 참조
VO 누적 이동량	199 임의 단위	미평가
GT 1프레임 이동	0.371~1.060m	비교 참조

지표	기존 결과 재집계	개선 알고리즘
VO 1프레임 이동	약 1 고정	미평가
ATE/RPE/heading error/latency	미측정	미평가

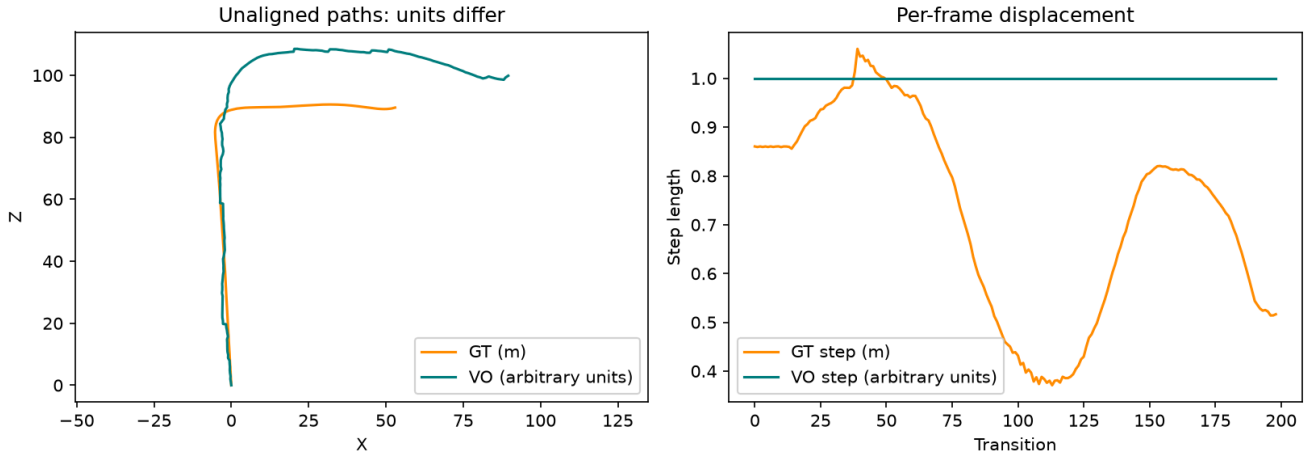


그림 1. 스케일 미정렬 궤적과 이동량. 서로 다른 단위이므로 곡선 차이를 미터 오차로 읽으면 안 된다. GT의 이동량 변화에 비해 VO의 길이는 고정된다.

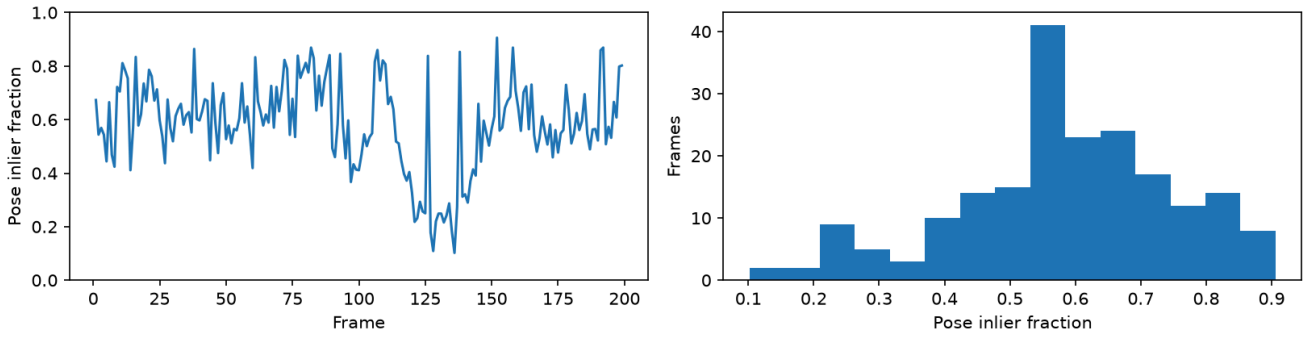


그림 2. 매칭 중 pose 복원 후 살아남은 비율. 정답 대응 label이 없으므로 precision/recall은 아니다.

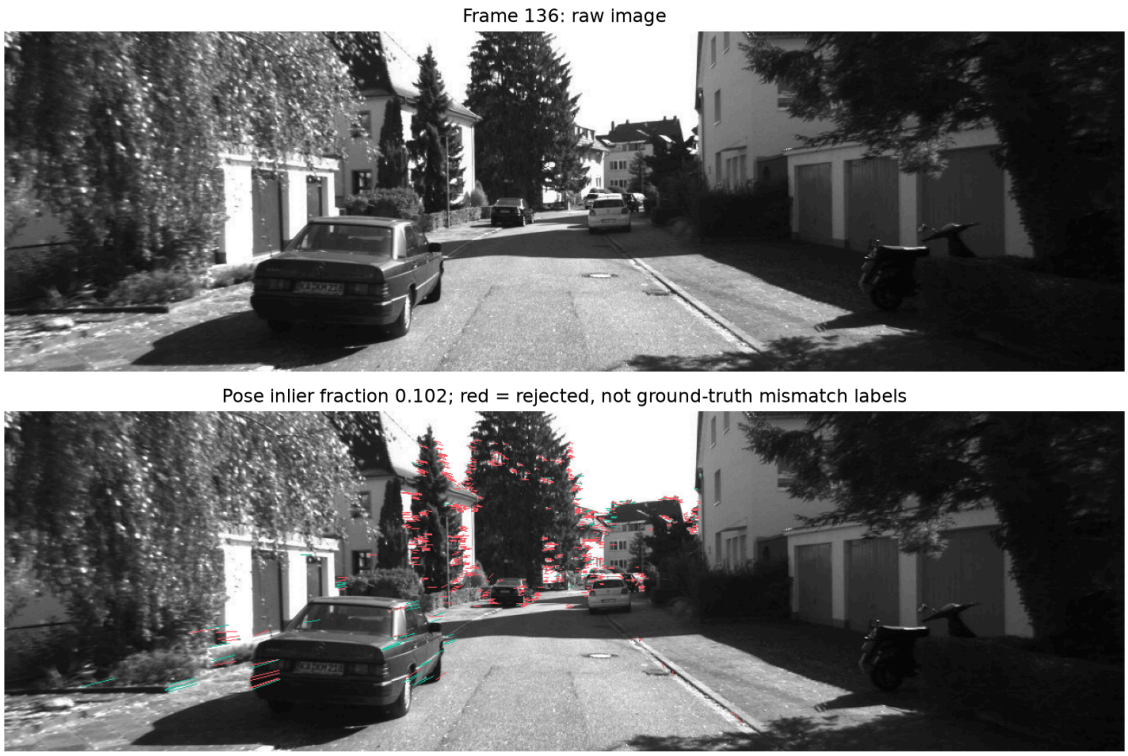


그림 3. 구간 내 최저 inlier 비율인 frame 136. 빨간 선은 기하/cheirality 검사에서 제외된 대응이며 실제 오매칭임을 보증하지 않는다.

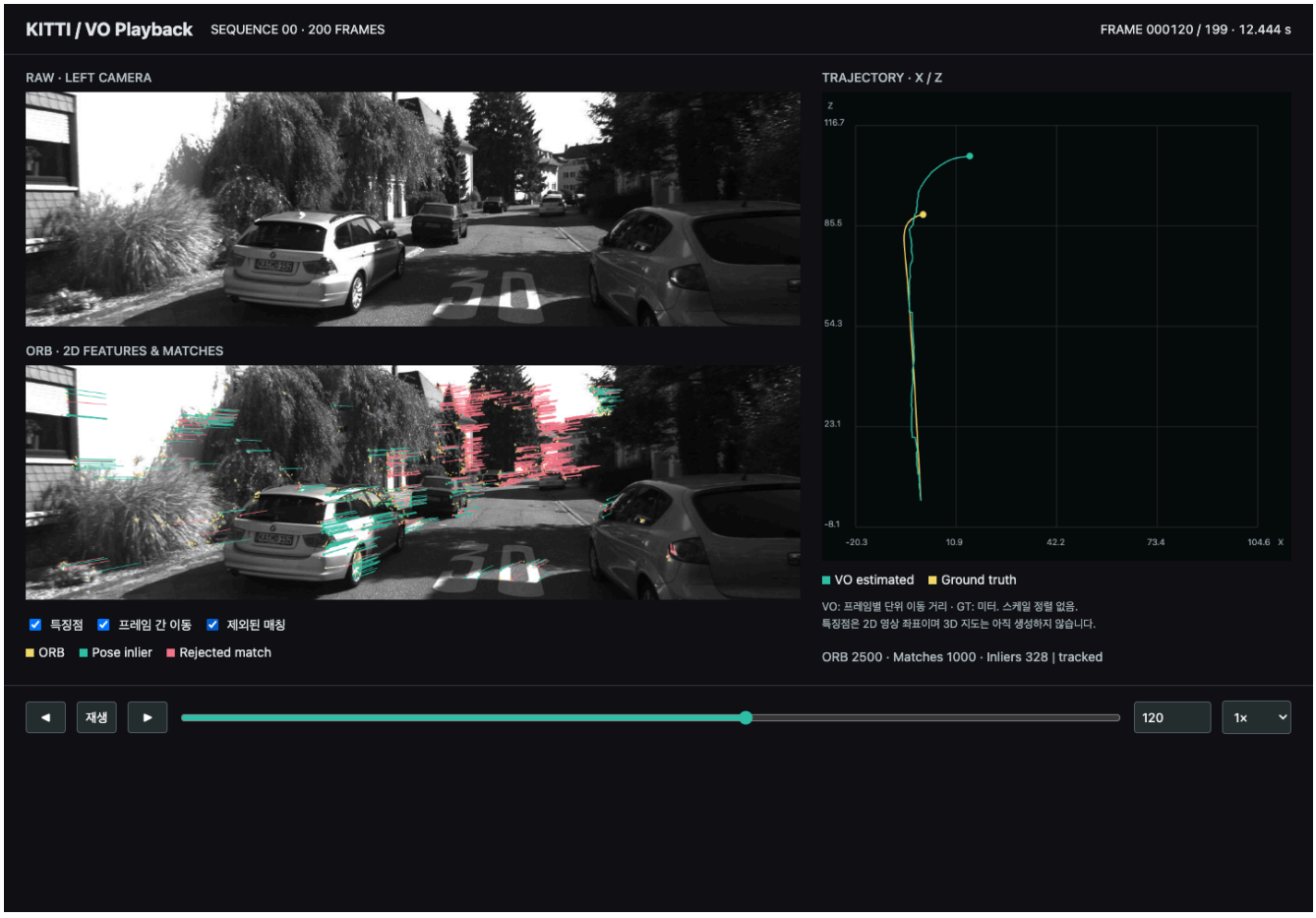


그림 4. 이전에 저장한 데스크톱 UI(frame 120). 원본 영상/특징점/궤적의 동기 관찰이 가능하다. 과거 Playwright 재생·탐색·모바일 검증은 대화 실행 기록에 근거하며 이번에는 브라우저 검증을 다시 실행하지 않았다.

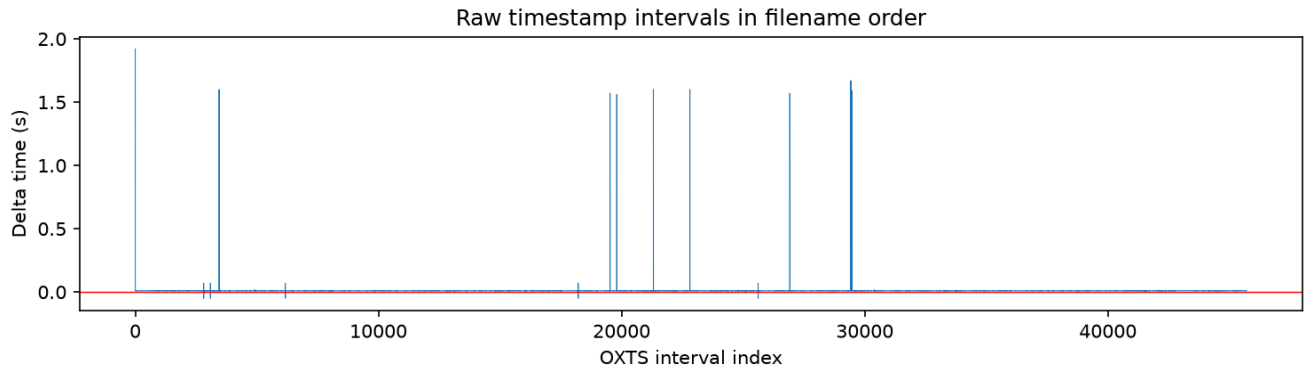


그림 5. OXTS 파일명 순서에서의 timestamp 차이. 중앙값 0.009996736초, 최소 -0.050010872초, 최대 1.919595343초, 비양수 간격 5개다. 원인은 아직 확인하지 않았다.

모듈 검증: 200프레임 파싱/유한 위치/영상 timestamp 단조 증가/inlier mask 일치 통과. IMU 45,700×30 유한값 및 timestamp 개수 일치 통과. **IMU 시간 단조성은 미달**이다. LiDAR 이미지 번호 일치와 첫/끝 XYZI 디코딩은 앞선 import 검증에서 통과했으며 전체 스캔 geometry 품질 검증은 아니다.

분석

Feature 강점/약점

Feature	강한 조건 또는 관찰	약한 조건 또는 한계	근거
ORB 위치	이번 구간에서 2,500개를 지속 확보	개수가 많아도 분포/독립성/정확성 보장 안 됨	frames.csv; 32칸 중 최소 13칸 점유
Binary descriptor/Hamming	실제 실행에서 프레임 대응 생성	반복무늬·시점/조명 변화 취약 가능성 [가정]	코드 및 그림 3; 거리별 GT 분별력 미측정
Pose inlier mask	기하 일관성에 따른 후보 분리	비율 변화 큼, 정확도/정답 label 아님	그림 2, 최소 10.2%
Stereo disparity	metric 깊이 추출 가능성	원거리·작은 disparity 취약 예상 [가정]	P0/P1와 우측 영상 존재; 아직 미추출
LiDAR XYZI	공간 좌표 관측 가능	동적 객체·희소성 영향 [미확인]	데이터 확보만 완료, 알고리즘 실험 없음
IMU 가속도/각속도	약 100Hz 시간 간격의 측정 존재	시간 역전/간격 이상, bias/noise 미검증	imu.json, 그림 5

표는 실측 관찰과 가설을 구분한다. Feature ablation, 실제 match label에 따른 분포 분리도, 오차 상관관계는 **미수행**이다. 2D 특징점 시각화를 3D 지도라고 부르지 않는다.

알고리즘의 약점 및 실패 사례

1. **Scale 부재:** 서로 독립적인 두 프레임 추정에서 t 방향만 얻어 길이 1로 누적한다. 이는 단순한 전역 scale 차이뿐 아니라 구간별 속도 변화에 따른 형상 왜곡을 만들 수 있다. 그림 1과 runner:40이 근거다.

2. **낮은 inlier 구간:** frame 136은 1,000개 중 102개만 pose inlier다. 추정 상태는 tracked지만 정답 자세가 맞는지는 별도 평가하지 않았다. 원인을 차량 운동/오매칭/작은 시차 중 하나로 단정하지 않는다.
3. **main의 실패 처리:** 매칭 부족 시 frame을 생략하고 GT를 단순 앞부분 slicing한다(`run_kitti.py:30`, `:51`). 실제 skip이 생기면 frame 대응이 어긋날 수 있다. 이번 보관 구간의 실제 skip 사례는 없다.
4. **추정 구현 중복:** exporter는 자체 E/pose 처리와 최소 inlier gate를 갖고 runner와 동작이 다르다. main의 inlier 집계 오류도 남아 있다. 두 실행 결과를 동일 구현으로 무조건 취급하면 안 된다.
5. **IMU 입력 이상:** 파일 순서와 시간 순서가 일부 다르다. timestamp만 정렬하면 해결되는지, packet 순서/중복/누락 문제인지 확인 필요. 측정값·시각 대응을 깨뜨리는 임의 정렬은 하지 않았다.
6. **다운로더 재개:** feature 코드 `scripts/download_kitti_odometry.py:32` 부근은 Range 응답이 206인지와 Content-Range를 검증하지 않고 append한다. 서버가 Range를 무시하면 손상 가능성이 있다. 이번 다운로드에서 실제 손상이 있었다는 뜻은 아니다.

문제 해결 정도

데이터 확보와 시각 디버깅, 관측 집계까지 완료됐다. metric VO 개선 여부, VIO/LIO 성능은 평가할 단계가 아니다. 00의 약 200프레임 결과만으로 전 시퀀스/고속/야간/동적 객체 성능을 일반화할 수 없다. 목표 성능 수치를 과거에 정하지 않았으므로 임의의 달성률은 제시하지 않는다.

부록

가정 목록

- 다음 성능 연구는 동일 조건의 baseline 대비 개선으로 목표를 시작한다. 구체적 임계값은 아직 없다.
- Stereo/PnP의 metric scale 확보, grid/gate/KLT의 품질 효과와 예상 공수는 설계 판단이며 실험 결과가 아니다.
- 시점/조명 변화나 작은 disparity의 영향은 일반적 가설이며 이 데이터에서 ablation하지 않았다.

미확인 사항

- IMU 비단조 시간/긴 gap 원인 및 Raw↔Odometry 정확한 시간 대응.
- IMU noise/bias, field별 적용 축, 동적 객체 GT, LiDAR 전체 스캔 품질.
- ATE/RPE/회전 오차, latency, 메모리, 다른 시퀀스 일반화 성능.
- 기존 HTML 생성의 정확한 환경/난수 상태/커밋 provenance; 이번 hash로 현재 파일만 식별.
- 원본 대용량 ZIP 전체 재검증은 이번 회고에서 하지 않았다.

참고문헌

1. OpenCV ORB 공식 설명 — Rublee 외, ORB, 2011의 개념 및 API 배경.
2. OpenCV Feature Matching — Hamming/cross-check 동작.
3. OpenCV calib3d — Essential, recoverPose, PnP API.
4. KITTI Odometry — 시퀀스/GT와 공식 segment 평가 정의. 이번에는 공식 100~800m 평가를 실행하지 않았다.
5. KITTI Raw — 센서/보정/동기화 형식.
6. Mur-Artal, Tardos, ORB-SLAM2, arXiv 2016 / T-RO 2017.
7. Qin, Li, Shen, VINS-Mono, arXiv 2017 / T-RO 2018.

제출 점검

- 회고 기준·baseline 재집계·그림 설명·모듈 산출물·코드 근거·재현 명령: 포함.
- 새 알고리즘 A/B 개선, feature ablation, full pose 정확도 평가: 미수행으로 표시.
- 과거 시스템의 E/R/descriptor 중간 산출물: 미보관 사실을 명시.
- 프로젝트 지침: 루트 CODEX.md에 연구 워크플로우 기록, AGENTS.md에서 참조.

7. 재현 및 문서 갱신

최신 문서 갱신 절차

1. docs/report.md의 해당 주제를 직접 수정한다. 새 날짜별 보고서를 만들지 않는다.
2. 같은 문서 맨 위 버전과 아래 변경 이력을 함께 갱신한다. 오탈자는 patch, 설명/결과 확장은 minor, 구조·평가 계약의 큰 변경은 major를 올린다.
3. 그림은 실험 디렉터리의 실제 산출물을 상대경로로 연결하고 출처·설정·코드 버전을 기록한다.
4. 다음 명령으로 docs/report.pdf를 덮어쓴다. 원본과 PDF를 같은 기능 커밋에 포함한다.

```
node tools/render_report.mjs
```

초기 통합에 사용한 tools/consolidate_reports.mjs는 일회성 이관 도구다. 최신 원본이 있으면 실패하도록 되어 있으며 평소에는 실행하지 않는다. 각 장에 보존한 옛 PDF 생성 명령은 과거 재현 정보일 뿐 현재 문서 갱신에는 사용하지 않는다.

알고리즘 산출물 재현

```
# 기존 연구 환경과 데이터가 있을 때 전체 VO/LO 실험
research/20261031_vo-lo-study/.venv/bin/python research/20261031_vo-lo-study/src/run.py --stage all

# 기존 결과로 통합 playback과 재방문 viewer만 갱신
research/20261031_vo-lo-study/.venv/bin/python research/20261031_vo-lo-study/src/run.py --stage playback

# ORB 상세 그림/수치 재현 (기존 orb/pyramid.npz 필요)
env MPLCONFIGDIR=.cache/matplotlib XDG_CACHE_HOME=.cache research/20261031_vo-lo-study/.venv/bin/python
research/20261101_orb-explained/src/analyze.py

# 통합 재생 검증
node research/20261031_vo-lo-study/src/check_playback.mjs

# 최신 PDF
node tools/render_report.mjs

# PDF 렌더 검증 결과 재확인
node tools/render_report.mjs --check-only
```

PDF 환경은 macOS Chrome, marked/playwright/mermaid다. 기본 의존성 경로는 /tmp/kitti-report-pdf/node_modules이고 REPORT_PDF_DEPS로 바꿀 수 있다. 설치 경로가 없으면 해당 패키지를 설치해야 한다. 다른 OS는 REPORT_CHROME으로 Chrome 실행 경로를 지정한다. 연구 Python 환경은 기존 requirements-lock.txt를 유지한다.

Git 작업 규칙

병합된 feature 브랜치에 다음 작업을 이어 붙이지 않는다. 최신 main에서 새 feat/ 브랜치를 만든다. pull 실패를 해결하려고 데이터나 작업 파일을 지우지 않는다. 고유한 로컬 커밋이 있는지 먼저 확인한다.

```
git fetch origin --prune
git switch main
git pull --ff-only origin main
git switch -c feat/<next-task>
```

이번 로컬 main의 divergence 원인은 이미 원격에 동일 내용으로 존재하는 초기 session-summary 커밋의 해시 차이였다. backup/main-before-sync-20261101을 만든 뒤 rebase로 중복을 제거하여 681582f와 일치시켰다. 이 방법을 내용이 다른 로컬 커밋에도 무조건 적용한다는 뜻은 아니다. 전역 pull 설정은 변경하지 않았다.

8. 버전 변경 이력

버전	갱신일	내용	알고리즘 변경
v2.0.0	2026-11-09	통합 문서를 학습 교재로 확장. 수학 기초 12단원, 24문제와 해설, 자체 생성 15그림, 실제 KITTI 단일 점 투영, 25개 계산 검사 및 PDF 점검 추가. 기존 응용 장 보존.	운영 VO/LO 변경 없음; 교육용 수치 실험 추가
v1.1.1	2026-11-03	로컬 폴더를 slam-lab으로 변경하고 두 가상환경의 실행 경로 및 현재 문서의 경로 안내를 수정.	없음
v1.1.0	2026-11-02	저장소를 slam-lab으로 변경. SLAM/오도메트리/융합의 관계와 top-down 수학 학습 순서, 좌표변환·MLE/MAP·융합 계산 예 추가. 로컬 작업 폴더는 유지.	없음
v1.0.0	2026-11-02	날짜별 5개 보고서의 상세 설명·실험·그림을 주제별 고정 문서로 통합. ORB는 최신 교정본 우선, 초기 감사는 역사적 기록으로 구분. 고정 PDF 빌드와 링크/다이어그램 검증 추가.	없음

과거 연구의 날짜·커밋·가정·미확인은 각 장에 보존했다. 추후 변경은 같은 파일과 Git 이력에 남긴다.